

Metodi avanzati di programmazione

Alessandro Carella

Indice

1	Astrazione nella progettazione	5
1.1	Astrazione funzionale	6
1.1.1	Specifiche semantiche	6
1.1.2	Limiti dell'astrazione funzionale	7
1.2	Astrazione dati	7
1.2.1	Information Hiding	7
1.2.2	Incapsulamento	8
1.3	Specifica sintattica e semantica	9
1.3.1	Specifiche assiomatiche	10
1.3.2	Specifiche algebriche	10
2	L'astrazione nella programmazione	16
2.1	Astrazione di funzione	17
2.1.1	Anomalie di progetto dei linguaggi	17
2.2	Astrazione di procedura	20
2.3	Astrazione di controllo	21
2.4	Astrazione di selettore	23
2.5	Astrazione dati	26
2.5.1	Tecniche di programmazione a supporto dell'astrazione dati	26
2.6	Astrazione nella dichiarazione di modulo	35
2.6.1	Vantaggi del tipo astratto	37
2.6.2	Svantaggi del tipo astratto	37
2.7	Astrazione generica	40
2.7.1	Generic package in Ada	41
2.7.2	Astrazione generica in C++	46

3	Paradigma Object Oriented	48
3.1	Gli oggetti	48
3.1.1	Identificatore di oggetto	49
3.2	Le classi	50
3.3	UML	51
3.3.1	Oggetti in UML	51
3.3.2	Le classi in UML	52
3.3.3	Gli stereotipi	55
3.3.4	Visibilità di attributi e metodi	55
3.3.5	Molteplicità di classe	56
3.3.6	Molteplicità di un attributo	57
3.3.7	Schema per la definizione di un'operazione	58
3.4	Classi attive	60
3.5	Classi template	60
3.6	Individuazione delle responsabilità	61
3.7	La relazione Instance-Of	62
3.8	Ereditarietà: una relazione fra classi	63
3.8.1	Ereditarietà per estensione	63
3.8.2	Ereditarietà per variazione funzionale	64
3.8.3	Principio di sostituibilità	65
3.8.4	Ereditarietà e relazione "is_a"	65
3.8.5	Ereditarietà di implementazione	65
3.8.6	Proprietà della relazione di generalizzazione	68
3.8.7	Ereditarietà singola	69
3.8.8	Ereditarietà multipla	69
3.8.9	Ereditarietà: la visibilità protetta	72
3.9	Classi astratte	73
3.10	Classi finali	75
3.11	Interfacce	76
3.11.1	Relazione di realizzazione	77
3.11.2	Ereditarietà	78
3.11.3	Realizzazione di più interfacce	79
3.11.4	Realizzazioni multiple di un interfaccia	80
3.12	Aggregazione di oggetti	80
3.12.1	Notazione in UML	81
3.12.2	Utilizzo dell'aggregazione	82
3.13	La composizione	82
3.13.1	Differenze tra ereditarietà e aggregazione	83

3.14	Raggruppare classi	85
3.15	Classi interne	88
3.16	Il polimorfismo	88
3.16.1	La coercizione	90
3.16.2	L'overloading	90
3.16.3	Il polimorfismo parametrico	92
3.16.4	Il polimorfismo di inclusione	93
4	Il linguaggio Java	97
4.1	Indipendenza dall'architettura	97
4.2	Interpretazione con elevate prestazioni	98
4.3	Ambienti di sviluppo: JDK	98
4.4	Oggetti in Java	99
4.5	Tipi primitivi	100
4.6	Numeri ad alta precisione	101
4.7	Variabili in Java	102
4.8	Creazione di nuovi tipi di dato	103
4.9	Overloading dei metodi	107
4.9.1	La parola chiave <code>this</code>	108
4.10	File e classi Java	109
4.10.1	Creazione dei package	109
4.10.2	Lo spazio dei nomi delle classi	110
4.10.3	Package e cartelle	111
4.11	I modificatori di accesso in Java	112
4.11.1	Il modificatore di accesso <code>friendly</code>	113
4.11.2	Il modificatore di accesso <code>public</code>	113
4.11.3	Il modificatore di accesso <code>private</code>	114
4.11.4	Il modificatore di accesso <code>protected</code>	115
4.11.5	Accesso alle classi	116
4.12	Le interfacce	117
4.13	Il riuso delle classi in Java	119
4.13.1	Il meccanismo di composizione	119
4.13.2	Il meccanismo di ereditarietà	121
4.14	La parola chiave <code>final</code>	126
4.14.1	<code>final</code> applicato ai dati	126
4.14.2	<code>final</code> applicato ai metodi	129
4.14.3	<code>final</code> applicato alle classi	129
4.15	Polimorfismo	130

4.15.1	Upcasting	130
4.15.2	Binding	131
4.15.3	Classi e metodi astratti	134
4.15.4	Costruttori e polimorfismo	135
4.15.5	Ereditarietà e finalizzazione	136
4.16	Passaggio di parametri e inizializzazione di oggetti	137
4.16.1	Clonazione di oggetti	138
4.16.2	Classi a sola lettura	140
4.16.3	Inizializzazione dei membri	143
4.17	Trattamento delle eccezioni	144
4.17.1	Il trattamento degli errori	144
4.17.2	Sollevare un'eccezione	145
4.17.3	Catturare un'eccezione	146
4.17.4	La parola chiave finally	148
4.17.5	La clausola throws	149
4.18	I contenitori	152
4.18.1	Array in Java	152
4.18.2	I contenitori	158
4.18.3	Iteratori	160
4.18.4	Funzionalità di List	161
4.18.5	Funzionalità di Set	162
4.18.6	Funzionalità di Map	163
4.19	RTTI - Run-Time Type Identification	166

Capitolo 1

Astrazione nella progettazione

In ambito scientifico la parola "astrarre" significa cambiare la rappresentazione di un problema e questo serve a concentrarsi su aspetti *rilevanti* dimenticando gli elementi *incidentali*.

Il termine astrazione sotto-intende:

- un **processo**: l'estrazione delle informazioni essenziali e rilevanti per un particolare scopo, ignorando il resto dell'informazione;
- un'**entità**: una descrizione semplificata di un sistema che enfatizza alcuni dei dettagli o proprietà trascurandone altri.

In programmazione, l'astrazione allude alla distinzione che si fa tra:

- *cosa* (*what*) un programma fa;
- *come* (*how*) viene implementato.

Per l'utente del codice l'essenziale è cosa fa il codice mentre non è interessato ai dettagli implementativi.

Nell'informatica l'astrazione è un concetto importante, in quanto, i sistemi software diventano sempre più complessi e per *padroneggiare la complessità* è necessario concentrarsi solo sui pochi aspetti che più interessano in un certo contesto e ignorarne altri.

L'astrazione permette ai progettisti di sistemi software di risolvere problemi complessi in modo organizzato e gestibile.

Esistono diverse tipologie di astrazione: l'*astrazione funzionale* e l'*astrazione dati*.

1.1 Astrazione funzionale

L'astrazione funzionale si riferisce alla *progettazione del software*, e in particolare alla possibilità di specificare un modulo software che **trasforma** dei dati di input in dati di output nascondendo i dettagli algoritmici della trasformazione. Sinteticamente, un modulo software deve trasformare un input in un output, cioè deve calcolare una funzione. I dettagli della trasformazione non sono visibili al consumatore del modulo, esso conosce solo le corrette convenzioni di chiamata, la *specifica sintattica*, e cosa fa il modulo, la *specifica semantica* e il consumatore deve fidarsi del risultato.

Un esempio di modulo software è il modulo che realizza un operatore per il calcolo del fattoriale. La *specifica sintattica* indica il nome del modulo, il tipo di dato in input e il tipo di risultato, in modo da permettere la corretta chiamata del modulo. Per la funzione del fattoriale:

$$fatt(intero) \rightarrow intero$$

La *specifica semantica* indica la trasformazione operata, cioè la funzione calcolata:

$$fatt(n) = \begin{cases} n(n-1) \cdots 2 \cdot 1 & n \geq 1 \\ 1 & n = 0 \end{cases}$$

Come la trasformazione è calcolata o realizzata non è noto al fruitore del modulo.

1.1.1 Specifiche semantiche

Un modo per specificare la semantica di un modulo è quello di esprimere, mediante due *predicati*, la relazione che lega i dati di ingresso e ai dati di uscita: se il primo predicato, detto *precondizione*, è vero sui dati di ingresso e se il programma termina su quei dati, allora il secondo, detto *postcondizione*, è vero sui dati di uscita.

Queste specifiche semantiche sono dette **assiomatiche**.

Nell'esempio del fattoriale:

- **Precondizione:** $n \in \mathbb{N}$
- **Postcondizione:** $fatt(n) = \begin{cases} n(n-1) \cdots 2 \cdot 1 & n \geq 1 \\ 1 & n = 0 \end{cases}$

Esercizio 1.1.1

Un modulo ha in input un intero x e un vettore $\mathbf{A}$ di n interi e restituisce un intero p e un vettore $\mathbf{B}$ di n interi.

Siano date le seguenti:

- **Precondizione:** $\{n > 0 \wedge \forall i \in [1, n] A[i] \in Z\}$
- **Postcondizione:** $\{\forall i \in [1, n] \exists j \in [1, n] B[i] = A[j] \wedge \forall i \in [1, p - 1] B[i] \leq x \wedge \forall i \in [p + 1, n] B[i] > x\}$

Questa è la specifica semantica del modulo dell'*x partition del quick sort*

1.1.2 Limiti dell'astrazione funzionale

L'astrazione funzionale non permette di progettare, e quindi sviluppare, moduli software *invarianti ai cambiamenti nei dati*, sono invariant solo ai cambiamenti nei processi di trasformazione che operano.

Questo rende difficoltosa la manutenzione delle soluzioni progettate ed è inappropriata per lo sviluppo di soluzioni a problemi complessi.

1.2 Astrazione dati

Alla base dell'*astrazione dati* c'è il principio che *non si può accedere direttamente alla rappresentazione di un dato, qualunque esso sia, ma solo attraverso un insieme di operazioni considerate lecite* e questo è detto **principio dell'astrazione dati**.

Un vantaggio dell'astrazione dati è che un cambiamento nella rappresentazione del dato si ripercuoterà solo sulle operazioni lecite, che potrebbero subire delle modifiche, mentre non inficerà il codice che utilizza il dato astratto.

1.2.1 Information Hiding

In generale un principio di astrazione suggerisce di *occultare l'informazione*, l'**information hiding**, sulla rappresentazione del dato sia perchè non necessaria al fruitore dell'entità astratta e sia perchè la sua rivelazione creerebbe delle inutili dipendenze che comprometterebbero l'invarianza ai cambiamenti.

Il principio dell'astrazione funzionale suggerisce di occultare i dettagli del processo di trasformazione. Il principio dell'astrazione dati identifica nella rappresentazione del dato l'informazione da nascondere.

1.2.2 Incapsulamento

L'**incapsulamento** è una tecnica di progettazione consistente nell'impacchettare una collezione di entità, creandone una barriera concettuale. Come l'astrazione, l'incapsulamento sottintende:

- un **processo**: l'impacchettamento;
- un'**entità**: il pacchetto ottenuto.

Ad essa corrispondono tecniche di programmazione che consentono l'incapsulamento dei dati.

L'incapsulamento non dice come devono essere le “pareti” del pacchetto, che potranno essere:

- **trasparenti**: permettendo di vedere tutto ciò che è stato impacchettato;
- **traslucide**: permettendo di vedere in modo parziale il contenuto;
- **opache**: nascondendo tutto il contenuto del pacchetto.

La combinazione del principio dell'astrazione dati con la tecnica dell'incapsulamento suggerisce che:

1. La rappresentazione del dato va nascosta;
2. L'accesso al dato deve passare solo attraverso operazioni lecite;
3. Le operazioni lecite, che ovviamente devono avere accesso all'informazione sulla rappresentazione del dato, vanno impacchettate con la rappresentazione del dato stesso.

L'astrazione dati ricalca ed estende quella funzionale. Attualmente la possibilità effettuare astrazione di dati è considerata importante almeno quanto quella di definire nuovi operatori con astrazioni funzionali. L'esperienza ha infatti dimostrato che la scelta delle strutture di dati è il primo passo sostanziale per un buon risultato dell'attività di programmazione.

L'astrazione funzionale stimola gli sforzi per evidenziare operazioni ricorrenti o comunque ben caratterizzate all'interno della soluzione di un dato problema.

L'astrazione di dati stimola in più gli sforzi per individuare l'organizzazione dei dati più consone alla soluzione del problema. Si è passati quindi da una progettazione *function centered* ad una *data centered*.

In generale, le astrazioni supportano la separazione dei diversi interessi di:

- **Utenti:** interessati a cosa si astrae (*what*);
- **Implementatori:** interessati a come (*how*) si realizza.

Per questa ragione una definizione di astrazione ha sempre due componenti:

- la **specificità**;
- la **realizzazione**.

Per descrivere una specificità occorre ricorrere a dei *linguaggi di specificità*, che sono diversi dai linguaggi usati per descrivere le realizzazioni delle astrazioni.

1.3 Specificità sintattica e semantica

La specificità potrà poi essere:

- **Sintattica:** stabilisce quali identificatori sono associati all'astrazione;
- **Semantica:** definisce il risultato della computazione inclusa nell'astrazione.

L'efficacia di un'astrazione può essere migliorata mediante l'uso di **parametri** per la comunicazione con l'ambiente esterno. Quando un'astrazione è chiamata, ciascun parametro formale verrà associato in qualche modo al corrispondente argomento.

Anche un'astrazione dati è costituita da una **specificità** e da una **realizzazione**:

- la **specificità** consente di descrivere un nuovo dato e gli operatori che ad esso sono applicabili;
- la **realizzazione** stabilisce come il nuovo dato e i nuovi operatori vengono ricondotti ai dati e agli operatori già disponibili.

Chi intende usare i nuovi dati con i nuovi operatori nella scrittura dei suoi programmi sarà tenuto a conoscere la specificità dell'astrazione dei dati, ma potrà astrarre dalle tecniche utilizzate per la realizzazione.

I linguaggi di specificità per astrazione dati più noti sono due:

1. il linguaggio logico-matematico usato nelle asserzioni, chiamate **specifiche assiomatiche**;

2. il linguaggio dell'algebra usato nelle equazioni definite fra gli operatori specificati nel dato astratto, chiamate **specifiche algebriche**.

1.3.1 Specifiche assiomatiche

Un linguaggio formale per la specifica di un dato astratto è costituito dalla notazione logico-matematica delle **asserzioni**. In questo caso si parla di specifica assiomatica.

Una specifica assiomatica è composta da:

- una specifica **sintattica**, detta *signature*, che fornisce:
 - l'elenco dei nomi dei domini e delle operazioni specifiche del tipo;
 - i domini di partenza e di arrivo per ogni nome di operatore.
- una specifica **semantica** che associa:
 - un insieme ad ogni tipo introdotto nella specifica sintattica;
 - una funzione ad ogni nome di operatore, esplicitando le condizioni sui domini di partenza e di arrivo:
 - * *precondizione*, che definisce quando l'operatore è applicabile;
 - * *postcondizione*, che stabilisce la relazione tra argomenti e risultato.

Limiti delle specifiche assiomatiche Si noti che il metodo di specifica assiomatica è preciso nella definizione della specifica sintattica, mentre è piuttosto informale per altri aspetti, infatti a volte si ricorre al linguaggio naturale per semplicità. Pertanto esso non consente di caratterizzare precisamente un dato astratto. In particolare, non consente di definire i valori che possono essere generati mediante l'applicazione di operatori, e non consente di stabilire quando l'applicazione di diverse sequenze di operatori porta al medesimo valore. Questo problema è superato dalle specifiche algebriche.

1.3.2 Specifica algebriche

Le specifiche algebriche si basano sull'algebra, piuttosto che sulla logica. Essenzialmente definiscono un dato astratto come un'**algebra eterogenea**, ovvero come una collezione di diversi insiemi su cui sono definite diverse operazioni.

Le algebre tradizionali sono omogenee. Un'algebra omogena consiste in un unico insieme e diverse operazioni.

Ad esempio, l'insieme $\mathbb{Z}$ con le operazioni di addizione e moltiplicazione è un'algebra omogenea.

Invece, se consideriamo un alfabeto Σ e consideriamo Σ^* l'insieme di tutte le stringhe, inclusa quella vuota, costruite con i simboli di Σ .

Σ^* con le operazioni di concatenazione e calcolo della lunghezza non sono un'algebra omogena, visto che il codominio dell'operazione di calcolo della lunghezza è $\mathbb{N}$ e non Σ^* . Quindi, quest'algebra consiste di due insiemi, stringhe e interi, su cui sono definite le operazioni di concatenamento e lunghezza delle stringhe. Una specifica algebrica consiste di tre parti:

1. **Sintattica:** elenca i nomi del tipo, le sue operazioni e il tipo degli argomenti delle operazioni. Se un'operazione è una funzione allora è specificato il codominio della funzione;
2. **Semantica:** consiste di un insieme di equazioni algebriche che descrivono in modo indipendente dalla rappresentazione le proprietà delle operazioni;
3. **Di restrizione:** stabilisce varie condizioni che devono essere soddisfatte o prima che siano applicate le operazioni o dopo che esse siano state completate.

A volte si possono trovare le specifiche di restrizione in quelle semantiche.

Uno degli aspetti più interessanti delle specifiche algebriche è la semplicità del linguaggio di specifica rispetto ai linguaggi di programmazione procedurale. Infatti, il linguaggio di specifica consiste di solo cinque primitive:

1. composizione funzionale;
2. relazione di equaglianza;
3. la costante *true*;
4. la costante *false*;
5. un numero illimitato di variabili libere.

La funzione *if then else* può essere facilmente descritta dalle seguenti equazioni:

$$if\ then\ else\ (true, q, r) = q$$

$$if\ then\ else\ (false, q, r) = r$$

Questa funzione è così importante che si assume già data come operatore infisso:

$$if\ then\ q\ else\ r$$

Inoltre si assume che sono predefiniti i valori interi e booleani

Esempio : Specifica algebrica di Pila

- **Specifica sintattica**

sorts: stack, item, boolean

operations:

- newstack() → stack
- push(stack, item) → stack
- pop(stack) → stack
- top(stack) → item
- isnew(stack) → boolean

Ogni insieme è detto un *sort* dell'algebra eterogenea. I *sort* item e boolean sono *ausiliari* alla definizione di stack.

- **Specifica semantica**

declare *stk*: stack, *i*: item

- pop(push(stk, i)) = stk
- top(push(stk, i)) = i
- isnew(newstack) = true
- isnew(push(stk, i)) = false

- **Specifica di restrizione**

- pop(newstack) = error
- top(newstack) = error

dove *error* è un elemento speciale indefinito.

Ovviamente avremmo potuto scrivere molte altre equazioni, come:

$$isnew(pop(push(newstack, i))) = true$$

che evidenzia come il predicato *isnew* sia vero anche per quelli stack ottenuti inserendo un generico elemento *i* in un **nuovo** stack e poi rimuovendolo.

Tuttavia questa equazione è ridondante, poichè è ricavabile da altre scritte in precedenza. Infatti, grazie alla prima e alla quarta equazione possiamo scrivere $isnewstack(pop(push(newstack, i))) = isnew(newstack) = true$.

Nelle specifiche semantiche è importante indicare l'insieme minimale di equazioni, dette *assiomi*, a partire dalle quali possiamo derivare tutte le altre. Le specifiche semantiche si diranno **incomplete** se non permetteranno di derivare tutte

le verità desiderate dell'algebra specificate.

Le specifiche semantiche si diranno **inconsistenti** se permetteranno di derivare delle equazioni indesiderate, cioè considerate false. Le specifiche semantiche si diranno **ridondanti** se alcune delle equazioni sono ricavabili dalle altre.

Scrivere delle specifiche semantiche complete, consistenti e non ridondanti può non essere un compito semplice. Per questo conviene introdurre una **metodologia**, che si basa sulla distinzione degli operatori di un dato astratto in:

- **costruttori**, che creano o istanziano il dato astratto;
- **osservazioni**, che ritrovano informazioni sul dato astratto.

Il comportamento di un'astrazione dati può essere specificata riportando il valore di ciascuna osservazione applicata a ciascun costruttore. Questa informazione è organizzata in modo naturale in una matrice, con i costruttori lungo una dimensione e le osservazioni lungo l'altra.

Un esempio di costruttori e osservazioni per la pila è il seguente.

<i>osservazioni</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>pop(stk')</i>	error	stk
<i>top(stk')</i>	error	i
<i>isnew(stk')</i>	true	false

Tutte le osservazioni viste finora sono unarie, nel senso che esse osservano un singolo valore del dato astratto, ma spesso, è necessario disporre di osservazioni più complesse. Per confrontare due valori è necessario osservare due istanze del dato astratto e questo complica la specifica, perchè il valore dell'osservazione dev'essere definito per tutte le **combinazioni di costruttori** possibili per i valori astratti che si devono confrontare.

Esempio : Predicato *equal*

Consideriamo il predicato *equal(l, m)* che è vero solo se le due pile contengono gli stessi elementi nello stesso ordine.

<i>Costruttore di m</i>	<i>Costruttore di l</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>newstack</i>	true	false
<i>push(stk', i')</i>	false	i=i' and equal(stk,stk')

Questa tabella può essere vista come l'aggiunta di una terza dimensione alla tabella di base per le osservazioni unarie.

Questa terza dimensione può essere rimossa andando a classificare esplicitamente solo il primo argomento dell'osservazione e referenziando in modo astratto il secondo argomento mediante una *variabile libera*.

<i>osservazione</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>equal(stk', m)</i>	isnew(m)	not isnew(m) and i=top(m) and equal(stk,pop(m))

Esempio : Specifica algebrica della coda

- **Specifica sintattica**

sorts: queue, item, boolean

operations:

- newq() → queue
- addq(queue, item) → queue
- deleteq(queue) → queue
- frontq(queue) → item
- isnewq(queue) → boolean

- **Specifica semantica**

declare *q*: queue, *i*: item

- deleteq(addq(q, i)) = if isnewq(q) then newq else addq(deleteq(q), i)
- frontq(addq(q, i)) = if isnewq(q) then i else frontq(q)
- isnew(newq) = true
- isnew(addq(q, i)) = false

- **Specifica di restrizione**
 - $\text{deleteq}(\text{newq}) = \text{error}$
 - $\text{frontq}(\text{newq}) = \text{error}$

Se volessi rappresentare costruttori e osservazioni mediante una tabella, il risultato è il seguente.

<i>osservazioni</i>	<i>Costruttore di q'</i>	
	<i>newq</i>	<i>addq(q, i)</i>
<i>isnewq(q')</i>	true	false
<i>frontq(q')</i>	error	if <i>isnewq(q)</i> then <i>i</i> else <i>frontq(q)</i>
<i>deleteq(q')</i>	error	if <i>isnewq(q)</i> then <i>newq</i> else <i>addq(deleteq(q), i)</i>

Capitolo 2

L'astrazione nella programmazione

È possibile costruire astrazioni su una qualunque classe sintattica, purchè le frasi di quella classe specifichino un qualche tipo di computazione.

Di seguito si esaminerà l'applicazione del principio di astrazione in sei classi sintattiche, in particolare:

1. Espressione → **astrazione di funzione**;
2. Comando → **astrazione di procedura**;
3. Controllo di sequenza → **astrazione di controllo**;
4. Accesso a un'area di memoria → **astrazione di selettore**;
5. Definizione di un dato → **astrazione di tipo**;
6. Dichiarazione → **astrazione generica**.

Tutte queste classi sintattiche sottointendono una computazione, infatti:

- L'astrazione di *funzione* include un'espressione da valutare.
- L'astrazione di una *procedura* include un comando da seguire.
- L'astrazione di *controllo* include delle espressioni di controllo dell'ordine di esecuzione delle istruzioni.
- L'astrazione di *selettore* include il calcolo dell'accesso ad una variabile.

- L'astrazione di *tipo* include un gruppo di operatori che definiscono implicitamente un insieme di valori.
- L'astrazione *generica* include una frase che sarà elaborata per produrre legami.

2.1 Astrazione di funzione

Un'astrazione di funzioni include un'espressione da valutare, e quando chiamata, darà un valore come risultato.

Un'astrazione di funzione è specificata mediante una definizione di funzione del tipo:

function $I(FP_1; \dots; FP_n)$ is E

dove I è un identificatore, $FP_1; \dots; FP_n$ sono parametri formali ed E è l'espressione da valutare.

In questo modo si lega I ad un'entità, l'astrazione di funzione, che ha la proprietà di dare un risultato ogni qualvolta che è chiamata con argomenti appropriati.

Una chiamata di funzione, $I(AP_1; \dots; AP_n)$ dove i parametri effettivi, $AP_1; \dots; AP_n$ determinano gli argomenti, ha due punti di vista:

1. Punto di vista dell'*utente*: la chiamata di una funzione trasforma gli argomenti in un risultato;
2. Punto di vista dell'*implementatore*: la chiamata valuta E , avendo precedentemente vincolato i parametri formali agli argomenti corrispondenti.

L'algoritmo codificato in E è di interesse per l'implementatore.

2.1.1 Anomalie di progetto dei linguaggi

Se l'astrazione di funzione include un'espressione E da valutare, è naturale attendersi che il corpo della funzione non includa dei comandi, come assegnazioni, istruzioni di salto, iterazioni, e altro, il cui effetto è quello di cambiare lo stato di un sistema, non di produrre valori.

Non sempre ciò accade, come si può osservare in questo esempio di funzione scritto in Pascal.

```

1  function power(x: real; n: integer) : real
2  begin
```

```

3   if n = 1 then power := x
4   else power := x * power(x, n - 1)
5   end
6

```

Nel corpo di questa funzione compaiono i comandi di assegnazione. Non è proprio possibile farne a meno, perchè per poter restituire un valore, in Pascal è necessario assegnare un valore ad una pseudo variabile, che ha lo stesso nome della funzione.

Il corpo della funzione è sintatticamente un comando, ma semanticamente è un'espressione, questa funzione, infatti, può essere invocata solo alla destra di operazioni di assegnazione.

Non tutti i linguaggi di programmazione includono comandi nel corpo di funzioni, infatti, nel linguaggio funzionale ML la funzione vista prima diventa:

```

1   function power(x: real; n: int) is
2   if n = 1 then x
3   else x * power(x, n-1)
4

```

Nel corpo della funzione definita in ML compare un'espressione condizionale la cui valutazione non modifica lo stato del sistema. Il motivo per il quale altri linguaggi di programmazione, come Pascal, Ada, o altro, permettono di avere comandi nel corpo di funzione è quello di poter sfruttare la potenza espressiva dell'assegnazione e dell'*iterazione* nella computazione di risultati. Diversamente saremmo costretti a esprimere ricorsivamente le espressioni da valutare. E la ricorsione, com'è noto, può essere fonte di inefficienze nell'uso delle risorse di calcolo.

Quindi, per ragioni di efficienza, molti linguaggi permettono di avere comandi nel corpo di funzioni e si lascia al programmatore il compito di utilizzare correttamente i comandi in modo da evitare che la funzione possa avere effetti collaterali, *side effect*, oltre quello di calcolare un valore.

Le funzioni in quanto astrazioni di espressioni, possono comparire ovunque occorra un'espressione. Pertanto, esse compaiono alla destra di operazioni di assegnazioni, ma anche al posto di parametri effettivi nelle chiamate di altre funzioni, laddove il valore potrebbe essere calcolato mediante un'espressione.

$$y := f(y, \text{power}(x, 2))$$

In molti linguaggi di programmazione è possibile definire dei parametri formali che sono riferimento ad una funzione.

```

1  function sommatoria(function F(R: real; M: integer): real; X: real; N:
integer) : real;
2      var I: integer;
3      var Sum: real;
4      begin
5          Sum := 0;
6          for I := 1 to N do Sum := Sum + F(x, I);
7          Sommatoria := Sum
8      end
9

```

Se la funzione F è la funzione *power* definita in precedenza, allora:

$$Y := Sommatoria(power, x, n)$$

calcola la sommatoria $\sum_{i=1}^n x^i$.

In alcuni linguaggi si separa il concetto di astrazione di funzione da quello di legame a un identificatore. Questo permette di passare come parametri delle funzioni anonime. Infine, le funzioni possono essere il risultato di valutazioni di espressioni o possono essere assegnate a variabili:

```

1  val cube = fn(x: real) => x * x * x

```

Riepilogando, in alcuni linguaggi di programmazione, le funzioni sono di *terza classe*, cioè possono essere solo chiamate, in altri sono di *seconda classe*, cioè possono essere passate come argomenti, mentre in altri linguaggi di programmazione esse sono di *prima classe* in quanto possono essere anche restituite come risultato della chiamata di altre funzioni o possono essere assegnate come valore a una variabile.

Cittadini di prima classe In generale, in programmazione una qualunque entità si dice *cittadino di prima classe* quando non è soggetta a restrizioni nel suo utilizzo. A tal proposito, osserviamo che i valori possono essere:

- *Denotabili*, se possono essere associati ad un nome;
- *Esprimibili*, se possono essere il risultato di un'espressione complessa;
- *Memorizzabili*, se possono essere memorizzati in una variabile.

2.2 Astrazione di procedura

Un'astrazione di procedura include un comando da eseguire, e quando chiamata, aggiornerà le variabili che rappresentano lo stato del sistema. Un'astrazione di procedura è specificata mediante una definizione di procedura, del tipo:

$$\text{procedure } I(\text{FP}_1; \dots; \text{FP}_n) \text{ is } C$$

dove I è un identificatore, $\text{FP}_1; \dots; \text{FP}_n$ sono parametri formali, e C è il blocco di comandi da eseguire.

In questo modo si lega I all'astrazione di procedura, che gode della proprietà di cambiare lo stato del sistema quando chiamata con argomenti appropriati.

Data una chiamata di procedura, $I(\text{AP}_1; \dots; \text{AP}_n)$ dove $\text{AP}_1; \dots; \text{AP}_n$ sono i parametri effettivi, il punto di vista dell'*utente* è che la chiamata aggiornerà lo stato del sistema in modo dipendente dai parametri, mentre il punto di vista dell'*implementatore* è che la chiamata consentirà l'esecuzione del corpo di procedura C , avendo precedentemente vincolato i parametri formali agli argomenti corrispondenti. L'algoritmo in C è di interesse solo per l'implementatore.

In Pascal e in C le procedure sono *cittadini di seconda classe*.

L'astrazione di funzione o di procedura sono due tecniche di programmazione di supporto all'astrazione funzionale, intesa come tecnica di progettazione del software seconda la quale occorre distinguere la specifica di un'operatore dalla sua realizzazione.

Quale sia la tecnica di programmazione, se di funzione o di procedura, più opportuna da adottare dipende da:

- **Tipo di operatore progettato:**

- ha effetti collaterali, come accade nell'astrazione di procedura;
- non ha effetti collaterali, come accade nell'astrazione di funzione.

- **Limiti imposti dal linguaggio di programmazione.**

Ad esempio, l'operatore di *bubblesort* non modifica l'array passato in ingresso, ma ne restituisce uno ordinato, pertanto sarebbe più opportuno implementarlo ricorrendo ad un'astrazione di funzione. Tuttavia, non tutti i linguaggi di programmazione consentono di restituire come un tipo di dato complesso e quindi l'operatore *bubblesort* dovrà essere realizzato mediante un'astrazione di procedura.

2.3 Astrazione di controllo

L'astrazione di controllo si applica alla classe sintattica struttura di controllo. Le strutture di controllo definiscono l'ordine in cui le singole istruzioni o i gruppi di istruzioni devono essere eseguiti.

Il linguaggio macchina generalmente fornisce due semplici meccanismi per governare il flusso di controllo delle istruzioni singole: l'**elaborazione in sequenza** e il **salto**.

Nei linguaggi assemblativi le istruzioni da eseguire in sequenza sono scritte l'una dopo l'altra: il salto è rappresentato da un'istruzione di jump:

jump to <indirizzo simbolico o label>

L'indirizzo simbolico è comunque un dettaglio di scarsa importanza per il programmatore: quello che conta è poter indicare le prossime istruzioni da eseguire. Per questa ragione i linguaggi di alto livello hanno introdotto strutture di controllo astratte come la **selezione**:

```
1  if cond then S1
2  else S2
```

corrisponde a:

```
1  jump on <cond> to A
2  S2
3  jump to B
4  A: S1
5  B: ...
```

Similmente sono state introdotte diverse strutture di controllo astratte per l'**itereazione**.

Inoltre per conservare gli indirizzi di ritorno dalle chiamate di funzione/procedura si è tradotta nella possibilità di effettuare chiamate **ricorsive**.

Di particolare interesse è l'attuale tendenza a offrire strutture di controllo iterative per quei dati astratti che sono collezioni omogenee, come insiemi, liste, array, ecc., di valori.

La struttura di controllo iterativa ha due parametri, il dato astratto e la variabile alla quale assegnare un valore contenuto nel dato astratto. Il meccanismo di visita della collezione di valori è incapsulato nel dato astratto, che dispone di operazioni lecite per consentire l'iterazione sulla collezione di oggetti.

Esempio : For-each in java

In Java è disponibile l'astrazione di controllo *for-each*, che permette di iterare su una collezione.

```
1      LinkedList<Integer> list = new LinkedList<Integer>();
2      ... // Inserimento elementi
3      for (Integer n : list) {
4          System.out.println(n);
5      }
```

L'operazione messa a disposizione di *LinkedList* per l'astrazione di controllo *for-each* è **iterator()**.

Se guardiamo alle strutture di controllo come espressioni che, quando valutate, definiscono l'ordine in cui eseguire dei comandi, possiamo specificare un'astrazione di controllo come segue:

$$\text{control } I(FP_1; \dots; FP_n) \text{ is } S$$

dove I è un identificatore di una nuova struttura di controllo, $FP_1; \dots; FP_n$ sono parametri formali, e S è un'espressione di controllo che definisce un ordine di esecuzione.

Esempio : Swap

```
1      control swap(boolean: cond, statement: S1, S2) is
2          if cond then
3              begin S1; S2 end
4          else
5              S2; S1
6          endif
```

Esempio : Alt

```
1      control alt_execution(statement: S1, S2, S3) is
2          if abnormal(S1) then
3              S2
4          else
5              S3
6          endif
```

abnormal è un predicato che ha in input un comando $S1$ e restituisce *true* se l'esecuzione di $S1$ è terminata in modo anomalo, *false* altrimenti.

I linguaggi di programmazione moderni mettono a disposizione dei meccanismi sofisticati per gestire le situazioni eccezionali come:

- errori aritmetici o di I/O;
- fallimento di precondizioni;
- condizioni imprecidibili.

Quando viene sollevata un'eccezione, essa dev'essere catturata e gestita. Tipici approcci per localizzare il gestore dell'eccezione sono:

- cerca un gestore nel blocco (o sottoprogramma) corrente;
- se non c'è, forza l'uscita dell'unità corrente e solleva l'eccezione nell'unità chiamante;
- continua a risalire la gerarchia delle unità chiamanti finchè non trovi un gestore oppure non raggiungi il livello più alto, il main.

Quando viene trovato il gestore e l'eccezione è trattata:

- si riparte dall'unità contenente il gestore, *modello di terminazione*, come in Ada;
- si ritorna ad eseguire il comando che ha generato l'eccezione, *modello di ripresa*, come in PL/I.

I linguaggi di programmazione sequenziale utilizzano la sequenza, la selezione e la ripetizione per definire un **ordinamento totale** sull'esecuzione dei comandi di un programma.

I linguaggi di programmazione paralleli utilizzano ulteriori costrutti del flusso di controllo, come *fork*, *begin* o cicli *for* paralleli, per introdurre in modo esplicito un *ordinamento parziale* sull'esecuzione dei comandi.

Dato che il parallelismo è una forma di controllo della sequenza di esecuzione, l'astrazione di controllo è particolarmente importante nella programmazione parallela.

2.4 Astrazione di selettore

Un linguaggio di programmazione dispone di costrutti per poter accedere ad una variabile. Ad esempio in Pascal abbiamo:

```

1  var r: real;
2  r := r * r;
3  var vett: array[1..10] of char;
```

```

4   vett[i] := vett[i+1];
5

```

Lo stesso identificatore può essere usato sia come nome di un valore (legame dinamico fra identificatore e valore) e sia come un indirizzo (legame statico fra identificatore e locazione di memoria). Identicamente potremmo avere:

```

1   type rec = record
2     a: real,
3     b: real
4   end
5   var r: rec;
6   ...
7   r.b := r.a * 3;

```

Un accesso a una variabile restituisce un *riferimento* ad una variabile. Tuttavia se osserviamo il seguente codice Pascal:

```

1   type queue = ...;
2   var Aq: queue;
3   function first(q: queue): integer
4     // Restituisce il primo elemento della coda
5   i := first(Aq)

```

Ci accorgiamo che la chiamata *first(Aq)* può comparire solo alla destra di un'assegnazione, perchè le funzioni restituiscono solo valori, mentre a sinistra *first(Aq) := 0* dovrebbe restituire riferimenti ad aree di memoria.

In Pascal abbiamo dei selettori predefiniti dal progettista del linguaggio:

- $F^{\wedge}$ è un riferimento ad un puntatore di F ;
- $V[E]$ è un riferimento ad un elemento del vettore di V ;
- $R.A$ è un riferimento ad un elemento di un record.

ma il programmatore non ha modo di definire un nuovo selettore. In altri termini, non c'è la possibilità di definire astrazioni di selettore, che restituiscano l'accesso a un'area di memoria.

In C++ invece, possiamo definire astrazioni di selettore, infatti:

```

1   class Queue {
2     ...
3   public:
4     static int& first(Queue*);
5   };

```

```

6
7     Queue q*;
8     ...
9     int i = first(q);
10    first(q) *= first(q);
11    first(q)++;

```

Una possibile implementazione è la seguente:

```

1     class Queue {
2         int items[100];
3         int front, rear;
4         public:
5             ...
6             int& first(Queue* q) { return q->items[front] };
7     };

```

Flessibilità dei linguaggi L'espressività di un linguaggio di programmazione dipende direttamente da:

- I *meccanismi di composizione*, ossia la definizione di operazioni complesse a partire da quelle semplici.
- I *meccanismi di controllo di sequenza*, ossia la possibilità di stabilire ordini di esecuzione in modo semplice.
- I *valori* che il linguaggio permette di rappresentare e manipolare dati.

I linguaggi che supportano:

- L'astrazione funzionale sono flessibili nei meccanismi di composizione;
- L'astrazione di controllo sono flessibili nei meccanismi di controllo di sequenza;
- L'astrazione dati sono flessibili nella definizione e manipolazione di nuovi valori.

2.5 Astrazione dati

2.5.1 Tecniche di programmazione a supporto dell'astrazione dati

Le tecniche di programmazione a supporto dell'astrazione dati, che è una tecnica di progettazione del software, sono due:

- definizione di **tipi astratti**, cioè l'astrazione sulla classe sintattica tipo;
- definizione di *classi di oggetti*, cioè l'astrazione sulla dichiarazione di moduli dotati di stato locale.

In entrambi i casi occorre poter *incapsulare* la rappresentazione del dato con le operazioni lecite. Tuttavia,

- nel tipo di dato astratto, il modulo rende visibile all'utilizzatore sia un identificatore di tipo che degli operatori;
- nelle classi di oggetti, il modulo rende visibile all'utilizzatore solo gli operatori

Inoltre:

- nel tipo astratto T , i valori sono associati a variabili dichiarate di tipo T ;
- nella classe di oggetti C , i valori sono associati a oggetti ottenuti per istanziazione della classe di oggetti C .

Tipo concreto e tipo astratto I linguaggi ad alto livello mettono a disposizione del programmatore un nutrito gruppo di tipi predefiniti, detti **concreti**. Essi si distinguono in *primitivi* o *semplici* (i valori associati al tipo sono atomici) e *composti* o *strutturati* (i cui valori sono ottenuti per composizione di valori più semplici).

Tuttavia un linguaggio di programmazione sarà tanto più espressivo quanto più semplice sarà per il programmatore definire dei suoi tipi di dato a partire dai tipi di dato concreti disponibili. I tipi definiti dall'utente, detti *user defined types* UDT, sono anche detti **astratti**.

È bene ricordare che i tipi concreti sono messi a disposizione del linguaggio e tipi astratti sono definiti dall'utente. Ritornando all'astrazione di tipo, l'*espressione di tipo*, detto anche *tipo*, è il costrutto con cui alcuni linguaggi di programmazione consentono di definire un nuovo tipo. Ad esempio, in Pascal:

```

1  type Person = record
2      name: packed array[1..20] of char;
3      age: integer;
4      height: real
5  end

```

In questo caso si stabilisce esplicitamente una rappresentazione per i valori del tipo *Person* e implicitamente gli operatori applicabili a valori di quel tipo. Gli operatori del tipo *Person*, dovranno essere generici e non è possibile specificarli. Una **astrazione di tipo**, o *tipo astratto di dato* o semplicemente *dato astratto*, ha un corpo costituito da una espressione di tipo. Quando è valutata, l'astrazione di tipo stabilisce sia una rappresentazione per un insieme di valori e sia le operazioni ad esse applicabili.

Analogamente a quanto detto per le altre astrazioni, l'astrazione di tipo potrà essere specificata come segue:

$$\text{type } I(FP_1; \dots; FP_n) \text{ is } T$$

dove I è un identificatore del nuovo tipo, $FP_1; \dots; FP_n$ sono parametri formali e T è un'espressione di tipo che specificherà la rappresentazione dei tipi di dato I e le operazioni ad esso applicabili. Analizziamo le astrazioni di tipo che alcuni linguaggi di programmazione (come C e Pascal) consentono di definire.

```

1  type complex = record
2      Re: real;
3      Im: real
4  end

```

consente di:

1. stabilire che *complex* è un identificatore di tipo;
2. associare una rappresentazione a *complex* espressa mediante tipi concreti già disponibili nel linguaggio.

Le operazioni associate al nuovo tipo *complex* sono tutte quelle che il linguaggio ha già previsto epr il tipo record, come assegnazione e selezione di campi.

I limiti di questa astrazione sono:

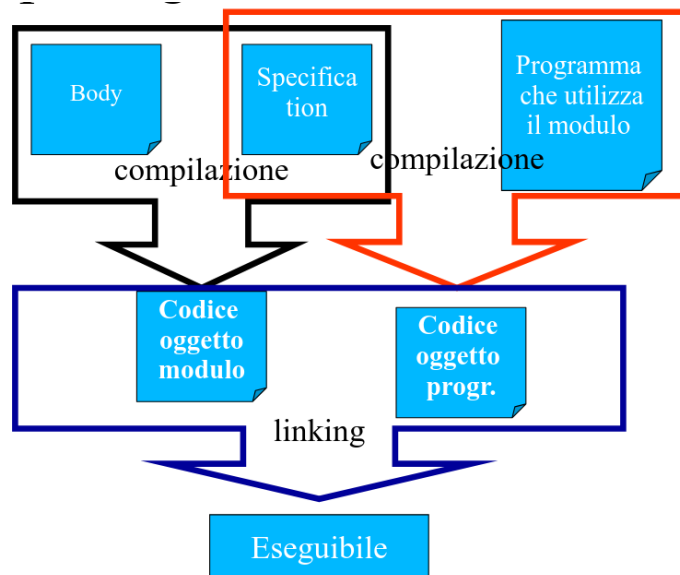
1. Il programmatore non può definire nuovi operatori specifici da associare al tipo.

2. Violazione del requisito di protezione: l'utilizzatore è consapevole della rappresentazione del tipo *complex*, sa che è un record e quali sono i campi, ed è in grado di operare mediante operatori non specifici del dato.
3. L'astrazione di tipo non è parametrizzata, la comunicazione con il contesto esterno non è ammessa.

Quindi, abbiamo detto che il programmatore non può definire nuovi operatori specifici da associare al tipo. Questo problema può essere risolto mediante un costrutto di programmazione che permette di **incapsulare**:

1. le rappresentazioni del dato;
2. gli operatori leciti.

Questo costrutto di programmazione è il **package**, ovvero *un gruppo di componenti dichiarate, come tipi, costanti, variabili, funzioni e persino sotto moduli*. Introduciamo quindi un costrutto package per supportare l'incapsulamento.



In Ada il modulo è chiamato package e si compone di due parti:

1. una **specification**, che interagisce con l'ambiente esterno: contiene le dichiarazioni di tipi, costanti, procedure e funzioni;

2. un **body**, che contiene, fra l'altro, l'implementazione di procedure e funzioni dichiarate nella *specification*, ed eventualmente una routine di inizializzazione del package.

A sua volta la *specification* si articola in due sottoparti:

1. **visible**: le entità dichiarate in questa parte possono essere rese note ad altre unità di programma per mezzo della clausola *use*;
2. **private**: le entità dichiarate in questa parte non possono essere né esportate e né dichiarate nel corpo.

La parte di specifica inizia con la parola chiave **package** seguita dall'identificatore del package e da **is**; seguono poi le dichiarazioni delle entità visibili e private.

```
1 package Type_complex is
2   type Complex is
3     record
4       RL, IM: Real;
5     end record;
6   I: constant Complex := (0.0, 1.0)
7   function "+"(x, y: Complex) return Complex
8   ...
9 end Type_complex;
```

Questo è un esempio di specifica di un package che realizza il tipo astratto Complex. In questo non c'è una parte privata nella specifica, sicché un programma che fa uso di questo package può operare su variabili complesse mediante espressioni come la seguente:

$$C.IM := C.IM + 1.0$$

invece di ricorrere alla forma più astratta:

$$C := C + 1;$$

Per evitare il problema possiamo nascondere la struttura del tipo Complex nella parte privata.

```
1 package Type_complex is
2   type Complex is private;
3   function "+"(x, y: Complex) return Complex;
4   ...
5 private
6   type Complex is record
```

```

7         RL, IM: Real;
8     end record;
9     I: constant Complex := (0.0, 1.0)
10 end Type_complex;

```

Avendo dichiarato il tipo Complex nella parte privata non è più possibile inizializzare la costante I nella parte pubblica, in quanto non è ancora nota la rappresentazione di Complex.

Anche il corpo del package incomincia con la parola package seguita però dalla parola body, dall'identificatore e da is. Nel corpo sono fornite le implementazioni delle procedure e funzioni dichiarate nella corrispondente specification.

```

1     package body Type_complex is
2         function "+"(x, y: in Complex) return Complex is
3             begin
4                 return (x.RL + y.RL, x.IM + y.IM);
5             end "+";
6             ...
7     end Type_complex;

```

Ovviamente la struttura del tipo Complex è visibile alla parte body del package, quindi si potrà accedere ai campi RL e IM dei record Complex. Il package potrà essere utilizzato come segue:

```

1     with Type_complex; use Type_complex;
2     procedure main is
3         cpx1, cpx2: Complex;
4         begin
5             ...
6             cpx1 := cpx2 + I;
7             ...
8     end main;

```

Si osservi che sia nella specifica e sia nel corpo di Type_complex non c'è alcuna dichiarazione di variabili esterne a procedure e funzioni. Ciò vuol dire che questo package non è dotato di uno stato locale, cioè non definisce un oggetto.

Per questa ragione il corpo del package non necessita di un "main": non si deve inizializzare un oggetto. Il package può in ogni caso avere un proprio main. Esso verrà specificato dopo le varie procedure e funzioni e sarà compreso fra un begin e l'end del package.

```

1     with Simple_io; use Simple_io;
2     package body Type_complex is

```

```

3      function "+"(x, y: in Complex) return Complex is
4      begin
5          return(x.RL + y.RL, x.IM + y.IM);
6      end "+";
7      ...
8      begin
9          put("Main of package Type_complex");
10     end Type_complex;

```

L'esecuzione del main del package avverrà al momento in cui si importa il package mediante la clausola use. Si vedrà quindi visualizzare la frase "Main of package Type_complex".

Esempio : Astrazione di tipo per realizzare il dato astratto Pila

La specification è la seguente:

```

1      package Type_stack is
2      type Stack is private
3      procedure push(s: in out Stack; x: in Integer);
4      procedure pop(s: in out Stack);
5      procedure top(s: in Stack; x: out Integer);
6      function empty(s: in Stack) return Boolean;
7      private
8      max: constant := 100;
9      type Stack is limited record
10         st: array(1..max) of Integer;
11         top: Integer range 0..max := 0;
12     end record;
13 end Type_stack;

```

Il body è il seguente:

```

1      procedure body Type_stack is
2      procedure push(s: in out Stack; x: in Integer) is
3      begin
4          s.top := s.top + 1;
5          s.st(s.top) := x;
6      end push;
7      procedure pop(s: in out Stack) is
8      begin
9          s.top := s.top - 1;
10     end pop;
11     procedure top(s: in Stack; x: out Integer) is
12     begin
13         x := s.st(s.top);
14     end top;

```

```

15         function empty(s: in Stack) return Boolean is;
16             begin
17                 return(s.top = 0);
18             end empty;
19         end Type_stack;

```

Le specifiche del tipo astratto Pila prevedono anche un costruttore CreaPila, che non ha un corrispondente in questa definizione del tipo astratto Stack. La ragione è che si stanno utilizzando costruttori impliciti forniti nel linguaggio Ada, come la dichiarazione di una variabile di tipo Stack. Tuttavia se un costruttore dovesse essere parametrizzato sarà necessario prevedere un metodo.

In Ada, definendo un tipo come *private* è possibile applicare su istanze di quel tipo tutti i metodi definiti nella parte pubblica della specifica, ma anche effettuare assegnazioni e confronti di (dis-)eguaglianza. Queste operazioni che il compilatore offre gratuitamente per un tipo privato devono necessariamente essere definite in modo generale, indipendentemente da come il tipo è poi definito.

Quindi saranno implementate semplicemente copiando o confrontando byte a byte le aree di memoria riservate a due dati dello stesso tipo dichiarato come privato. Questo, però, comporta dei problemi.

1. Le operazioni di assegnazione, confronto per eguaglianza e confronto per diseguaglianza, potrebbero non far parte della specifica di un dato astratto. Quindi potrebbe non essere corretto offrirle all'utilizzatore del tipo dichiarato come privato.
2. La semantica delle operazioni potrebbe essere diversa da quella stabilita dal compilatore.

Ad esempio, consideriamo il seguente codice:

```

1         with Type_stack; use Type_stack;
2         procedure main is
3             st1, st2: Stack;
4             cmp: Boolean;
5         begin
6             push(st1, 1);
7             push(st2, 1);
8             push(st1, 2);
9             pop(st1);
10            cmp := st1 = st2;
11        end main;

```

Il valore di `cmp` è `False`. Infatti i corrispondenti `top` dei record `st1` e `st2` sarebbero identici, mentre non avrebbero gli stessi valori i corrispondenti campi `st`. Eppure i due stack sarebbero identici secondo la specifica algebrica di $equal(l, m)$ data nel progetto.

Per evitare tutto ciò, è sufficiente dichiarare il tipo come **limited private**, che inibisce l'uso delle operazioni di assegnazione e confronto offerte per default dal compilatore.

```

1      package Type_stack is
2          type Stack is limited private
3          procedure push(s: in out Stack; x: in Integer);
4          procedure pop(s: in out Stack);
5          procedure top(s: in Stack; x: out Integer);
6          function empty(s: in Stack) return Boolean;
7          ...

```

In Ada è anche possibile definire **oggetti**, cioè moduli dotati di stato locale.

Esempio : Oggetto stack

La specifica è la seguente.

```

1      package Stack is
2          procedure push(x: in Integer);
3          procedure pop;
4          procedure top(x: out Integer);
5          function empty return Boolean;
6      end Stack;

```

Il body è il seguente.

```

1      package body Stack is
2          max: constant := 100;
3          type Table is array(1..max) of Integer;
4          st: Table;
5          top: Integer range 0..max := 0;
6
7          procedure push(x: in Integer) is
8              begin
9                  top := top + 1;
10                 st(top) := x;
11             end push;
12
13             procedure pop is
14                 begin
15                     top := top + 1;
16             end pop;

```

```

17
18     procedure top(x: out Integer) is
19     begin
20         x := st(top);
21     end top;
22
23     function empty return Boolean is
24     begin
25         return (top = 0);
26     end empty;
27 end Stack;

```

Si potrà quindi utilizzare l'oggetto Stack nel modo seguente:

```

1     with Stack; use Stack;
2     procedure main is
3     begin
4         ...
5         push(1);
6         push(2);
7         pop
8         if empty then push(1);
9         ...
10    end main;

```

In generale, un oggetto è un insieme di variabili interne ad un modulo e manipolabili esternamente solo mediante gli operatori definiti nel modulo stesso. Da quanto visto finora, per poter definire più oggetti “simili”, cioè con medesima rappresentazione e stesso insieme di operatori, si è costretti a definire tanti moduli quanti sono gli oggetti che si vogliono usare nel programma. Tutti questi moduli differiranno solo per l'identificatore del modulo, cioè l'**identificatore dell'oggetto**.

Per evitare l'inconveniente di dover duplicare un modulo si può pensare di definire un **package generico** che identifica una *classe* di oggetti simili. I singoli oggetti sono poi ottenuti con il meccanismo di *istanziamento* della classe.

In Ada un package che specifica e implementa un singolo oggetto può essere facilmente trasformato in un *generic package*, che definisce una classe di oggetti, permettendo la parola *generic* alla dichiarazione del modulo.

```

1     generic
2     package Stack
3     procedure push(x: in Integer);

```

In questo modulo si definisce solo una *matrice* degli oggetti da creare. Per otte-

nere i singoli oggetti dobbiamo *istanziare* il generic package:

```
1 package st1 is new Stack
2 package st2 is new Stack
```

Queste due dichiarazioni sono processate in *fase di precompilazione*. In particolare, per ogni occorrenza di istanziazione, si sostituisce la stringa dell'istanziamento con il comando di importazione di un package avente come nome quello dell'istanza, ad esempio with ST1; use ST1.

Si genera un package non generico utilizzando il generic package come matrice. Esso si ottiene rimuovendo la parola *generic* e sostituendo il nome dell'istanza al posto del nome del package. Il package è poi compilato separatamente.

Nell'esempio specifico, la precompilazione genera due package distinti, che variano solo nel nome del package, cioè nell'*identificatore dell'oggetto*.

Nota : Osservazione sul generic package

Di seguito alcune osservazioni sul generic package.

1. Il package generico **non è compilabile separatamente** in quanto il nome del package non identifica un oggetto.
2. La generazione di package distinti, uno per ogni istanziazione, comporta la creazione di molteplici copie dello stesso codice e questo porta ad una **inefficienza di spazio**.
3. La creazione dei legami al compile-time garantisce **efficienza in tempo**, poichè non è necessario effettuare computazioni di legami al run-time.
4. L'ambiguità dovuta alle molteplici occorrenze di metodi con stesso identificatore richiede il ricorso, nel programma utilizzatore, alla notazione <identificatore di oggetto>.<nome metodo>, ad esempio ST1.push(10)

2.6 Astrazione nella dichiarazione di modulo

La precedente definizione di una classe corrisponde a una particolare forma di astrazione, quella della classe sintattica '**dichiarazione di un modulo**'.

L'operazione di istanziazione corrisponde alla invocazione di questa astrazione ed ha l'effetto di creare legami. In particolare, si crea un legame fra

- identificatore dell'oggetto;
- nome della classe.

Pertanto la definizione di una classe corrisponde a una particolare forma di *astrazione generica*, cioè di astrazione applicata alla classe sintattica dichiarazione. Si è già visto che un modulo può essere usato per incapsulare un *tipo astratto* con gli operatori leciti. Vediamo se ha senso definire *generic* un modulo di questa fatta e poi istanziarlo. Riepilogando, se in fase di progettazione si identifica l'esigenza di un dato astratto, in fase realizzativa si può:

1. *Definire un oggetto*: la scelta è appropriata nel caso in cui si necessita di una sola occorrenza del dato astratto;
2. *Definire un tipo astratto*: l'astrazione riguarda la classe sintattica tipo;
3. *Definire una classe*: l'astrazione riguarda la dichiarazione di un modulo (dotata di stato locale).

In tutti i casi la rappresentazione del dato astratto viene nascosta e la manipolazione dei valori è resa possibile solo mediante operazioni fornite allo scopo. Tuttavia, ci sono delle differenze fra tipo astratto e classe di oggetti.

- **Sintattica**: nel tipo astratto gli operatori hanno un parametro in più, relativo proprio al tipo che si sta definendo;
- **Realizzativa**: nel caso del tipo astratto gli operatori sono definiti una sola volta, mentre nel caso della classe gli operatori sono definiti più volte, tante quante sono le istanze. Le diverse copie degli operatori agiranno su diversi dati in memoria centrale;
- **Concettuale**: richiamando la suddivisione delle operazioni su un dato astratto in osservazioni e costruttori, si può dire che:
 - il tipo astratto è organizzato intorno alle osservazioni. Ogni osservazione è implementata come un'operazione su una rappresentazione concreta derivata dai costruttori. Anche i costruttori sono implementati come operazioni che creano valori. La rappresentazione è condivisa dalle operazioni, ma è nascosta ai fruitori del tipo astratto.
 - La classe è organizzata intorno ai costruttori. Le osservazioni diventano metodi dei valori. Un oggetto è definito dalla combinazione di tutte le osservazioni possibili su di esso.

In breve, un tipo astratto di dato può essere inteso come un *insieme con operazioni*, mentre le classi sono *insiemi di operazioni*.

2.6.1 Vantaggi del tipo astratto

Nei linguaggi imperativi, i valori di un tipo astratto vengono trattati alla stregua dei valori di un tipo concreto, cioè sono cittadini di prima classe. Al contrario, i valori rappresentati mediante oggetti sono trattati come cittadini di terza classe in quanto:

- una procedura non può restituire l'istanza di un generic package;
- non è possibile creare dinamicamente degli oggetti in quanto le istanze sono stabilite al momento della compilazione.

I tipo astratti sono utili in tutti i paradigmi di programmazione, mentre gli oggetti, essendo variabili aggiornabili, si adattano bene solo a un paradigma di programmazione side-effecting. La notazione usata per chiamare un'operazione di un tipo astratto è più naturale perchè valori e variabili del tipo astratto sono argomenti espliciti.

2.6.2 Svantaggi del tipo astratto

I vantaggi del tipo astratto sono i seguenti.

- **Scarsa estendibilità:** l'aggiunta di un nuovo costruttore comporta dei cambiamenti intrusivi nelle implementazioni esistenti degli operatori. Ogni operatore dovrà essere opportunamente rivisto in modo da prevedere il trattamento di rappresentazioni ottenute con nuovi costruttori.

Esempio

Si vuole implementare il dato astratto *geometricShape* la cui specifica algebrica è fornita di seguito:

<i>osservazioni</i>	<i>Costruttore di g</i>	
	<i>square(x)</i>	<i>circle(r)</i>
<i>area(g)</i>	x^2	πr^2

```
1 package geometricShape_type
2   type geometricShape is limited private
3   function square(x: real): geometricShape;
4   function circle(x: real): geometricShape;
5   function area(g: geometricShape): real;
```

```

6  private
7      type geometricShape = record
8          shape: char;
9          value: char;
10     end
11 end geometricShape_type;
12 package body geometricShape_type
13     function square(x: real): geometricShape
14     var g: geometricShape
15     begin
16         g.shape := 's';
17         g.value := x;
18         return g
19     end
20
21     function circle(x: real): geometricShape
22     var g: geometricShape
23     begin
24         g.shape := 'c';
25         g.value := x;
26         return g
27     end
28
29     function area(g: geometricShape): real
30     begin
31         if g.shape = 's' then return g.value * g.value;
32         else return 3.14 * g.value * g.value;
33     end
34 end geometricShape_type

```

Realizzando il dato astratto mediante *classi* si possono definire due moduli generici, uno per ogni forma geometrica (o **costruttore**).

```

1  generic package Circle
2      function area(): real;
3      procedure init(real);
4  end Circle
5  package body Circle
6      var raggio: real;
7      procedure init(x: real)
8      begin
9          raggio := x
10     end;
11
12     function area(): real
13     begin

```

```

14     return 3.14 * raggio * raggio
15 end;
16 end Circle
17
18 generic package Square
19     function area(): real;
20     procedure init(real);
21 end Square
22 package body Square
23     var lato: real;
24     procedure init(x: real)
25     begin
26         lato := x
27     end;
28
29     function area(): real
30     begin
31         return lato * lato
32     end;
33 end Square

```

Per utilizzare una forma geometrica si deve istanziare una delle due classi e invocare il metodo *init*.

Se si estende la specifica del dato astratto in modo da considerare anche i rettangoli, tramite il costruttore *rectangle(l, m)*, se abbiamo specificato un **tipo astratto**, siamo stati costretti a cambiare la rappresentazione in modo da memorizzare due valori e non uno.

```

1 type geometricShape = record
2     shape: char;
3     value: real;
4     value2: real
5 end

```

Inoltre dobbiamo aggiungere l'opportuno costruttore

```

1 function rectangle(x, y: real): geometricShape
2 var g: geometricShape
3 begin
4     g.shape := 'r';
5     g.value := x;
6     g.value2 := y;
7     return g
8 end
9 function area(g: geometricShape): real
10 begin

```

```

11  if g.shape = 's' then return g.value * g.value
12  else if g.shape = 'c' then return 3.14 * g.value * g.value
13  else return g.value * g.value2
14  end

```

Diversamente, avendo realizzato il dato astratto mediante classi basta aggiungere un'altra classe.

```

1  generic package Rectangle
2    function area(): real;
3    procedure init(real, real);
4  end
5  package body Rectangle
6    var base, altezza: real;
7    procedure init(x, y: real)
8    begin
9      base := x;
10     altezza := y;
11   end;
12   function area(): real
13   begin
14     return base * altezza
15   end;
16 end Rectangle

```

Nella programmazione orientata a oggetti, per evitare la riscrittura di codice comune ad altre classi già definite è possibile ricorrere ai meccanismi di *ereditarietà* tra le classi.

2.7 Astrazione generica

Il concetto di classe di oggetti va inquadrato in un tema più generale, quello dell'*astrazione generica*. Il principio di astrazione suggerisce che si può astrarre anche sulla classe sintattica *dichiarazione*, in quanto essa sottointende una computazione. In particolare la valutazione di una dichiarazione comporta la creazione di **legami** detti *bindings*.

Un'astrazione generica è un'astrazione su una dichiarazione, pertanto il corpo della dichiarazione di un'astrazione generica è a sua volta una dichiarazione. La chiamata di un'astrazione generica è detta **istanziamento** e produce dei legami elaborando la dichiarazione contenuta nel corpo dell'astrazione generica.

Analogamente a quanto detto per le altre astrazioni, l'astrazione generica potrà essere specificata come segue:

$$\text{generic } I(FP_1; \dots; FP_n) \text{ is } D$$

dove I è un identificatore dell'astrazione generica, $FP_1; \dots; FP_n$ sono parametri formali, e D è una dichiarazione che, quando elaborata, produrrà dei legami. D funge da *matrice* dalla quale ricavare le dichiarazioni per istanziazione. Una dichiarazione D può essere:

- la dichiarazione di un tipo;
- la dichiarazione di un modulo;
- la dichiarazione di una funzione;
- la dichiarazione di una procedura;
- ...

Occorre pensare a dei meccanismi per poter distinguere le diverse dichiarazioni che si ottengono per istanziazione. Un modo è quello di specificare un diverso identificatore all'atto dell'istanziazione, come nel seguente esempio

A instantiation of I ;

Così, la seguente dichiarazione generica

```

1 generic typeRxR is
2   typeRxR = record
3     x: real;
4     y: real
5   end;
```

potrà essere utilizzata come matrice per generare le dichiarazioni per i tipi *Point2d* e *Complex*:

```

1 Point2D instantiation of typeRxR
2 Complex instantiation of typeRxR
```

Si è visto un utilizzo dell'astrazione generica nella dichiarazione di moduli dotati di stato locale. Mediante l'operazione di istanziazione si ottengono diverse copie dell'oggetto che differiscono solo per il nome dell'identificatore. L'astrazione generica di un oggetto corrisponde al concetto di classe.

2.7.1 Generic package in Ada

Il *generic package* di Ada è una esemplificazione di astrazione generica. L'espressione:

package ST1 is new STACK

è un esempio di istanziazione generica.

Le astrazioni generiche, come qualsiasi altra astrazione, possono essere parametrizzate.

Esempio

Nel seguente esempio, viene definita una classe coda in Ada. La variabile **items** è un array di caratteri. Al fine di svincolare la definizione di classe da particolari costanti legate all'applicazione si dota l'astrazione generica del *parametro formale* **capacity**, che è utilizzato per dimensionare l'array. L'istanziazione deve consentire di specificare il parametro effettivo, che sarà un valore da associare al parametro formale. In Ada l'istanziazione sarà specificata come segue:

```
1 package line_buffer is new queue_class(120);
2 generic
3     capacity: Positive;
4 package queue_class is
5     procedure append(newitem: in Character);
6     procedure remove(olditem: out Character);
7 end queue_class;
8 package body queue_class is
9     items: array(1..capacity) of Character;
10    size, front, rear: Integer range 0..capacity;
11    procedure append(newitem: in Character) is
12        ...;
13    procedure remove(olditem: out Character) is
14        ...;
15 begin
16    ...
17 end queue_class;
```

In principio, si può applicare l'astrazione generica a qualunque dichiarazione, incluso le procedure e le funzioni. Ad esempio, si potrebbe dichiarare una procedura **T_swap** per scambiare dati di un tipo **T** predefinito:

```
1 generic
2 procedure T_swap(a, b: in out T);
3 procedure T_swap(a, b: in out T) is
4     tmp: T;
5 begin
6     tmp := a;
7     a := b;
8     b := tmp;
9 end T_swap;
```

e ottenere diverse copie di essa per istanziiazione:

```
1 procedure swap1 is new T_swap
2 procedure swap2 is new T_swap
```

In realtà è poco utile disporre di due funzioni identiche ma di nome diverso. Diversa sarebbe la situazione se potessimo dichiarare una generica procedure **T_swap** che opera sui dati di tipo T qualunque, e potessimo specificare il tipo al momento dell'istanziiazione. Per ottenere questo risultato necessitiamo di una particolare classe di parametri, quelli di tipo.

```
1 generic
2   type T is private;
3   procedure T_swap(a, b: in out T);
4   procedure T_swap(a, b: in out T) is
5     tmp: T;
6   begin
7     tmp := a;
8     a := b;
9     b := tmp;
10  end T_swap;
```

La clausola *generic* introduce un **parametro di tipo** e la dichiarazione che segue introduce la matrice di una procedura che scambia due dati di un tipo T generico. Le procedure effettive sono ottenute istanziando la procedura generica con i parametri di tipo effettivi da sostituire a T.

```
1 procedure int_swap is new T_swap(Integer);
2 procedure str_swap is new T_swap(String);
```

In questo modo si è:

- Svincolato la definizione dello scambio di due elementi da un fattore marginale, come il tipo degli elementi da scambiare;
- Garantito comunque il **controllo statico dei tipi** fra parametri formali e parametri effettivi delle diverse procedure ottenute, e fra sorgente e destinazione di una assegnazione.

L'uso dei parametri di tipo in astrazioni generiche offre un buon **compromesso** fra necessità di dover effettuare il controllo statico dei tipi e desiderio di definire componenti software riutilizzabili.

I parametri di tipo possono essere utilizzati anche quando si definiscono delle classi, come indicato in questo esempio.

```

1 generic
2   max: Positive;
3   type ITEM is private;
4 package Stack is
5   procedure push(x: in ITEM);
6   procedure pop;
7   procedure top(x: out ITEM);
8   function empty return Boolean;
9 end Stack;
10 package body Stack is
11   type Table is array(1..max) of ITEM;
12   st: Table;
13   top: Integer range 0..max := 0;
14   procedure push(x: in ITEM) is
15   begin
16     top := top + 1;
17     st(top) := x;
18   end push;
19   procedure pop is
20   begin
21     top := top - 1;
22   end pop;
23   procedure top(x: out ITEM) is
24   begin
25     x := st(top);
26   end top;
27   function empty return Boolean is
28   begin
29     ...

```

In questo caso per creare i singoli oggetti scriveremo:

```

1 declare
2   package STACK_INT is new STACK(10, INTEGER);
3   with STACK_INT; use STACK_INT;
4   package STACK_REAL is new STACK(10, REAL);
5   with STACK_REAL; use STACK_REAL;
6   A: real; B: Integer;
7 begin
8   push(12); push(15.0); top(B); top(A)
9 end

```

Si osservi che non è necessario utilizzare la notazione puntata in quanto push e top sono differenziate dal contesto (tipo di parametro effettivo passato).
L'astrazione generica è quindi di supporto all'astrazione dati, in quanto permette

di definire delle classi che sono invarianti ad alcuni tipi di dati necessari per definirle. Inoltre, l'astrazione generica, mediante i parametri di tipo, è applicabile a tipi astratti che possono essere così ugualmente svincolati dalla necessità di specificare il tipo degli elementi sui quali operare.

Consideriamo, ad esempoio, di definire dei tipi astratti per una applicazione che usa:

1. stack di interi;
2. stack di reali;
3. stack di un tipo astratto point3D

Un modo di procedere sarebbe quello di scrivere una definizione separata per ciascuno dei tre tipi, ma, questo presenta degli svantaggi.

1. **Codice duplicato** in quanto plausibilmente simile per tutte le definizioni;
2. **Sforzo di programmazione ridondante**;
3. **Manutenzione complicata** poichè le modifiche, come l'aggiunta di un nuovo operatore, vanno plausibilmente effettuate in tutte le versioni.

Una alternativa sarebbe quella di separare le proprietà di uno stack dalle proprietà dei loro elementi e questo è possibile farlo utilizzando i parametri di tipo.

Se un'astrazione è parametrizzata rispetto a un valore, possiamo usare l'argomento valore anche se non sappiamo nulla al di fuori del suo tipo. Analogamente se un'astrazione è parametrizzata rispetto a una variabile, possiamo ispezionare e aggiornare la variabile argomento anche se non sappiamo nulla oltre il suo tipo. Ma quando si parametrizza rispetto al tipo la situazione cambia. Nell'esempio di T_swap avevamo le assegnazioni:

```
tmp := a;  
a := b;  
b := tmp;
```

ma nessuno garantisce che l'assegnazione sia un'operazione valida per i dati di tipo T.

In Ada, l'espressione "*type T is private*" sottointende che l'assegnazione e i predicati $=$ e $/=$ sono operazioni valide per il tipo effettivo denotato da T.

2.7.2 Astrazione generica in C++

In C++ l'astrazione generica è supportata mediante i *template*. Un template è del codice generico dotato di parametri che possono assumere valori specifici al momento della compilazione. Ad esempio, piuttosto che scrivere due classi, ListOfInts e ListOfStrings, si potrebbe scrivere una singola classe template:

```
template <class T> class List
```

dove il parametro di tipo T del template può essere rimpiazzato da un qualunque tipo quando il codice è compilato. Ciò è ottenuto mediante un'istanziamento del template:

```
List<int> I1;  
List<string> h2;  
List<int> I3;
```

Il compilatore genererà tre definizioni di classi, una per ogni istanziazione. In generale ogni istanziazione di un template produce una copia del codice template. La copia è creata in fase di precompilazione.

Si distinguono due categorie di template in C++:

1. **Template di classe:** definisce la struttura e le operazioni per un insieme illimitato di tipi correlati.
Ad esempio, un template di classe Stack, potrebbe fornire una definizione comune per una pila di Int, di float, e così via.
2. **Template di funzione:** definisce un insieme illimitato di funzioni correlate.
Ad esempio, una famiglia di funzioni di ordinamento potrebbe essere dichiarata in questo modo:

```
1 template <class T> void sort(vector<T>);
```

Una funzione generata da una template di funzione è chiamata **funzione template**.

In C++ non è necessaria l'istanziamento esplicito dei template. Nell'esempio, saranno istanziate due funzioni

```
sort(vector<complex>)  sort(vector<int>)
```

Le diverse istanze sono generate sulla base dei tipi dei parametri effettivi delle chiamate a funzioni generiche. Nell'invocazione della funzione sort c'è ambigui-

tà, in quanto la stessa funzione può avere parametri differenti, questo è un caso di **overloading**, ovvero di associazione dello stesso identificatore di funzione a realizzazioni differenti. Il compilatore risolve questa ambiguità sulla base del tipo degli argomenti effettivi.

Template vs. Generics

Nonostante la similarità sintattica, i template del C++ e le Generics di Java sono piuttosto differenti. La differenza è soprattutto nel modo in cui sono trattate dal compilatore. Il compilatore C++ genera del codice specifico per ogni istanziazione del template. Il compilatore Java introduce dei controlli al compile-time sulle classi generiche in modo da controllare ai metodi della classe. Il codice della classe generica non è ricompilato.

Capitolo 3

Paradigma Object Oriented

3.1 Gli oggetti

Si deve osservare che nella programmazione imperativa è possibile definire degli oggetti, tuttavia

1. il loro utilizzo è legato all'auto disciplina dei programmatori e non viene in alcun modo forzato;
2. gli oggetti **non sono cittadini di prima classe**.

Confrontando il paradigma orientato a oggetti rispetto a quello imperativo si può dire che esso costituisce:

1. Un'*evoluzione* in quanto permette agli oggetti di essere cittadini di prima classe;
2. Una *rivoluzione* in quanto gli oggetti assumono un ruolo **fondamentale** nella progettazione e nella programmazione. L'information hiding e l'incapsulamento sono principi cardine nel paradigma orientato agli oggetti.

Gli oggetti incapsulano uno *stato* e un *comportamento*. Lo stato è identificato dal contenuto di una certa area di memoria. Il comportamento è definito da una collezione di procedure e funzioni, detti *metodi*, che possono operare sulla rappresentazione dell'area di memoria associata all'oggetto.

Da una prospettiva di progetto, gli oggetti modellano le *entità* presenti nel dominio dell'applicazione.

Esempio

In un gioco elettronico che usa palla, si può pensare alla palla come un oggetto dotato di uno:

- **Stato:** dimensione, posizione in uno spazio di riferimento, ecc.
- **Comportamento:** una palla può “apparire” o “scompare” dallo schermo, può muoversi, rimbalzare, ecc.

Le variabili e i metodi definiti nell’oggetto “palla” stabiliscono lo stato e il comportamento che sono rilevanti all’uso della palla nel gioco elettronico.

3.1.1 Identificatore di oggetto

Un oggetto ha la sua *identità*, cioè è riconoscibile indipendentemente dal suo stato corrente. Per questo ogni oggetto ha un *identificatore di oggetto*, detto object identifier OID, che lo identifica univocamente. In alcuni contesti gli OID sono anche detti *referimenti*.

Un identificatore di oggetto è *immutabile*, cioè non può essere modificato da una qualche operazione di programmazione. Cambiare l’OID di un oggetto equivale alla cancellazione dell’oggetto e alla creazione di un altro oggetto con lo stesso dato.

Normalmente gli OID sono assegnati in modo automatico agli oggetti, sicché non hanno un significato nel mondo reale. In molti ambienti di programmazione object-oriented, l’OID corrisponde all’indirizzo dell’area di memoria che conserva lo stato dell’oggetto.

Quasi mai il programmatore utilizza esplicitamente i riferimenti. Generalmente questi vengono legati a delle variabili e si fa riferimento agli oggetti mediante identificatori di variabile. Variabili distinte possono riferirsi al medesimo oggetto. In questo caso si hanno degli **alias**.

Nota

La presenza degli alias non significa che un oggetto non è identificato univocamente, ma semplicemente che diversi identificatori di variabile sono stati legati al medesimo riferimento di oggetto.

Lo stato di un oggetto può anche contenere il riferimento ad un altro oggetto. Si dice che un oggetto *punta* ad un altro. Il puntamento è asimmetrico. La simmetria è ottenuta mediante le reciprocità di puntamento.

3.2 Le classi

Una *classe* è la descrizione di una famiglia di oggetti che condividono la stessa struttura (gli attributi) e il medesimo comportamento (operazioni). Nella programmazione OO *ogni oggetto è un'istanza di una classe*, cioè *un oggetto non può essere definito se non si definisce la classe di appartenenza*. Analogamente nella modellazione OO *le istanze esistono in quanto ci sono le loro astrazioni*.

Idealmente una classe è una realizzazione di un dato astratto. Questo significa che i dettagli della realizzazione sono normalmente nascosti.

Ogni classe ha una doppia componente:

1. **Una componente statica**, i dati, costituita da *campi* o *attributi* dotati di nome, che contengono un valore. I campi caratterizzano lo stato degli oggetti durante l'esecuzione del programma.

Gli attributi si distinguono in base al loro ambito d'azione, detto *scope*:

- **Attributi d'istanza**: sono associati ad un'istanza e hanno un tempo di vita pari a quello dell'istanza alla quale sono associati;
- **Attributi di classe**: sono associati alla classe e condivisi da tutte le istanze della classe. Il loro tempo di vita è lo stesso della classe.

Gli attributi d'istanza contribuiscono a caratterizzare lo stato di ogni singolo oggetto, mentre gli attributi di classe contribuiscono a definire il fattore comune allo stato di tutti gli oggetti di una classe.

2. **Una componente dinamica**, i metodi o operazioni, che rappresentano il comportamento comune degli oggetti appartenenti alla classe, cioè i servizi che possono essere richiesti a un oggetto di una classe. I metodi manipolano gli attributi.

I metodi possono essere classificati:

- **Metodi costruttori**: sono invocati per creare (istanziare) gli oggetti e inizializzarli;
- **Metodi di accesso**: restituiscono astrazioni significative dello stato di un oggetto;
- **Metodi di trasformazione**: modificano lo stato di un oggetto;
- **Metodi distruttori**: sono invocati quando si rimuovono gli oggetti dalla memoria.

I metodi di accesso e trasformazione possono essere distinti in:

- 2.1. **Metodi di istanza:** operano su almeno un attributo di istanza, pertanto possono essere invocati solo specificando l'istanza;
- 2.2. **Metodi di classe:** operano esclusivamente su attributi di classe, pertanto possono essere invocati specificando la classe. Si possono invocare metodi di classe anche quando non è stato creato alcun oggetto per quella classe.

In linea di principio, l'invocazione di un metodo di classe può avvenire anche specificando un oggetto, tuttavia ciò è sconsigliato perché non evidenzia il fatto che si manipolano solo attributi di classe.

Invece, non è possibile invocare un metodo di istanza sulla classe.

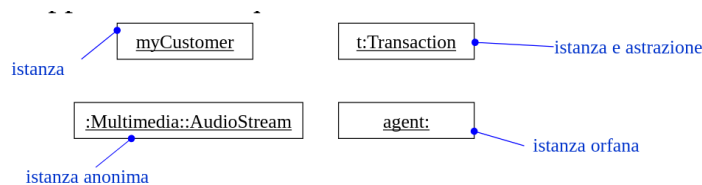
3.3 UML

In UML, Unified Modeling Language, un linguaggio visuale utilizzato per definire, progettare, realizzare e documentare sistemi mediante un approccio object oriented. È un linguaggio di **rappresentazione dei sistemi** ed è **universale**: può rappresentare sistemi eterogenei per architettura, tecnologie, tipologia applicativa ecc.

UML è di supporto alla progettazione di un nuovo sistema e alla documentazione di un sistema esistente.

3.3.1 Oggetti in UML

In UML un oggetto, o *istanza*, è graficamente rappresentato in questo modo:

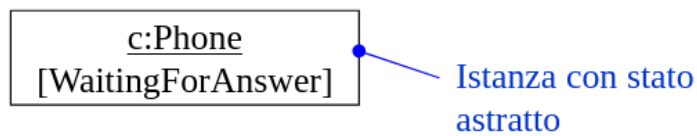


È possibile indicare:

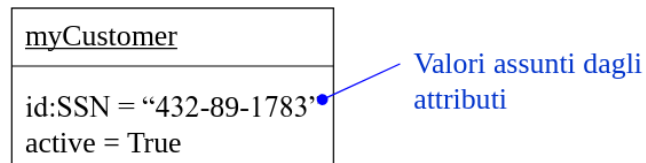
- solo il nome (identificatore) dell'istanza;

- nome dell'istanza e dell'astrazione (la classe) cui appartiene;
- solo il nome dell'astrazione, qualora non si conosca a priori il nome dell'istanza;
- un'istanza orfana, se non si conosce a priori la sua astrazione.

Lo stato di un oggetto può essere rappresentato in modo astratto:



oppure indicando esplicitamente i valori assunti dagli attributi dell'oggetto



3.3.2 Le classi in UML

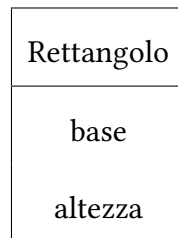
In UML una classe è resa graficamente mediante un rettangolo:



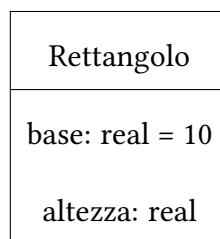
Ogni classe deve avere un nome che la contraddistingue dalle altre. Questo può essere semplice o indicare un percorso.



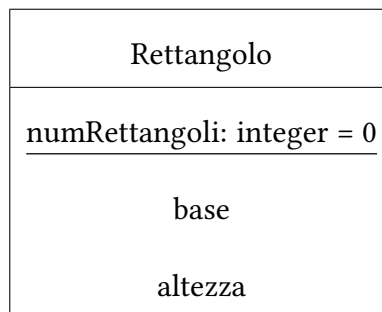
Graficamente gli attributi sono sotto il nome della classe.



Di un attributo si può specificare l'insieme dei valori assunti (oggetti di una classe) e una eventuale inizializzazione.

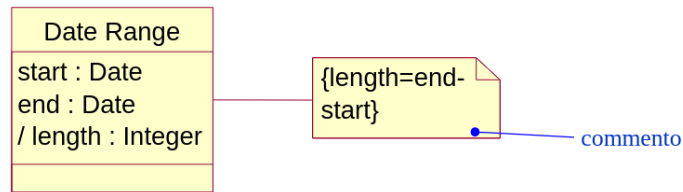


Un attributo di classe, detto **statico** in UML, è indicato come sottolineato.



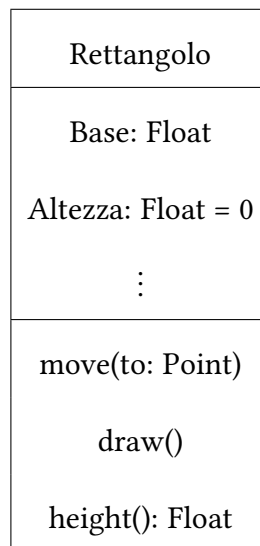
L'attributo numRettangoli indica il numero di oggetti della classe Rettangoli che sono stati istanziati. È un attributo statico in quanto condiviso da tutte le istanze della classe Rettangoli. Esso è inizializzato a 0 e verosimilmente sarà incrementato dai costruttori della classe Rettangoli, mentre sarà decrementato dai distruttori.

Gli attributi **derivati** sono quelli che possono essere calcolati partendo da altri attributi. UML prevede una rappresentazione specifica mediante una /.



Il commento fa parte della notazione standard UML e in questo caso è utilizzato per indicare come si calcola l'attributo derivato length.

I commenti possono essere aggiunti a qualunque elemento della notazione UML. In UML le operazioni sono indicate graficamente in una sezione al di sotto degli attributi:



Di una operazioni si può specificare la **segnatura**.

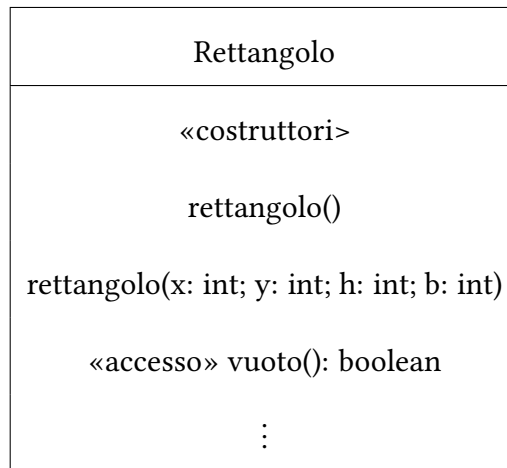
Una classe può essere rappresentata a diversi livelli di dettaglio:

- specificandone solo il nome;
- specifandone gli attributi e i metodi.

I diversi livelli di dettaglio consentono a chi progetta/modella di attribuire maggiore o minore importanza a determinati fattori a seconda della vista del sistema che si sta considerando.

3.3.3 Gli stereotipi

Consideriamo la seguente classe.



In questa classe si è fatto uso della notazione «costruttori» e «accesso» e questi sono due tipici esempi di **stereotipi**. Gli stereotipi sono dei tipici *meccanismi di estendibilità* di UML. Infatti essi estendono il vocabolario di UML, permettendo di creare nuovi blocchi per la costruzione dei modelli, derivandoli da blocchi già esistenti ma rendendoli specifici per il particolare dominio.

Nel precedente esempio è stato esteso il blocco “operazioni” in modo da poter distinguere le diverse tipologie di operazioni. Gli stereotipi sono identificabili perchè racchiusi da una coppia di caporali, « ».

3.3.4 Visibilità di attributi e metodi

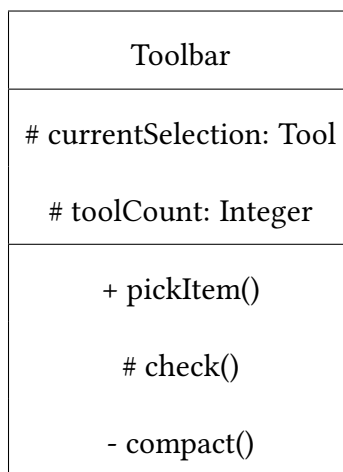
Gli attributi e i metodi di una classe possono avere diversi livelli di *visibilità*. Un elemento, attributo o metodo, ha visibilità **pubblica** quando può essere visto (utilizzato, invocato) da altre classi. Ha visibilità **privata** quando può essere visto solo dalla classe di appartenenza.

Altri livelli di visibilità sono:

- **protetta**: l'elemento è visibile all'interno del package e all'esterno solo ai discendenti della classe di appartenenza;
- **package**: l'elemento è visibile solo agli elementi del package che contiene la classe in cui l'elemento è definito.

UML consente di specificare i livelli di visibilità di attributi e metodi utilizzando la seguente notazione:

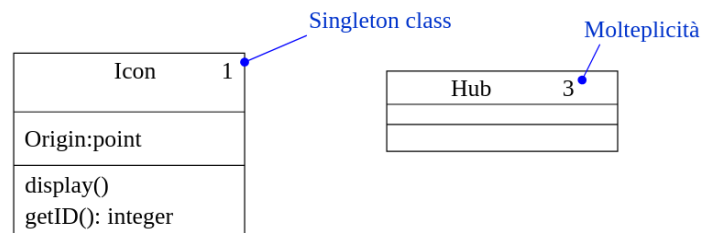
- **public**: l'elemento è preceduto da un +;
- **protected**: l'elemento è preceduto da un #;
- **private**: l'elemento è preceduto da un -;
- **package**: l'elemento è preceduto da un



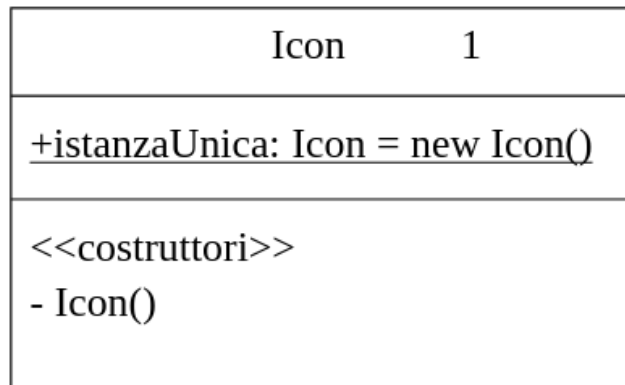
3.3.5 Molteplicità di classe

Con il termine **molteplicità di classe** si intende il numero di istanze che essa può avere. Generalmente non si pone un limite, tuttavia in alcuni casi è necessario indicare che la classe può avere una sola istanza, detta **classe singoletto**, o comunque un numero ben definito di istanze.

UML impone che tale valore sia indicato in alto a destra nell'icona rappresentante la classe.



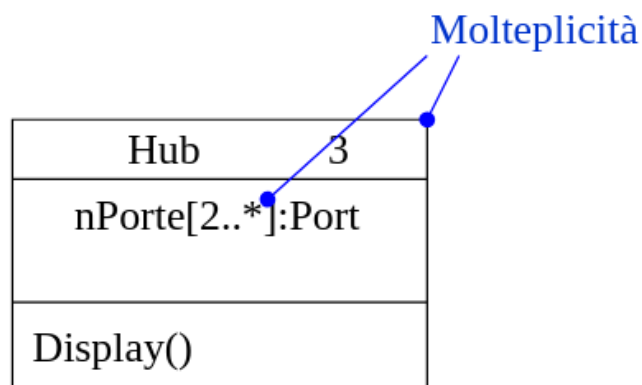
Per realizzare una classe singoletto, ad esempio



basta rendere privati i costruttori della classe e dotando la classe di un attributo statico che è inizializzato all'unico oggetto di quella classe. Ovviamente, nessun metodo della classe singoletto invocherà il costruttore, nè potrà farlo un utente della classe. Inoltre può essere utile disporre di un metodo di classe che restituisce l'unica istanza memorizzata nella stessa classe singoletto.

3.3.6 Molteplicità di un attributo

È possibile indicare la molteplicità anche per gli attributi, subito dopo il loro nome.



Si è visto come è possibile specificare in UML le varie caratteristiche degli attributi di una classe. A riepilogo, mostriamo lo schema generale per la definizione di un attributo:

[visibilità] nome [molteplicità] [:tipo] [= valore iniziale] [proprietà]

Di seguito alcuni esempi.

- origine solo il nome;
- + origine visibilità e nome;
- origine: point nome e tipo;
- testa: *elemento nome e tipo complesso;
- nome[0..1]: String nome, molteplicità e tipo;
- origine: Point = (0, 0) nome, tipo e valore iniziale;
- id: Integer frozen nome, tipo e proprietà.

In UML esistono tre **proprietà** predefinite che possono essere utilizzate con gli attributi:

- **changeable**: non vi sono restrizioni sulla modificabilità dell'attributo;
- **addOnly**: per gli attributi con molteplicità maggiore di uno, i valori possono essere aggiunti, ma una volta creati, non possono essere più rimossi o modificati;
- **frozen**: il valore dell'attributo non può più essere modificato dopo che l'oggetto è stato inizializzato.

Nel caso in cui la proprietà non viene specificata si sottintende che assume valore *changeable*.

3.3.7 Schema per la definizione di un'operazione

Prima di definire lo schema per la definizione di un'operazione, è necessario sottolineare che UML distingue tra operazione e metodo:

- un **operazione** è un servizio che può essere richiesto alla classe;
- un **metodo** è un'implementazione del servizio.

Infatti, possono anche esistere diversi metodi per la stessa operazione nei diversi livelli della gerarchia delle classi.

Lo schema per la definizione di un'operazione è la seguente:

[visibilità] nome [(lista dei parametri)] [: valore di ritorno] [proprietà]

Di seguito alcuni esempi di definizioni lecite di operazione.

- `visualizza` solo il nome;
- `+ visualizza` visibilità e nome;
- `set(n: Nome; s: String)` nome e parametri;
- `getId(): Integer` nome e tipo del dato restituito;
- `riparti() guarded` nome e proprietà.

Ogni parametro riportato nella segnatura prende la forma:

[direzione] nome : tipo [= valore iniziale]

La direzione può assumere uno dei seguenti valori:

- **in** parametro di input, non può essere modificato;
- **out** parametro di output, può essere modificato per comunicare un'informazione al chiamante;
- **inout** parametro di input che comunque può essere modificato.

UML fornisce diverse proprietà predefinite per le operazioni.

- **isQuery**: l'esecuzione dell'operazione lascia lo stato del sistema immutato, un'operazione con tale proprietà è quindi priva di *side-effect*;
- **leaf**: l'operazione non può essere più specializzata (overriding) nelle sottoclassi;
- **sequential**: i chiamanti (callers) di questo oggetto devono coordinarsi affinché solo una volta richieda il servizio. Nel caso di sovrapposizione la semantica e l'integrità dell'oggetto **non sono garantite**;
- **guarded**: simile al caso precedente, in ogni istante un solo chiamante può usufruire del servizio, tuttavia, in questo caso, la sequenzialità del servizio è gestita dalla classe proprietaria del servizio stesso;

- **concurrent:** la semantica e l'integrità dell'oggettum è garantita anche in caso di chiamate multiple.

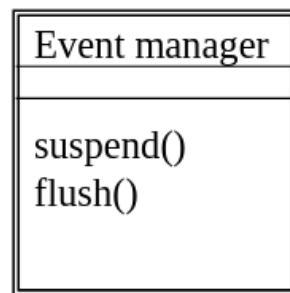
3.4 Classi attive

Le ultime tre proprietà sono rilevanti solo in classi *attive*, cioè in classi i cui oggetti sono attivi.

Un oggetto è attivo se esso ha un thread e può far partire un thread concorrente.

Una classe attiva è simile ad una classe con l'eccezione che le sue istanze rappresentano elementi il cui comportamento è concorrente con gli altri.

Essa è mostrata con bordi raddoppiati.



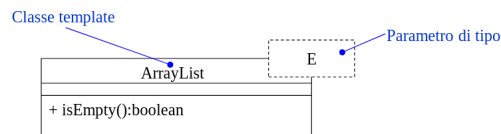
3.5 Classi template

Una classe *template* definisce una famiglia di classi parametrizzate (con parametro di tipo). Non è possibile usare direttamente una classe template. È necessario prima specificare il tipo (operazione di istanziiazione). In Java, una classe template corrisponde ad una classe generica.

Esempio

```
1 public class ArrayList<E>
```

In UML:

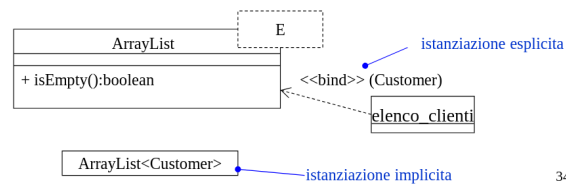


Si potrebbe voler istanziare questo template per creare una lista di clienti.

```
ArrayList<Customer> elenco_clienti;
```

L'istanziatura di una classe template può essere effettuata in due modi:

1. Implicitamente, dichiarando una classe il cui nome esplicita i parametri;
2. Esplicitamente, mediante una dipendenza stereotipata «bind»



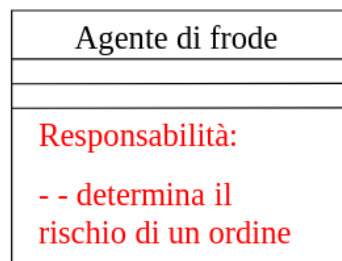
34

3.6 Individuazione delle responsabilità

In una buona modellazione OO di un sistema software è necessario stabilire le responsabilità da attribuire a ciascuna classe individuata.

UML consente di modellare le responsabilità in due modi.

1. specificandole all'interno della classe:



2. utilizzando delle note:

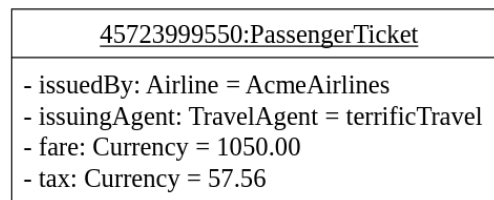


3.7 La relazione Instance-Of

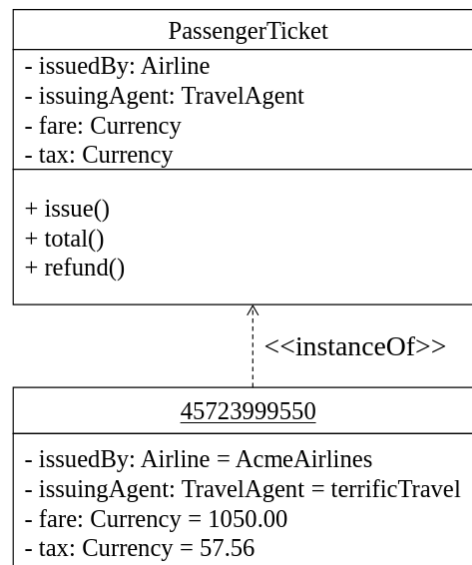
Fra un oggetto e una classe esiste una relazione “*instance of*” che specifica che un oggetto è un’istanza di una classe. In UML questa relazione è resa con lo stereoptipo «instanceOf».

Le seguenti notazioni grafiche sono semanticamente equivalenti ma cambia il livello di dettaglio.

La prima è la seguente:



In alternativa,



3.8 Ereditarietà: una relazione fra classi

Nella progettazione e programmazione OO una relazione fondamentale è quella esistente fra le classi: *la relazione di ereditarietà (inheritance)*.

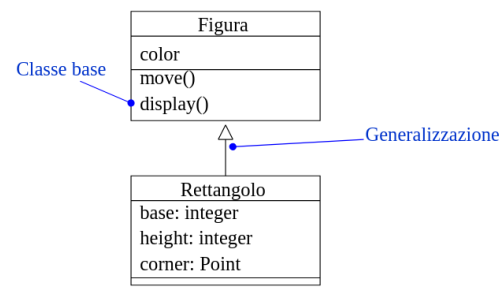
Una classe è considerata come *repertorio di conoscenze* a partire dalla quale è possibile definire classi più specifiche, che completano le conoscenze della loro classe madre.

Una *sottoclasse* è dunque, una **specializzazione** della descrizione di una classe, detta la sua superclasse, della quale essa mutua (parte di) gli attributi e i metodi. Concettualmente è come se le informazioni della superclasse fossero ricopiate nella sottoclasse. A libello fisico, comunque, le duplicazioni sono evitate.

Ci sono diverse forme di ereditarietà. Ne riportiamo alcune.

3.8.1 Ereditarietà per estensione

La sottoclasse introduce delle caratteristiche (attributi e metodi) non presenti nella superclasse e non applicabili a istanze della superclasse. La visibilità (pubblica, protetta, private e package) degli attributi degli attributi e delle operazioni ereditate dalla superclasse non è modificata.



Nota

È bene notare che la superclasse non conosce le classi figlie.

La specializzazione per estensione permette di sviluppare del codice estendibile. Allorquando si dovesse avvertire la necessità di aggiungere ulteriori funzioni, occorrerà individuare le classi interessate e derivare da queste nuove classi alle quali verranno aggiunti gli attributi e i metodi necessari per implementare le nuove funzioni.

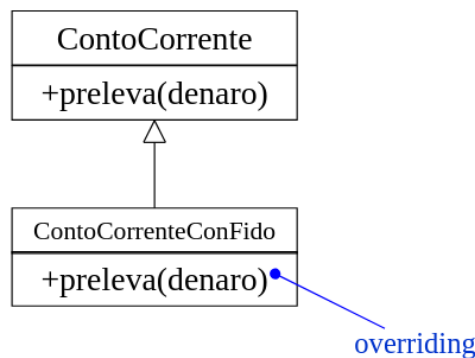
3.8.2 Ereditarietà per variazione funzionale

Si ridefiniscono alcune caratteristiche (metodi) della superclasse quando quelle ereditate si rivelano inadeguate per l'insieme di oggetti descritti dalla sottoclasse.

La ridefinizione, detta **overriding**, del metodo ereditato riguarda solo l'implementazione e non la segnatura.

Ogni richiesta di esecuzione del metodo ridefinito da parte di un oggetto della sottoclasse, farà riferimento alla nuova implementazione fornita nella sottoclasse.

Analizziamo il seguente esempio.



Nella classe *ContoCorrente* il metodo *preleva* controlla che il conto non vada in rosso. Il metodo *preleva* della classe *ContoCorrenteConFido* ridefinisce quello della superclasse per controllare che il prelievo non vada oltre il fido concesso.

Se si applicano delle modifiche al corpo della funzione *preleva* nella superclasse, la modifica non viene apportata automaticamente anche alla classe *ContoCorrenteConFido*.

La **ridefinizione non è incrementale**, quindi i cambiamenti nel metodo originale devono essere riportati anche nei metodi ridefiniti. Purtroppo non c'è garanzia che questo accada e si possono introdurre degli errori.

Anche nell'ereditarietà per variazione funzionale, la visibilità degli attributi e delle operazioni ereditate della superclasse non è modificata. Osserviamo, inoltre, che la variazione funzionale attiene solo le operazioni di accesso e trasformazione di una classe e non i costruttori degli oggetti.

3.8.3 Principio di sostituibilità

Data una dichiarazione di una variabile o di un parametro il cui tipo è dichiarato come X, una qualunque istanza di una classe che è discendente di X può essere usato come valore effettivo senza violare la semantica della dichiarazione e il suo uso.

In altri termini, l'istanza di un discendente può essere sostituita all'istanza di un ascendente.

Il principio di sostituibilità è legato al *polimorfismo di inclusione*, nella programmazione orientata agli oggetti.

La conseguenza del principio di sostituibilità è che una *sottoclasse non può rimuovere o rinunciare a proprietà/metodi della superclasse*. Altrimenti un'istanza della sottoclasse non sarà sostituibile in una situazione in cui si dichiara l'uso delle istanze della superclasse.

In effetti, preservando la visibilità degli attributi e dei metodi ereditati, così come accade nelle tre forme di ereditarietà viste, si garantisce che gli oggetti della sottoclasse offrano quanto meno gli stessi servizi degli oggetti della superclasse. Pertanto, il principio di sostituibilità, o *polimorfismo di inclusione*, è compatibile con l'ereditarietà per estensione e variazione funzionale.

3.8.4 Ereditarietà e relazione “is_a”

Nell'ereditarietà per estensione o per variazione funzionale, la relazione di ereditarietà fra classi corrisponde a una relazione di *generalizzazione* o detta *is_a*. Ciò perché ogni istanza di una classe derivata da una classe base va considerata come (è anche) un'istanza della classe base.

3.8.5 Ereditarietà di implementazione

La sottoclasse utilizza il codice della superclasse per implementare l'astrazione associata.

Esempio

Una classe *Vettore* mette a disposizione una rappresentazione di un vettore, un costruttore, un metodo di accesso e uno di trasformazione.

Una classe *Pila* può essere definita come sottoclasse di *Vettore*, in modo da potersi basare sulla rappresentazione del vettore per poter rappresentare una pila. Si dice che la realizzazione del dato astratto *Pila* è delegata a quella del vettore. Se si cambia la rappresentazione del vettore si cambia anche quella

Base/Derivata	Visibile ai clienti	Visibile alla sottoclasse
Public Spec.	Si	Si
Private Spec.	No	No

Base/Derivata	Visibile ai clienti	Visibile alla sottoclasse
Public Spec.	No	Si
Private Spec.	No	No

della pila.

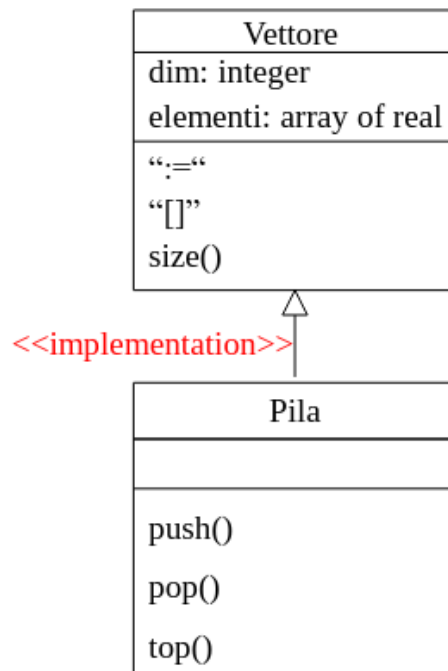
Grazie all'ereditarietà per implementazione, la realizzazione degli operatori *push*, *pop*, *top* previsti per una pila può basarsi sugli operatori messi a disposizione dalla classe *Vettore*.

Si noti bene che la *Pila* ha un diverso insieme di operatori rispetto a quelli del *Vettore*. L'ereditarietà riguarda solo la realizzazione del vettore e l'implementazione dei metodi, non l'**interfaccia** della classe.

Pertanto, l'ereditarietà di implementazione comporta la modifica alla visibilità delle caratteristiche ereditate.

L'ereditarietà a base privata, supportata dal C++ e da Ada-95, permette di rendere privati gli attributi e gli operatori pubblici ereditati.

Visibilità con *ereditarietà a base pubblica*: Visibilità con *ereditarietà a base privata*:
In UML l'ereditarietà di implementazione è indicata utilizzando lo stesso simbolo della generalizzazione, ma specificando a fianco lo stereotipo «implementation».

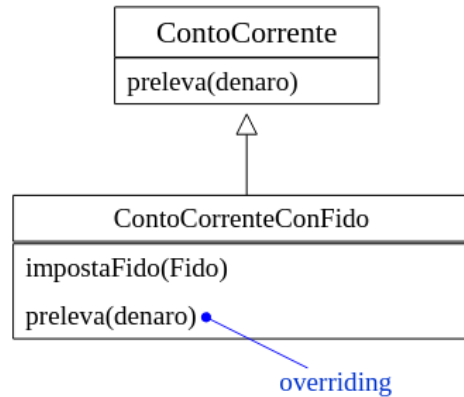


L'ereditarietà di implementazione **non è compatibile** con il principio di sostituibilità.

Pertanto se la classe Y eredita solo l'implementazione della classe X, non si può riutilizzare su istanze di Y tutto il codice in cui si dichiarano e utilizzano dati di classe X, in quanto non vale il polimorfismo di inclusione.

L'ereditarietà di implementazione permette un riuso parziale del codice, pertanto secondo alcuni andrebbe bandita. Altri, invece, sostengono che questa forma di ereditarietà, insieme a molte altre incompatibili con il principio di sostituibilità, sono comunque teoricamente legittime e praticamente indispensabili.

Tornando al concetto di ereditarietà, nella progettazione di una classe si possono combinare diverse forme di ereditarietà. Analizziamo il seguente esempio.



In questo esempio, si ha ereditarietà per estensione (si aggiunge il metodo *impostaFido*) e per variazione funzionale (si ridefinisce il metodo *preleva*).

3.8.6 Proprietà della relazione di generalizzazione

La rappresentazione della relazione di generalizzazione fra un insieme di classi definisce un *grafo di ereditarietà* che è un *grafo orientato aciclico*.

La relazione di generalizzazione è transitiva e antisimmetrica.

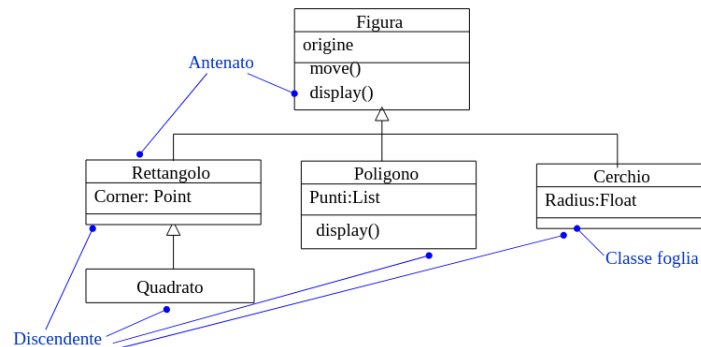
La transitività comporta che le caratteristiche delle classi superiori sono ereditate dalle classi inferiori.

L'antisimmetria definisce una direzione di attraversamento del grafo di ereditarietà che porta dalla sottoclasse alla superclasse.

Partendo da una classe *C* e seguendo la direzione che porta al genitore, si trovano tutte le classi *antenate*, che per estensione sono chiamate superclassi di *C*.

Seguendo la direzione opposta si trovano tutte le classi *discendenti* di *C*.

Il grafo di ereditarietà è il seguente.



Ogni classe C ha il proprio grafo di ereditarietà $G(C)$ che è una restrizione del grafo di ereditarietà completo alle sole superclassi della classe in esame.

3.8.7 Ereditarietà singola

In questo caso specifico, il grafo è in realtà un albero: ogni classe ha una sola superclasse diretta. Si dice che l'ereditarietà è *semplice* o *singola*.

In questo caso il grafo di ereditarietà $G(C)$ di una classe C è una catena di antenati. Gli elementi della catena sono ordinati secondo una relazione d'ordine totale.

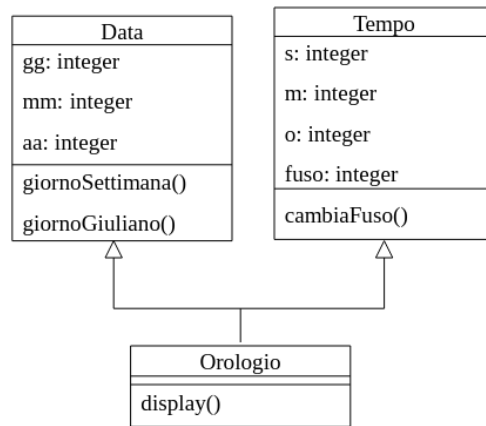
Poichè un metodo può essere ridefinito in più classi si pone il seguente problema.

Sia dato un metodo m , eventualmente ereditato, della classe C_1 . Da quale classe $C_2 \in G(C_1)$ si eredita m ?

Come prima cosa si determina la catena di antenati di C_1 e, successivamente, si ricerca la prima occorrenza della (ri-)definizione di m a partire dall'estremità C_1 della catena.

3.8.8 Ereditarietà multipla

Una classe può avere più superclassi. In questo caso si parla di ereditarietà multipla.

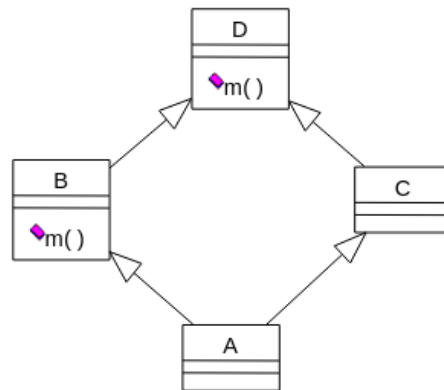


In questo esempio, la classe *Orologio* necessita dei servizi messi a disposizione sia da *Data* che da *Tempo*. In più ne implementa degli altri.

Il grafo di ereditarietà non è più un albero. Data una classe C , $G(C)$ non è più una catena ma un grafo aciclico orientato. L'ordine fra le classi in $G(C)$ è parziale.

Anche nell'ereditarietà multipla un metodo può essere ridefinito in diverse classi e si pone il seguente problema. Sia dato un metodo m , eventualmente ereditato, della classe C_1 . Da quale classe $C_2 \in G(C_1)$ si eredita m ?

Analizziamo il seguente esempio.



Se $C_1 = A$ c'è un conflitto fra le diverse definizioni di m in B e D , entrambe ereditabili. Il conflitto può essere facilmente risolto in questo caso. Si considera

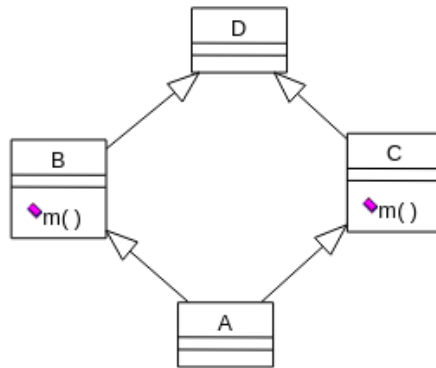
una qualunque *linearizzazione* del grafo $G(C_1)$. Si possono avere due casi:

$$A \rightarrow B \rightarrow C \rightarrow D$$

oppure,

$$A \rightarrow C \rightarrow B \rightarrow D$$

In entrambi i casi, B precede D , il che significa che il metodo m è ereditato da B . Il metodo m di B è quello più specifico e maschera quello omonimo di D . Consideriamo, invece, il seguente schema.



Le due linearizzazioni definite prima, non aiutano a scegliere. Esistono dei criteri euristici per gestire queste situazioni conflittuali:

1. La molteplicità dell'ereditarietà.

Nel definire che A deriva dalle due superclassi occorrerà elencarle in un qualche ordine:

$$A \rightarrow B, C \text{ piuttosto che } A \rightarrow C, B$$

L'ordine delle classi è considerato per risolvere il conflitto. Questo significa che l'ordine delle classi è utilizzato per preferire una delle due linearizzazioni. Tuttavia questo principio può essere in contraddizione con quello che indica di preferire l'eredità da classi più specifiche.

Ad esempio:

$$A \rightarrow B, C \quad C \rightarrow B, D$$

Se m è definito in B e in C si dovrebbe preferire C in quanto C precede sempre B nelle linearizzazioni:

$$A \rightarrow C \rightarrow B \rightarrow D$$

2. La modularità.

Si può scomporre un grafo di ereditarietà in moduli che corrispondono ai diversi punti di vista sull'oggetto.

Le linearizzazioni non devono mescolare i diversi sottografi associati ai moduli (non ci sono relazioni di ereditarietà tra classi appartenenti a moduli diversi). Anche in questo caso potrebbero esserci contraddizioni (da risolvere specificando l'ordinamento dei moduli).

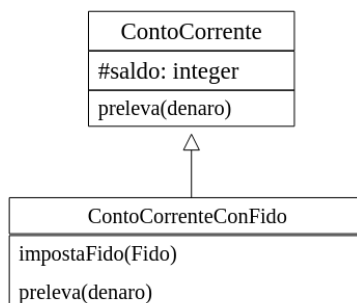
Queste contraddizioni mettono in evidenza le difficoltà che si incontrano a dare una semantica a un metodo quando si permette l'ereditarietà multipla. I diversi principi per gestire le situazioni conflittuali non sono universalmente accettati. La risoluzione dei conflitti, in realtà, non può essere efficace se non si prende in considerazione le conoscenze specifiche legate all'applicazione.

Per questo molti sconsigliano l'ereditarietà multipla, in quanto i benefici sono pochi mentre i problemi legati a una semantica ben definita sono molti.

3.8.9 Ereditarietà: la visibilità protetta

La relazione di ereditarietà introduce un ulteriore livello di visibilità: quella *protetta* (*protected*). Una caratteristica, un attributo o un metodo, con visibilità protetta può essere vista solo da una classe e da tutte le classi del package e dalle classi discendenti (anche in altri package).

Consideriamo il seguente esempio.



In questo esempio la classe *ContoCorrenteConFido* potrà accedere al dato saldo, mentre ciò non sarà permesso ad altre classi dislocate in altri package che non ereditano da *ContoCorrente*.

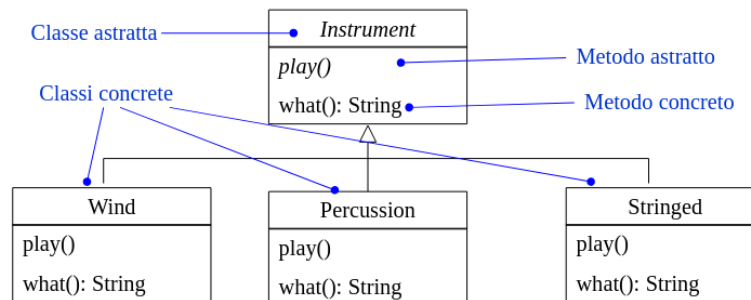
3.9 Classi astratte

Se in programmazione object-oriented non può esistere un oggetto senza che sia stata creata la classe di appartenenza, è invece possibile che esistano classi per le quali non è possibile generare delle istanze. Esse sono chiamate *classi astratte*.

Una classe astratta **può essere** una classe non completamente specificata. In particolare, non è definito il metodo corrispondente ad un'operazione, si dice che il metodo è *astratto*.

Una classe astratta non può essere istanziata perchè il comportamento dei suoi oggetti non sarebbe completamente definito. La relazione di ereditarietà riguarda anche le classi astratte.

In UML, la notazione di una classe astratta è la seguente.



In questo esempio, la classe degli strumenti musicali è astratta perchè non possiamo definire il metodo *play*. Il metodo *play* è implementato nelle sottoclassi degli strumenti a fiato, degli strumenti a percussione, e dei cordati. Il metodo *what* è invece **concreto** (è implementato) e restituisce il nome della classe. Esso è ridefinito in ogni classe.

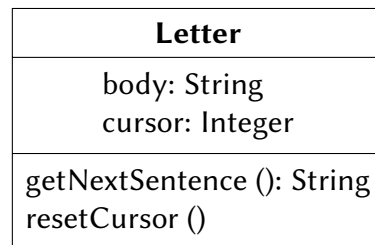
Le classi astratte sono strumenti per fattorizzare proprietà comuni tra classi simili e poterle organizzare in una gerarchia di ereditarietà. Non potremo mai creare oggetti a partire da una classe astratta, ma possiamo servircene per dare una

radice comune a un insieme di classi che condividono le stesse proprietà e poter quindi sfruttare il polimorfismo di inclusione e il binding dinamico.

Le classi astratte fungono da **serbatoi di ereditarietà**.

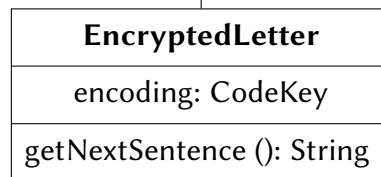
Consideriamo il seguente esempio.

concrete class



← concrete operation

concrete class

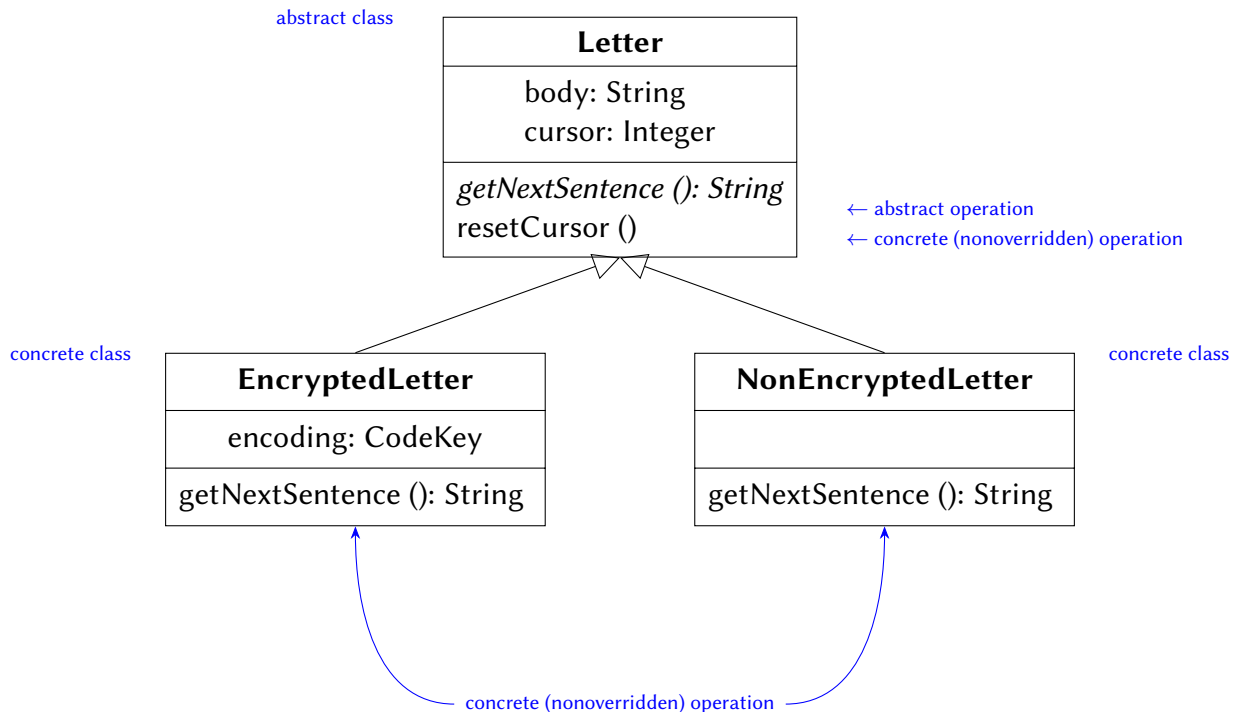


← concrete (overridden) operation

La classe *Letter* ha un metodo che restituisce la frase suggestiva da leggere e l'operazione *resetCursor* che riporta all'inizio del testo. La sottoclasse *EncryptedLetter* rappresenta una lettera crittografata. Il metodo *getNextSentence* è stato sovrascritto perchè il testo dev'essere decrittato prima di essere restituito.

L'implementazione dell'operazione *getNextSentence* è completamente diversa da quella ereditata. Il problema è che il progettista di *EncryptedLetter* non ha alcuna informazione per comprendere quale metodo di *Letter* dev'essere sovrascritto.

Pertanto, tramite l'utilizzo di classi astratte, si ottiene:



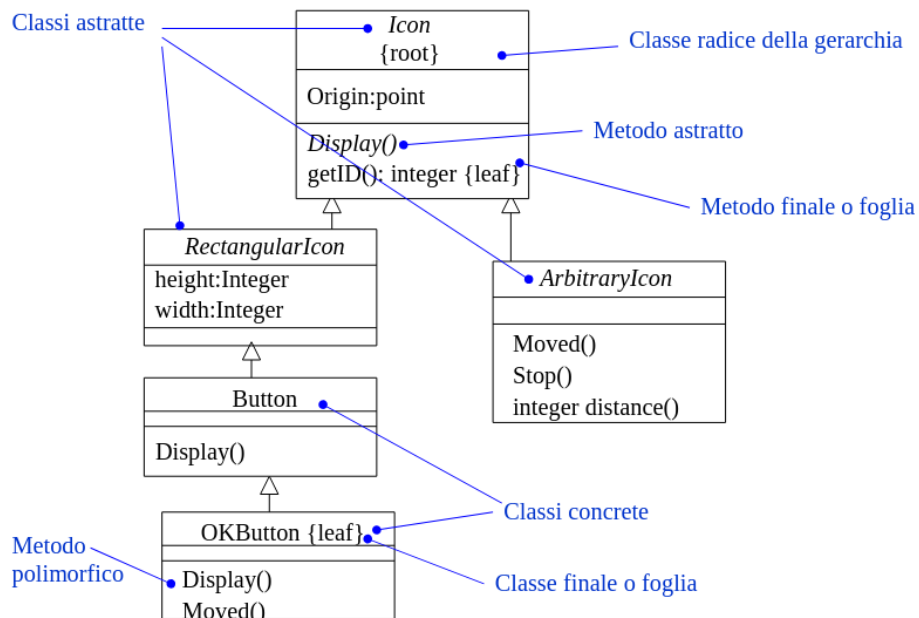
Se il metodo `getNextSentence` di *Letter* fosse astratto il progettista verrebbe informato della necessità di ridefinire il metodo. La classe astratta *Letter* fattorizza i metodi concreti comuni alle sue sottoclassi, come `resetCursor`.

3.10 Classi finali

Una classe è detta finale, *final*, o foglia (*leaf*), quando non può essere ulteriormente specializzata, e quindi non può essere modificata. Si definisce una classe foglia quando il comportamento della classe dev'essere ben stabilito per ragioni di **affidabilità**.

La dichiarazione di una classe foglia permette anche la generazione di codice ottimizzato in quanto facilita l'espansione in linea del codice, che è impossibile nel caso di metodi sovrascrivibili nelle sottoclassi.

Il seguente diagramma UML riassume tutte le notazioni utilizzate per classi.



3.11 Interfacce

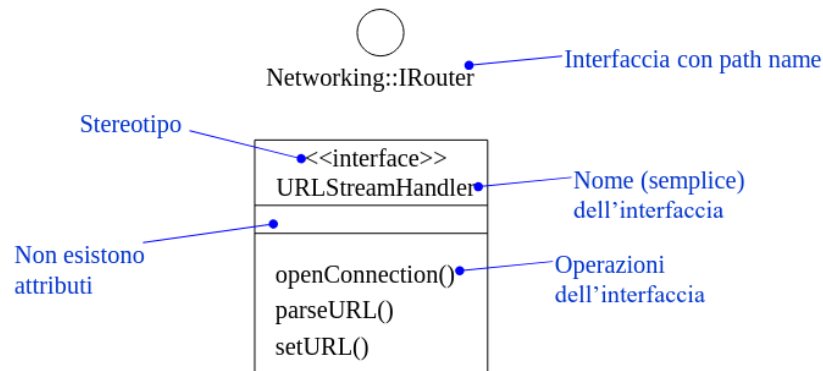
Un'interfaccia è la descrizione del comportamento degli oggetti senza specificarne un'implementazione. Essa è una collezione di **operazioni**, cioè di servizi che possono essere richiesti, priva di informazioni sulle implementazioni dei servizi (i metodi).

Similmente ad una classe, un'interfaccia può avere un qualsiasi numero di operazioni.

Diversamente da una classe, un'interfaccia non specifica una struttura (non saranno inclusi attributi, se non statici) e non fornisce un'implementazione (non saranno quindi specificati i metodi che implementano le operazioni).

Un'interfaccia è simile a una classe astratta i cui metodi sono tutti astratti e non dispone di attributi.

Nella sua rappresentazione più generale, un'interfaccia è rappresentata mediante l'utilizzo di un cerchio, sopprimendo la visualizzazione delle operazioni. Tuttavia, se lo si ritiene importante per la comprensione del modello, è possibile modellare un'interfaccia come una classe stereotipata.



3.11.1 Relazione di realizzazione

Una o più classi possono realizzare/implementare le operazioni indicate in un'interfaccia. La relazione che si stabilisce fra un'interfaccia e una classe che la implementa è detta *relazione di realizzazione*. In UML è indicata con una freccia tratteggiata.



La relazione di realizzazione si presenta come un valido strumento per scindere la specifica di un “contratto” (cosa una classe deve implementare) e la sua implementazione (come si rendono i servizi del contratto).

La relazione di realizzazione unisce il “cosa” (indicato nell'interfaccia) al “come” (indicato nella classe).

Esempio

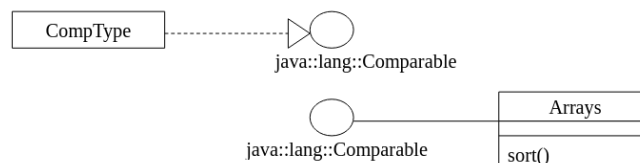
L'interfaccia *java.lang.Comparable* specifica una sola operazione: *public int compareTo(Object o)* che prende un oggetto generico *Object* come argomento e restituisce un valore negativo se l'argomento è più piccolo dell'oggetto corrente, zero se è uguale e un valore positivo se è maggiore.

Una qualunque classe che implementa questa interfaccia deve fornire un'implementazione per *compareTo*.

Le interfacce servono a *disaccoppiare la definizione delle operazioni dalla loro implementazione*. Per poter usare un certo oggetto è sufficiente conoscere la sua interfaccia: non serve conoscere l'implementazione.

Ad esempio, il metodo *sort* della classe *Arrays*, che ordina un array di oggetti utilizzando l'algoritmo di ordinamento per fusioni successive (mergesort), necessita solo di sapere che gli elementi dell'array offrono il servizio *compareTo*, in modo da poterli confrontare. Non serve sapere "come" si realizza il servizio, cioè come sono effettivamente confrontati gli oggetti. Si crea così una dipendenza di implementazione fra la classe *Arrays* e l'interfaccia *Comparable*.

UML permette di rappresentare in modo compatto le seguenti relazioni:

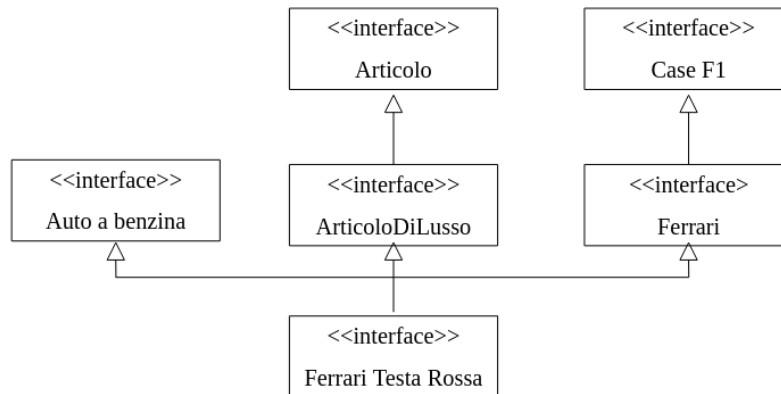


attraverso la notazione:



3.11.2 Ereditarietà

Anche le interfacce possono ereditare altre interfacce. Poiché non ci sono implementazioni, la relazione di ereditarietà è una relazione di generalizzazione "*is_a*".



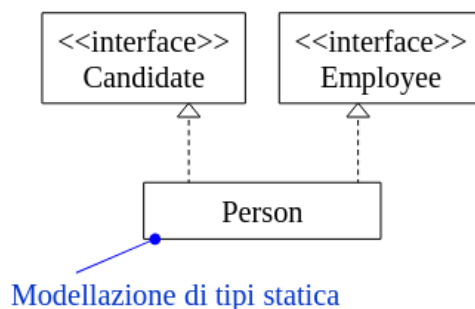
Poichè non si considerano le implementazioni delle operazioni, l'ereditarietà multipla su interfacce non pone problemi di conflitto di realizzazione. Per questo alcuni linguaggi di programmazione, come Java, permettono l'ereditarietà singola sulle classi mentre permettono l'ereditarietà multipla sulle interfacce.

Questo distingue le classi astratte (per le quali l'ereditarietà multipla può porre dei problemi riguardo ai metodi implementati) dalle interfacce.

3.11.3 Realizzazione di più interfacce

Poichè non possono sorgere problemi di conflitto di realizzazione, è permesso a una classe di realizzare più interfacce, per di più non correlate da una relazione di generalizzazione.

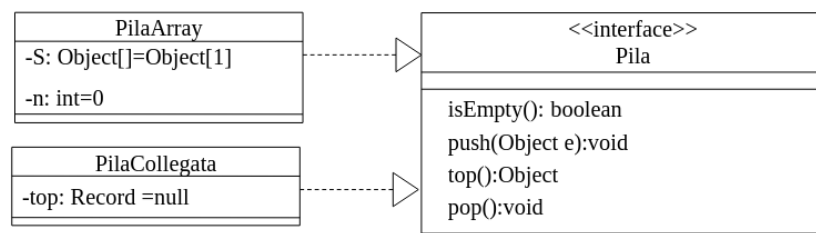
Ad esempio, una classe *Person* può implementare due interfacce:



In questo modo tutto il codice scritto per *Candidate* e *Employee* può essere utilizzato su oggetti di classe *Person*.

3.11.4 Realizzazioni multiple di un interfaccia

Infine, più classi possono implementare la stessa interfaccia. È questo il caso di un'interfaccia definita per dati astratti molto utilizzati nello sviluppo del software, come pile, code, liste, alberi e grafi, per i quali sono possibili molteplici realizzazioni che variano per l'efficienza degli operatori.



Il codice applicativo che intende fare uso di una pila dichiarerà delle variabili di tipo *Pila*, svincolandosi dalla specifica realizzazione. Questa sarà specificata solo al momento della inizializzazione della variabile. Si garantisce così una forte invarianza ai cambiamenti delle realizzazioni di una pila.

3.12 Aggregazione di oggetti

L'ereditarietà offre molti vantaggi, ma non tutti gli oggetti si ottengono bene derivandoli da altri oggetti. Spesso un oggetto è ottenuto *aggreagando* altri oggetti.

Esempio

Ricorrendo all'ereditarietà multipla è allettante definire una classe *Automobile* partendo da classi *Carrozzeria*, *Sedile*, *Ruota* e *Motore*. Si tratta di un errore concettuale poiché l'ereditarietà multipla permette di definire due oggetti per fusione e non per composizione. Un'automobile è composta da una carrozzeria, da un motore, da sedili e da ruote; ma il suo comportamento non è in alcun caso l'unione dei comportamenti di queste differenti parti.

Nell'esempio specifico, sembrerebbe che il ricorso all'ereditarietà di implementazione sia la soluzione. L'ereditarietà di implementazione permette il riuso della

sola implementazione ma non dei comportamenti. Ma neanche questa forma di ereditarietà è adatta allo scopo. Infatti, ereditando dalla classe *Ruota* si ha la possibilità di rappresentare e manipolare una sola ruota, mentre un'automobile ha quattro ruote.

Una *composizione di oggetti* può essere rappresentata permettendo alle variabili di istanza di una classe di puntare a oggetti di altre classi. Infatti si era detto che:

Lo stato di un oggetto può anche contenere il riferimento ad un altro oggetto. Si dice che un oggetto punta ad un altro. Il puntamento è asimmetrico.

Si possono stabilire legami con più istanze di una classe che descrive un componente (per esempio, più ruote).

La relazione che si stabilisce in questo modo fra le classi detta di **aggregazione** o **composizione** (o relazione “*has_a*”).

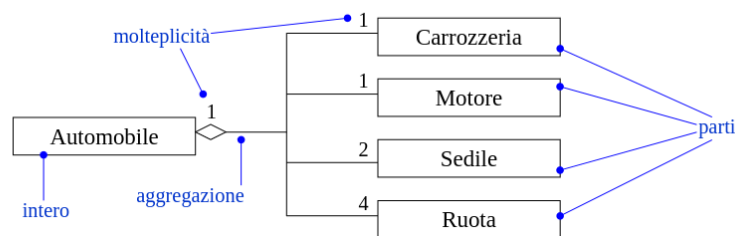
Essa è quindi un'altra possibile relazione fra le classi, diversa dall'ereditarietà (e della generalizzazione o “*is_a*”).

Una classe *A* si dice essere in relazione di aggregazione con una classe *B* quando alcune istanze di *B* contribuiscono a formare una parte delle istanze di *A*.

L'aggregazione è, come l'ereditarietà, una relazione asimmetrica.

3.12.1 Notazione in UML

In UML l'aggregazione è resa mediante una linea piena e un piccolo rombo dalla parte del contenitore.



La **multiplicità** consente di indicare quanti oggetti possono essere aggregati.

3.12.2 Utilizzo dell'aggregazione

L'utilizzo dell'aggregazione è suggerito nelle seguenti situazioni.

1. **Contenimento fisico:** la pagine di un libro.
2. **Appartenenza** (membership): il giocatore di una squadra di calcio.
3. **Composizione funzionale:** le ruote di un'automobile.

Si deve osservare che l'aggregazione **non implica una dipendenza esistenziale**: un'automobile può essere distrutta ma alcune sue parti possono essere riutilizzate. Una squadra di calcio può fallire, ma i suoi giocatori non vengono “soppressi”.

3.13 La composizione

Le aggregazioni sono associazioni deboli fra parti e intero. Questo significa che le parti possono esistere senza l'intero.

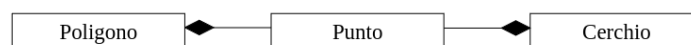
Un'associazione forte fra parti e intero è detta **composizione** e si rappresenta in UML mediante un rombo pieno. La composizione comporta una **dipendenza esistenziale**, in quanto le parti non esistono senza il contenitore. Ciò presuppone che:

1. Creazione e distruzione delle parti avvengano nel contenitore.
2. I componenti non siano parti di altri oggetti.

In questi esempi, la vita delle “parti” che compongono l’“intero” dipende dalla vita dell'intero stesso:

1. i dipartimenti non esisterebbero se non esistesse la società;
2. il tempo di vita dei diversi oggetti grafici d'interazione dipende da quello della finestra che li contiene.

Consideriamo il seguente diagramma.



Un'istanza di *Punto* può essere parte di un poligono oppure il centro di un cerchio, ma non entrambe le cose. Questa è detta **regola di non condivisione**: benchè una classe possa essere componente di molte altre classi, ogni sua istanza può essere componente di un solo oggetto.

Un diagramma delle classi può mostrare più classi di potenziali possessori di oggetti componenti, ma ogni sua istanza di componente deve appartenere a un solo oggetto possessore. Questa regola è caratterizzante della composizione.

La composizione fra classi stabilita in fase di progettazione offre delle informazioni importanti al programmatore che andrà a implementare le classi. Ad esempio, se il programmatore utilizzerà C++ farà sì che gli oggetti componenti finiscano nello stack e non nell'heap, non userà puntatori/new.

Se utilizza il Java, che pone tutto nell'heap, si preoccuperà di allocare i componenti nella classe contenitore e di non passare mai all'esterno della classe il riferimento ai componenti, in modo da impedirne la condivisione con oggetti di altre classi. Quindi, il componente sarà privato e non ci sarà alcun metodo che restituisce il suo riferimento.

3.13.1 Differenze tra ereditarietà e aggregazione

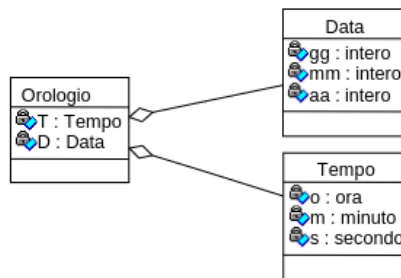
In molti casi è possibile associare due classi mediante ereditarietà o aggregazione/composizione e la sua scelta non è immediata come negli esempi visti precedentemente.

Esempio

Si supponga di disporre di una classe *Data* e di una classe *Tempo*, che rappresentano e manipolano, rispettivamente, date e istanti di tempo. Volendo definire una nuova classe *Orologio* per rappresentare sia le date che gli istanti di tempo, si può:

1. derivare *Orologio* da *Data* e *Tempo*, se l'ereditarietà multipla è permessa.
2. Comporre *Orologio* con le due classi.

Sono anche possibili soluzioni ibride (ereditarietà da una classe e composizione con l'altra).



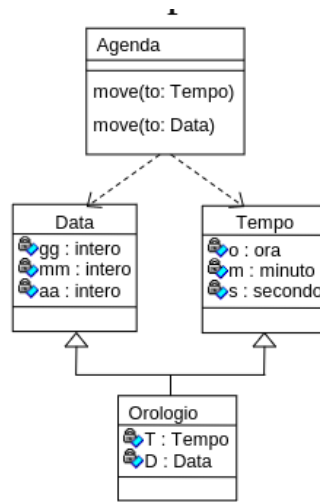
Vediamo le differenze tra le due soluzioni sopra elencate.

- Nell'ereditarietà, ogni oggetto o della classe *Orologio* contiene tutti i campi definiti nelle classi *Data* e *Tempo* e può accedervi direttamente.
- Nell'aggregazione/composizione, ogni oggetto della classe *Orologio* contiene due campi di tipi *Data* e *Tempo* rispettivamente. Per accedere all'informazione sul giorno occorrerà invocare un metodo (per esempio *day()*) sull'oggetto *D*.
- Se l'ereditarietà corrisponde a una relazione di generalizzazione (*is_a*) vale il polimorfismo di inclusione, cioè ogni oggetto di *Orologio* è utilizzabile come un'istanza di *Data* e *Tempo*, mentre questo non è vero nel caso della composizione.

Le scelte di progetto sono totalmente diverse.

- L'ereditarietà permette di poter “dimenticare” il fatto che **gg** è definito nella classe *Data* e di utilizzarlo direttamente come campo di *Orologio*. Questa caratteristica della derivazione è particolarmente utile se si vogliono utilizzare più derivazioni successive e creare vere e proprie gerarchie di classi. In questo caso non è necessario conoscere a quale livello della gerarchia è definito un dato campo, ma è possibile utilizzarlo direttamente come se fosse un elemento della classe stessa.
- L'ereditarietà permette di riutilizzare tutti i metodi delle varie classi che operano su oggetti delle classi *Data* e *Tempo*.

Consideriamo il seguente schema.



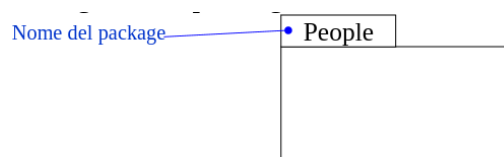
Agenda ha due metodi che possono operare anche su istanze di *Orologio*. Il codice dei metodi polimorfi *move* è riutilizzabile per istanze di *Orologio*.

Il meccanismo di aggregazione/composizione è generalmente usato quando si vogliono utilizzare i servizi di una classe predefinita ma non la sua interfaccia. L'ereditarietà di implementazione, qualora non dovesse essere permessa da un linguaggio di programmazione, potrebbe essere resa da una relazione di aggregazione/composizione.

3.14 Raggruppare classi

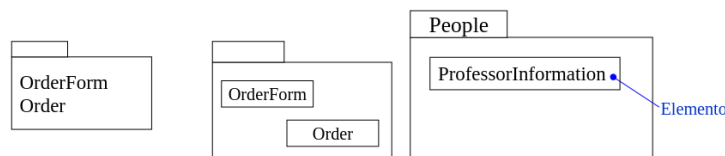
La mole di classi aumenta in modo considerevole all'aumentare della complessità del sistema da modellare. È importante, quindi, organizzare tali classi in gruppi separati al fine di rendere più facilmente individuabili, e quindi accessibili, le singole parti che compongono il nostro sistema.

I *package* sono un meccanismo generale per organizzare le classi in gruppi. In UML un package è rappresentato come segue:



In genere i package si usano per riunire classi, ma nella notazione UML essi possono includere qualsiasi costrutto UML e possono essere persino eterogenei, ad esempio, possono contenere classi e interfacce.

È consentito mostrare il contenuto di un package sia testualmente sia graficamente:

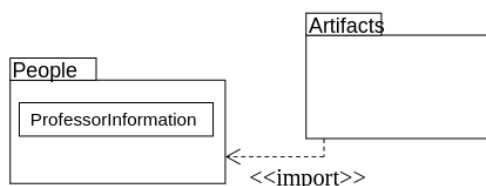


Un package definisce un **namespace** (spazio degli identificatori) per i suoi elementi. Questo significa che ogni classe di un package di classi deve avere un nome distinto all'interno del package che la racchiude. Il nome completo, o **qualificato**, della classe sarà ottenuto indicando prima il nome del package che la contiene mediante una notazione ::, ad esempio, *People :: ProfessorInformation*.

Elementi della stessa specie, inseriti all'interno di uno stesso package, devono avere necessariamente nomi differenti. In generale è possibile avere nomi uguali per elementi di tipo differente. Ad esempio è possibile avere una classe o interfaccia *Cliente* e un package *Cliente*, ma non due classi *Cliente*, due interfacce *Cliente* o una classe *Cliente* e un'interfaccia *Cliente*.

È possibile avere nomi uguali per elementi della stessa specie se sono inseriti in package differenti.

Per evitare la necessità di utilizzare nomi qualificati, un package può importare gli elementi o il contenuto di un altro package nel proprio namespace. Un elemento nel package che importa può quindi riferirsi a un elemento importato come se esso fosse definito direttamente nel package. L'import di un package viene ottenuto stereotipando la relazione di dipendenza.



Nota

La relazione di importazione non è transitiva. Se A importa B e B importa C , allora A può usare gli elementi di B e B può usare gli elementi di C , ma non è detto che A possa usare gli elementi di C , a meno che A non importi anche C .

Se c'è conflitto di nomi in due elementi importati, nessuno dei due elementi è aggiunto al namespace.

Se il nome di un elemento importato è in conflitto con il nome di un elemento definito localmente a un package, il nome dell'elemento interno ha precedenza sul nome importato che non viene aggiunto al namespace, che sarà raggiungibile mediante qualificazione.

I package possono essere innestati senza alcun limite di profondità, anche se tale possibilità deve comunque essere utilizzata con parsimonia se non si vuole compromettere la leggibilità del modello.

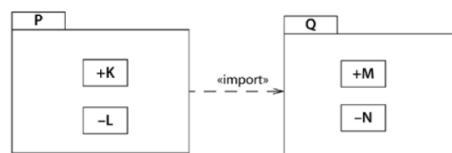
Un package innestato ha accesso a tutti gli elementi contenuti direttamente nei package esterni, senza necessità di importazione.

Si può anche specificare la **visibilità** degli elementi di un package:

- **public (+)** sono visibili ad altri elementi del package stesso, a uno dei package innestati o a package che li importano;
- **private (-)** non sono visibili all'esterno del package.

Esempio

Consideriamo il seguente schema.



Il package P importa il package Q . Le classi K e L nel package P possono usare M come nome di classe senza qualificarlo poiché M è pubblica, ma non possono vedere la classe N , in quanto privata. Le classi M e N nel package Q possono usare il nome qualificato $P :: K$ per referenziare la classe pubblica K nel package P , ma non possono vedere in alcun modo la classe L .

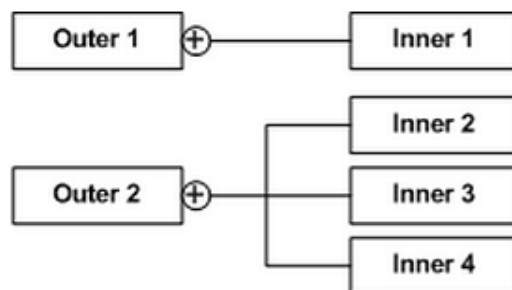
3.15 Classi interne

Una classe interna, chiamata *inner class*, è una classe la cui dichiarazione si trova all'interno di un'altra classe ospite, detta *top level class*.

Le classi interne sono identificate da un nome al pari delle classi top level. Una classe interna può essere privata. In tal caso non è visibile all'esterno della classe ospite. (In Java, una classe top level non può mai essere privata).

La inner class può accedere a tutti i metodi e i campi della classe ospitante, mentre la classe ospitante può vedere solo la parte pubblica della inner class. Un oggetto di classe inner non può esistere se non esiste un oggetto della classe ospitante. Inoltre, una classe interna non può avere campi statici.

UML non ha una notazione standard per le classi interne. Tuttavia, si è suggerita la seguente notazione.



3.16 Il polimorfismo

Con il termine polimorfismo si intende la possibilità di associare a un'operazione diverse realizzazioni, chiamati *morfismi*. Il concetto di polimorfismo è stato definito in vari modi. Strachey ha distinto due tipi principali di polimorfismo.

- **Polimorfismo parametrico:** è ottenuto quando una funzione lavora uniformemente su una gamma di tipi. Questi tipi normalmente esibiscono una qualche struttura comune.
- **Polimorfismo ad hoc:** è ottenuto quando una funzione lavora, o sembra lavorare, su tipi differenti (che potrebbero non esibire una struttura comune) e potrebbe comportarsi in modo scorrelato per ciascuno di essi.

Una classificazione più raffina dei diversi meccanismi di polimorfismo è data da Cardelli e Wegner.

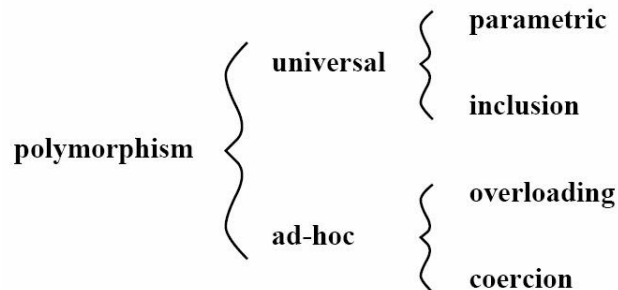


Figure 1: Varieties of polymorphism.

La classificazione di Cardelli e Wegner introduce una nuova forma di polimorfismo, quello *per inclusione*, al fine di modellare i concetti di sottotipo e di ereditarietà. Il polimorfismo per inclusione e parametrico sono classificati come due sottocategorie del *polimorfismo universale*, che è posto in contrasto al polimorfismo non universale o ad hoc.

L'idea di polimorfismo universale è quella di poter operare su un numero infinito di tipi, a patto che essi rispettino alcuni vincoli.

- **Universale:**

- Il polimorfismo è su un numero potenzialmente illimitato di tipi.
- I diversi morfismi sono generati automaticamente.
- C'è una base unificante, comune a tutti i diversi morfismi che può assuere l'entità polimorfa.

- **Ad hoc:**

- Il polimorfismo è su un numero finito di tipi, spesso pochissimi.
- I diversi morfismi sono generati in modo manuale o semimanuale.
- Non c'è una base comuni a tutti i morfismi, al di là delle intenzioni del progettista.

3.16.1 La coercizione

La *coercizione* è il meccanismo di conversione implicita operata da un compilatore per applicare un operatore definito per oggetti tipo T_1 anche a oggetti di tipo T_2 .

Ad esempio, l'operatore $+$, $3.14 + 5$, è definito per valori reali, ma lo si può usare su un insieme di tipi più grande di quello per il quale è stato definito. Senza coercizione, avremmo un errore di tipo.

Altri esempi di coercizione sono, in Java, l'autoboxing di un *int* in un *Integer* e l'unboxing di un *Integer* in un *int*.

Le coercizioni possono essere stabilite staticamente, inserendole automaticamente fra gli argomenti e le funzioni al momento della compilazione, oppure potrebbero essere determinate dinamicamente da test al run-time sugli argomenti. La coercizione è la forma di polimorfismo più semplice. Essa opera a un livello *semantico*, cioè cambiando la rappresentazione del dato.

Pascal e Ada supportano questa modalità di polimorfismo. Tuttavia, in questi linguaggi di polimorfismo è più un'eccezione che la regola. Pertanto sono considerati per lo più monomorfi.

3.16.2 L'overloading

Si ha polimorfismo per *overloading* quando si usa lo stesso identificatore per metodi differenti e si ricorre a informazione di contesto per decidere quale metodo è denotato da una particolare occorrenza dell'identificatore. La disambiguazione necessaria per una corretta compilazione si basa sul tipo degli argomenti del metodo o sulla classe dell'oggetto a cui si richiede il servizio.

Possiamo immaginare che una pre-compilazione del programma potrebbe disambiguare ed eliminare l'overloading dando nomi differenti a metodi differenti. In questo senso, l'overloading è giunto una conveniente abbreviazione *sintattica*.

L'overloading è presente nella maggior parte dei linguaggi, anche imperativi, dove gli operatori aritmetici $+$, $*$, $-$ ecc. sono applicabili a più di un tipo. Ad esempio, in Pascal, l'operatore $+$, se applicato a 2 interi genera un'istruzione *add* fra interi, mentre se applicato a 2 numeri reali genera un'istruzione *add* fra floating point.

In Ada, come in C++, è possibile ridefinire alcuni operatori, ad esempio l'operatore `+`, per operare su tipi/classi definite dall'utente. Ad esempio, l'operatore `+`, può essere ridefinito per numeri complessi, rappresentabili mediante un record.

Un esempio di overloading di operatori in C++.

```
1 class Rational {  
2     public: Rational(double);  
3     const Rational& operator+(const Rational& other);  
4     ...  
5 };
```

Un esempio di overloading di funzioni in C++.

```
1 double max(double d1, double d2);  
2 char max(char c1, char c2);  
3 char* max(char* c1, char* c2);  
4 const char* max(const char* c1, const char* s2);
```

L'overloading può efficacemente interagire con la coercizione. Ad esempio, l'operatore `+` può essere definito per due interi e due reali, ragione per cui le espressioni:

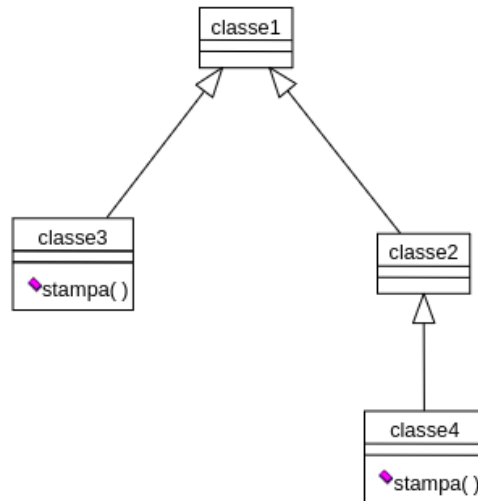
$$3 + 4$$
$$3.0 + 4.0$$

non causano errore.

Combinando l'overloading di `+` con il meccanismo di coercizione, non avremo errore di tipo anche nei seguenti casi:

$$3 + 4.0$$
$$3.0 + 4$$

nel paradigma a oggetti si ha overloading anche nel caso di funzioni con medesimo nome ma definite in classi non correlate gerarchicamente. Ad esempio, consideriamo il seguente schema.



In questo esempio l'informazione di contesto utilizzata per disambiguare è la classe di appartenenza dell'istanza sulla quale si invoca il metodo stampa. Questo polimorfismo è **meno potente** di quello universale, perchè l'unicità del nome del metodo nasconde una molteplicità di implementazioni.

3.16.3 Il polimorfismo parametrico

Nel polimorfismo parametrico, una funzione polimorfa ha un parametro di tipo esplicito o implicito, che determina il tipo dell'argomento per ciascuna applicazione della funzione. Le funzioni che esibiscono il polimorfismo parametrico sono anche dette **funzioni generiche**.

Una funzione generica può lavorare su argomenti di molti tipi, generalmente esibendo lo stesso comportamento indipendentemente dal tipo dell'argomento.

Le funzioni *generic* dell'Ada ma anche le funzioni *template* del C++ sono un esempio di funzioni generiche.

```
1 template <class T> void sort(vector<T>);
2
3 vector<complex> cv(100);
4 vector<int> c1(200);
5 void f(vector<complex> &cv, vector<int> &ci) {
6     sort(cv);
7     sort(ci);
8 }
```

Si deve osservare che il polimorfismo parametrico di Ada, ma anche di C++, è *sintattico*, dal momento che un'istanziatura generica è effettuata al momento della compilazione sulla base dei tipi effettivi che devono essere determinabili al compile-time. Pertanto, le procedure generica di Ada o di C++, possono essere considerate delle abbreviazioni per insiemi di procedure monomorfe. Rispetto al polimorfismo parametrico propriamente inteso, esse hanno il vantaggio che si può generare del codice ottimizzato per forme di input differenti.

Al contrario, nei veri sistemi polimorfici, il codice è generato **una sola volta** per ogni procedura generica, come avviene in Java.

3.16.4 Il polimorfismo di inclusione

Il polimorfismo di inclusione, nasce da una relazione di inclusione fra insiemi di valori. L'inclusione di insiemi di valori è tipico dei *sottotipi*.

Ad esempio, in Pascal, ogni valore di un subrange di intero può essere usato nel contesto di un intero. Tuttavia, il polimorfismo di inclusione non è necessariamente legato ai sottotipi.

Degli esempi di polimorfismo per inclusione built-in sono:

- Pascal: il valore Nil appartiene a tutti i tipi puntatore.
- C: Il valore 0 è polimorfo. Appartiene a tutti i tipi puntatore.
- C++: Il tipo void* è un supertipo di tutti i tipi puntatore. Un qualunque puntatore può essere assegnato a un void*.

Nella programmazione orientata a oggetti, si ha polimorfismo per inclusione se un oggetto appartiene a una classe e a tutte le sue superclassi. Esso si manifesta in almeno due modi:

- Si può assegnare un oggetto di una qualsiasi sottoclasse di una classe C a una variabile definita di classe C.
- Una funzione che opera su un oggetto di classe C può essere applicata anche a oggetti di classe C', sottoclasse di C.

Questi due utilizzi del polimorfismo di inclusione sono limitatamente interessanti. Non a caso, essi non sono molto dissimili dall'utilizzo che permette il concetto di sottotipo anche nella programmazione imperativa.

Il legame statico/dinamico L'utilizzo più interessante si ha quando le invocazioni dei metodi su oggetti di classe diverse produce un comportamento differente, anche se la definizione della funzione è unica. Ciò dipende dal tipo di legame statico/dinamico fra identificatore di funzione e relativa realizzazione.

Nella maggior parte dei linguaggi di programmazione la visibilità degli identificatori e dei legami dei nomi alle dichiarazioni è determinata al momento della compilazione. Si parla di **ambito d'azione statico**.

Nell'ambito d'azione **dinamico**, il legame fra l'uso di un identificatore e la sua dichiarazione dipende dall'ordine di esecuzione, e così è differito al momento dell'esecuzione.

Si consideri il seguente programma Pascal.

```
1 program dynamic(input, output);
2 var x: integer;
3 procedure A;
4 begin
5     ...; write(x); ...
6 end; {A}
7 procedure B;
8 var x: real;
9 begin
10     ...; A; ...
11 end; {B}
12 begin
13     ...; B; ...; A; ...
14 end {dynamic}
```

Poichè il Pascal adotta la regola dell'ambito statico, l'uso della variabile x in A è legato alla variabile intera x nel programma principale. Questo permette di tradurre la $write(x)$ semplicemente in una chiamata a funzione di libreria I/O per la scrittura degli interi.

Tuttavia, se il legame nome-dichiarazione fosse dinamico, l'uso di x in A sarebbe vincolato dalla dichiarazione di x più recente. Pertanto, quando la procedura A è chiamata dalla procedura B , l'uso di x in A viene vincolato dalla dichiarazione della variabile reale x nella procedura B , mentre quando A è chiamata nel programma principale, l'uso di x sarebbe vincolato alla dichiarazione della variabile intera x nel programma principale.

Legame e tipizzazione Con un dynamic-binding, la traduzione della chiamata $write(x)$ può essere determinata solo al run-time. Questo non vuol dire che il controllo del tipo non possa essere effettuato, ma solo che viene ritardato al momento dell'esecuzione, quando è noto il tipo al quale x è vincolato.

Si osservi che con lo static-binding il legame dei nomi ai tipi è anch'esso fissato al momento della compilazione.

Nella programmazione orientata a oggetti il legame fra nome di funzione e sua realizzazione può essere determinato al run-time. Si consideri il seguente esempio.

```
1 class A{
2     public: void f() { cout << "A"; }
3 };
4 class B : public A {
5     public: void f() { cout << "B"; } // Overriding
6 };
7
8 class Tester {
9     public void someF(A a){ a.f(); ... }
10 };
11
12 A obj1;
13 Tester obj2;
14 ...
15 obj1 = new B();
16 obj2 = new Tester();
17 obj2.someF(obj1);
18 ...
```

Il comportamento di `someF()` dipende da quando si effettua il legame fra l'identificatore di funzione f in `someF` e la sua implementazione. In C++ è possibile definire la funzione f come **virtuale**. Questo significa che il legame deve essere dinamico e quindi verrà stampata una B. Diversamente, il legame sarebbe statico e verrebbe stampata una A. La selezione del metodo, nel caso di funzioni virtuali, avviene al run-time.

Combinando polimorfismo di inclusione con legame dinamico si hanno i vantaggi di **estendibilità** del codice.

C++ è un esempio di linguaggio dove la tipizzazione è statica, ma il legame può essere dinamico (*statically-typed dynamic binding*). Il legame si riferisce a nome di funzione con la sua realizzazione.

Lo stesso dicasi per il Java. Ma in questo linguaggio, il legame dinamico è la regola, non l'eccezione. Solo nei metodi final/static il legame nome di funzione con la sua realizzazione diventa statico.

Consideriamo il seguente codice

```
1 void f(int) { cout << "int"; }
2 void f(char) { cout << "char"; }
3 void f(char*) { cout << "char*"; }
4
5 g() {
6     f(0);
7 }
```

Questa codice combina overloading, coercizione e polimorfismo di inclusione. Il compilatore C++ associa al valore 0 il tipo int, quindi la funzione *g* stampa *int*. Anche lo 0 può essere visto come carattere, ma l'associazione di tipo diretta tra 0 e int, ha precedenza rispetto alla coercizione con char o char*.

Diversi linguaggi di programmazione OO proibiscono la ridefinizione dei metodi e la coercizione per evitare questi problemi.

Capitolo 4

Il linguaggio Java

Il linguaggio Java presenta diverse peculiarità che possiamo riassumere nel seguente elenco.

- Semplicità.
- Object-Oriented.
- Solidità.
- Multi-threading.
- Indipendenza dall'architettura.
- Interpretazione con elevate prestazioni.
- Possibilità di lavorare in modo distribuite.
- Dinamicità.

4.1 Indipendenza dall'architettura

Un programma Java può essere eseguito su architetture diverse. Esiste un'indipendenza a livello di:

- *codice sorgente*: tipi di dati primitivi completamente specificati, ordine di valutazione delle espressioni rigorosamente definito.

- *bytecode*: un programma Java viene compilato in un codice intermedio detto **bytecode**. I file di codice intermedio sono interpretati dalla *Java Virtual Machine (JVM)*. È sufficiente implementare una JVM specifica per ogni piattaforma, senza per questo dover modificare il bytecode.

L'implementazione della JVM non è a carico dell'utente. La JVM è pertanto un'emulazione software di un'architettura hardware ideale in grado di eseguire bytecode in Java.

4.2 Interpretazione con elevate prestazioni

Il bytecode può essere interpretato su qualsiasi sistema che offre una Java Virtual Machine. Altri sistemi interpretati come Basic e Perl presentano deficit di prestazioni insormontabili. Pur avendo delle prestazioni inferiori ai linguaggi direttamente compilati, il bytecode di Java è stato progettato in modo tale da poter essere tradotto senza difficoltà in codice macchina nativo garantendo così delle prestazioni abbastanza elevate.

I sistemi run-time Java che eseguono questa ottimizzazione “just in time” o “on-fly” non perdono nessuno dei vantaggi offerti dal codice indipendente da piattaforma e allo stesso tempo hanno delle buone prestazioni.

Nota

La compilazione just-in-time è oggetto di ricerca. I compilatori convenzionali sono pensati per produrre codice altamente ottimizzato **senza prestare attenzione al tempo di compilazione**.

4.3 Ambienti di sviluppo: JDK

Esistono diversi ambienti di sviluppo per Java, il primo, denominato *Java Development Kit (JDK)* e messo a disposizione dalla Sun, è il più diffuso. Ogni altro ambiente di sviluppo per Java aggiunge funzionalità, nuovi tool, nuove classi, pur avendo come piattaforma base un Java Development Kit.

Un tool fornito dalla JDM è la tecnologia *Java Archive (Jar)*, un formato file indipendente dalla piattaforma che aggrega più file in un singolo package.

Sun struttura la piattaforma Java in due edizioni:

- Java Runtime Environment (JRE);

- Java Software Development Kit (Java SDK).

La prima è rivolta a utenti di applicazioni Java, la seconda è rivolta agli sviluppatori di applicazioni Java.

La JRE permette di eseguire applicazioni Java per cui non include il compilatore e altre componenti di sviluppo.

La Java SDK mette a disposizione degli sviluppatori il compilatore, le librerie, i sorgenti, le specifiche API e, in generale, tutto quanto è necessario per lo sviluppo di applicazioni Java.

L'edizione SDK si distingue in:

- Java 2 Standard Edition J2SE;
- Java 2 Enterprise Edition J2EE;
- Java 2 Micro Edition J2ME.

La prima rappresenta la piattaforma Java standard e include tutti gli applicativi e le librerie per lo sviluppo di applicazioni desktop, applet e client-server.

Gli strumenti di base presenti nel JDK sono:

- **javac** che è il compilatore Java; item **java** che è l'interprete Java.

4.4 Oggetti in Java

Ogni linguaggio di programmazione ha i propri mezzi per manipolare i dati. Il programmatore deve essere costantemente consapevole del tipo di manipolazione che si sta effettuando. In particolare, trattando gli oggetti siamo interessati a sapere se *si sta manipolando direttamente un oggetto, o si sta trattando qualche tipo di rappresentazione indiretta (un puntatore in C o C++) che dev'essere assoggettata a una qualche sintassi specifica.*

In C++ sono possibili entrambi i casi. Tutto ciò è semplificato in Java. In Java tutto viene trattato come oggetto in modo tale da avere una sintassi consistente da usare in ogni situazione. Sebbene ciò sia sempre vero, l'identificatore che si va a manipolare è in realtà il *riferimento* ad un oggetto. Questo comporta che *quando si ha un riferimento, non è detto che ci sia necessariamente un oggetto ad esso legato.*

Esempio

Se abbiamo bisogno di una stringa allora si può creare un riferimento del tipo:

```
String box;
```

Se a questo punto si vuole mandare un messaggio a box si otterrà un errore al run-time perchè non abbiamo inizializzato la stringa

```
String box = "abcd";
```

Una volta creato un riferimento, è possibile connetterlo ad un nuovo oggetto mediante l'operatore **new**.

```
String box = new String("abcd")
```

che oltre a creare un nuovo oggetto di tipo String, indica anche "come" creare l'oggetto in questione.

4.5 Tipi primitivi

In Java abbiamo una grande varietà di tipi predefiniti ai quali si aggiungono quelli che è possibile creare da sé. In Java i tipi primitivi godono di un trattamento particolare.

Ciò è dovuto al fatto che la creazione mediante l'operatore *new* non è molto efficiente, visto che viene allocata nell'*heap*.

Per i tipi primitivi, Java si affida all'approccio C/C++ in cui viene creata una variabile (contenente il valore assegnato) memorizzata nello *stack*, anziché un riferimento ad essa come avviene con l'operatore *new*.

Lo *stack* si serve di uno stack pointer per allocare e rilasciare memoria. Ciò ne fa un modo estremamente veloce ed efficiente per allocare memoria secondo solo ai registri. Il compilatore deve sapere, mentre alloca il programma, l'esatta dimensione e tempo di vita di tutti i dati contenuti nello stack in modo da generare il codice per gestire lo stack pointer. Questo vincolo limita la flessibilità dei programmi, così accade che mentre i riferimenti agli oggetti sono nello stack, gli oggetti non possono esservi memorizzati.

La *heap* è una memoria generale dove vivono gli oggetti Java. Quando si crea un oggetto con l'operatore *new*, viene allocata memoria nell'*heap* a run time. Non si

ha il vincolo posto sullo stack, ma il prezzo da pagare per la maggiore flessibilità è la quantità di tempo necessaria per l'allocazione.

Il progetto del linguaggio Java definisce la dimensione di ogni tipo primitivo e tale dimensione non varia da un tipo di piattaforma ad un'altra garantendo in tal modo la portabilità.

Type	Description	Default	Size	Example Literals
boolean	true or false	false	1 bit	true, false
byte	twos complement integer	0	8 bits	(none)
char	Unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\\', '\'', '\n', 'ß'
short	twos complement integer	0	16 bits	(none)
int	twos complement integer	0	32 bits	-2, -1, 0, 1, 2
long	twos complement integer	0	64 bits	-2L, -1L, 0L, 1L, 2L
float	IEEE 754 floating point	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F
double	IEEE 754 floating point	0.0	64 bits	1.23456e300d, -1.23456e-300d, 1e1d

Tutti i tipi numeri sono con segno, detti **signed**. I tipi unsigned del C/C++ non sono supportati. La dimensione del tipo *boolean* non è esplicitamente definita. È specificato che deve solo contenere valori letterali *true* o *false*.

I tipi di dati primitivi possono avere una classe *wrapper* associata per poter avere oggetti corrispondenti allocati nell'heap.

4.6 Numeri ad alta precisione

Java include due classi per l'aritmetica ad alta precisione:

- *BigInteger*: supporta gli interi a “arbitrary-precision” ovvero è possibile rappresentare accuratamente valori integrali di qualsiasi dimensione, senza perdita di informazione durante le operazioni.
- *BigDecimal*: supporta numeri decimali in notazione fixed-point a “arbitrary-precision”. Possono essere utilizzati per accurati calcoli monetari.

Entrambe possiedono dei metodi analoghi agli operatori disponibili per i tipi primitivi `int` e `float`.

4.7 Variabili in Java

In Java esistono tre tipi di variabili:

1. *variabili di istanza*: definiscono gli attributi e lo stato di un oggetto;
2. *variabili di classe*: definiscono gli attributi e lo stato di una classe (si dichiarano con `static`)
3. *variabili locali*: utilizzate nella definizione dei metodi.

La dichiarazione di una variabile in Java presenta la seguente forma:

DICHIARAZIONE = TIPO + NOME

ove:

- NOME:
 - può iniziare con una lettera, trattino, segno di sottolineatura, \$;
 - non può cominciare con una cifra;
- TIPO:
 - uno tra gli otto tipi primitivi;
 - nome di una classe o interfaccia;
 - un array.

All'atto di una dichiarazione è anche possibile provvedere alla inizializzazione della variabile.

4.8 Creazione di nuovi tipi di dato

Se in Java tutto può essere considerato un oggetto, per rappresentare il fatto che un insieme di oggetti sono accomunati dalle medesime caratteristiche e comportamenti si utilizza il concetto di *classe* della programmazione object-oriented.

In Java per definire una classe è usata la parola chiave `class` seguita dal nome della classe.

Esempio : Creazione di una classe in Java

```
1 class Box {  
2     double width;  
3     double height;  
4     double depth;  
5 }
```

Nota

Una dichiarazione `class` non crea l'oggetto ma introduce un nuovo tipo, un modello. Per avere l'oggetto dobbiamo usare l'operatore **new**. Ad esempio:

```
1 Box a = new Box();
```

Dove *a* è un riferimento ad un oggetto della classe *Box* ovvero *a* è un'istanza della classe *Box*

Quando si definisce una classe è possibile specificare:

- *Componente dati* o *attributi*, per definire lo **stato** di un oggetto;
- *Componente funzioni* o *metodi* per definire il **comportamento** di un oggetto.

Le *componenti dati* possono essere tipi primitivi oppure dei riferimenti ad altri oggetti. Se la componente dato è un tipo primitivo allora può essere inizializzata direttamente nel punto di definizione all'interno della classe. Nel caso in cui la componente dato non è un tipo primitivo deve essere necessariamente inizializzata attraverso un metodo apposito chiamato **costruttore**.

Esempio : Uso del costruttore

```
1 class DataOnly {  
2     int i;  
3     float f;  
4     boolean b;
```

```

5 }
6 ...
7 /* dot notation: uso dell'oggetto */
8 DataOnly mydate = new DataOnly()
9 mydate.i = 47;
10 mydate.f = 1.1f;
11 mydate.b = false;
12 ...

```

Quando un tipo di dato primitivo è membro di una classe Java ci assicura la sua inizializzazione ad un valore di default nel caso non venga inizializzato dall'utente.

Primitive type	Default
boolean	false
char	'\u0000' (null)
byte	(byte) 0
short	(short) 0
int	0
long	0L
float	0.0f
double	0.0d

Se la variabile è locale, Java non la inizializza ad un valore di default pertanto la mancata inizializzazione verrà segnalata come errore nel momento della compilazione.

I metodi in Java sono dei messaggi che permettono agli oggetti di comunicare fra loro. Un metodo in Java deve essere necessariamente definito in una classe, **non esistono metodi stand-alone**. Lo schema per la definizione di un metodo Java è il seguente

```

1 tipo_risultato NomeMetodo(/*lista di argomenti*/) {
2     /* corpo del metodo */
3     return espressione;
4 }

```

ove:

- `tipo_risultato`: indica il tipo primitivo o la classe dell'oggetto restituito dal metodo;
- `lista di argomenti`: lista dei tipi e dei nomi passati al metodo. I parametri di tipo oggetto sono passati per riferimento mentre quelli di tipo primitivo sono passati per valore.
- `return`: è una parola chiave utilizzata per la restituzione del valore elaborato. Se il tipo da restituire è `void` allora `return` ha solo la funzione di uscire dal metodo.

Esempio

```
1 class Prova {
2     boolean flag() { return true; }
3     float logBaseNaturale() { return 2.718f; }
4     void non_fa_nulla() { return; }
5     void non_fa_nulla2() { }
6     ...
7     boolean flag = false;
8     Prova objProva = new Prova();
9     /* flag = false */
10    flag = objProva.flag();
11    /* flag = true */
12 }
```

Un metodo è identificato univocamente dal suo nome e dalla lista di argomenti. Un metodo può essere chiamato solo per un oggetto che deve essere a sua volta capace di eseguirne la chiamata. Quando il tipo di ritorno è `void`, la parola chiave `return` è usata solo per uscire dal metodo, tuttavia non è necessaria nel caso in cui si esca solo dalla fine del metodo.

Costruttori Prima di poter essere usati, gli oggetti devono essere creati ovvero *inizializzati*. Ciò è fatto attraverso un particolare metodo della classe chiamato **costruttore**. Java infatti richiama automaticamente (con l'operatore `new`) il rispettivo costruttore non appena l'oggetto è stato creato e prima che tale oggetto sia a disposizione dell'utente.

Analogamente al C++, anche in Java il costruttore assume il nome della classe a cui appartiene. Se abbiamo una classe priva di costruttore, il compilatore Java ne crea uno in maniera automatica.

Esempio

```
1 class Rock {
2     Rock() {
3         System.out.println("Creating Rock");
4     }
5 }
6
7 public class SimpleConstructor {
8     public static void main(String[] args) {
9         for (int i = 0; i < 10; i++) {
10             new Rock();
11         }
12     }
13 }
```

Quando si crea l'oggetto Rock, viene allocata memoria e richiamato il rispettivo costruttore. L'effetto è quello di stampare su console la stringa "creating Rock".

Un costruttore può avere anche degli argomenti.

Esempio : Costruttore con argomenti

```
1 class Rock2 {
2     Rock2(int i) {
3         System.out.println("Creating Rock number " + i);
4     }
5 }
6
7 public class SimpleConstructor2 {
8     public static void main(String[] args) {
9         for(int i = 0; i < 10; i++) {
10             new Rock2(i);
11         }
12     }
13 }
14 }
```

Distruttori A differenza del C++, in Java non si riscontra il concetto di **distruttore** di oggetti, per la presenza di un metodo ordinario per ripulire la memoria chiamato *garbage collector*.

4.9 Overloading dei metodi

L'overloading dei metodi è essenziale per permettere a metodi che hanno lo stesso nome di essere usati con differenti tipi di argomento in modo tale da supportare il polimorfismo. È indispensabile usare l'overloading nella definizione dei costruttori in quanto il nome è protetto. Sebbene tale funzionalità sia obbligatoria per i costruttori, generalmente conviene utilizzarla anche nella definizione dei metodi comuni.

Esempio : Utilizzo dell'overloading

```
1 // Demonstration of both constructor and ordinary method overloading.
2 import java.util.*;
3 class Tree {
4     int height;
5     Tree() {
6         System.out.println("Planning a seedling");
7         height = 0;
8     }
9     Tree(int i) {
10        System.out.println("Creating new Three that is " + i + " feet
11        tall");
12        height = i;
13    }
14    void info() {
15        System.out.println("Tree is " + height + " feet tall");
16    }
17    void info(String s) {
18        System.out.println(s + ": Tree is " + height + " feet tall");
19    }
20 }
21
22
23 public class Overloading {
24     public static void main(String[] args) {
25         for (int i = 0; i < 5; i++) {
26             Tree t = new Tree(i);
27             t.info();
28             t.info("overloaded method");
29         }
30         // Overloaded constructor
31         new Tree();
32     }
33 }
```

4.9.1 La parola chiave `this`

La parola chiave `this` è usata all'interno di un metodo per mantenere il riferimento all'oggetto in uso corrente. La parola chiave `this` è anche utilizzata per risolvere i conflitti di nome che potrebbero verificarsi tra variabili di istanza e variabili locali. Se `this` si trova in un costruttore ed ha una lista di argomenti di input, effettua una chiamata esplicita al costruttore che corrisponde a quella data lista di parametri.

Esempio : Il significato di `this`

```

1 public class Flower {
2     int petalCount = 0;
3     String s = new String("null");
4     Flower(int petals) {
5         petalCount = petals;
6         System.out.println("Constructor w/ int arg only, petalCount = " +
7             petalCount);
8     }
9     ...
10    Flower(String s, int petals) {
11        this(petals);
12        // !this(s); // Can't call two!
13        this.s = s;
14        System.out.println("String & int args");
15    }
16    Flower() {
17        this("hi", 47);
18        System.out.println("default constructor (no args)");
19    }
20
21    void prn(String s, int petals) {
22        this.s = s;
23        System.out.println(s + petals);
24    }
25
26    public static void main(String args[]) {
27        Flower a = new Flower(2);
28        Flower b = new Flower("seconda", 2);
29        a.prn("a: ", 2);
30        b.prn("b: ", 3);
31        ...

```

```
31 }  
32 }
```

4.10 File e classi Java

Un file contiene codice sorgente Java, detto comunemente *unità di compilazione* o *unità di traduzione*, deve sempre avere estensione *.java*.

Esso può contenere più definizioni di classi **ma solo una può essere public**. Tale classe deve avere lo stesso nome del file (rispettando le maiuscole ma escludendo l'estensione *.java*). Pertanto se il file contiene il programma principale di un'applicazione, questo dovrà necessariamente essere il main della classe **public**. Le altre classi sono **nascoste** al mondo esterno perchè si intendono solo di supporto alla classe **public**.

Quando si compila un file Java si otterrà in output un file (contenente esattamente lo stesso nome ma con estensione *.class*) per ogni classe presente nel file *.java*. Un programma funzionante è praticamente un insieme di file *.class* che possono essere “impacchettati” e eventualmente compressi in un JAR file (usando l'applicazione **jar** fornita insieme al JDK). Sarà compito dell'interprete Java occuparsi del ritrovamento, caricamento e interpretazione di questi file.

4.10.1 Creazione dei package

Come già osservato, per raggruppare delle classi Java è possibile ricorrere alla loro inclusione in un unico file. In alternativa, Java permette di raggruppare delle classi mediante clausola `package`. Ad esempio, per raggruppare un insieme di classi in un package denominato `mypackage` basterà inserire la clausola:

```
package mypackage;
```

all'inizio di ogni file Java contenente le classi da raggruppare.

Nota

Per convenzione, i nomi dei package si definiscono con le lettere minuscolo.

I package predefiniti della JDK sono:

<i>Package</i>	<i>Fornisce il supporto per</i>
<code>java.applet</code>	creare applet
<code>java.swing</code>	creare interfacce grafiche
<code>java.io</code>	eseguire funzioni di I/O
<code>java.lang</code>	funzioni generiche
<code>java.math</code>	eseguire calcoli matematici
<code>java.net</code>	comunicare attraverso una rete
<code>java.rmi</code>	creare applicazioni distribuite
<code>java.security</code>	imporre restrizioni sulla sicurezza
<code>java.sql</code>	interagire con basi di dati in SQL
<code>java.text</code>	formattare il testo per la visualizzazione
<code>java.util</code>	Servizi generali

Per importare un'intera libreria predefinita di Java si usa la clausola `import`. Ad esempio:

```
import java.util.*;
```

Volendo invece importare solo una classe della libreria, ad esempio *ArrayList*, si dovrebbe scrivere:

```
import java.util.ArrayList;
```

Se all'interno del codice ci si vuole riferire a tale classe, occorre specificare solo *ArrayList* se si è precedentemente importato il package altrimenti si deve indicare l'intero percorso `java.util.ArrayList`;

Nota

Le classi del package `java.lang` sono automaticamente rese disponibili durante la scrittura di un programma, cioè esse sono importate automaticamente.

4.10.2 Lo spazio dei nomi delle classi

La ragione alla base della scelta di importare i package è fornire un meccanismo di gestione dello “spazio dei nomi delle classi”. Sappiamo che i nomi di tutti i membri delle classi sono isolati gli uni dagli altri. Un metodo `f()` della classe *A*

non entrerà mai in conflitto con un metodo `f()` (avrete la stessa lista di argomenti) della classe *B*. A distinguerli sarà l'oggetto sul quale sono invocati. Come possiamo ottenere un risultato simile per i nomi delle classi? Questo problema è particolarmente avvertito per quelle applicazioni che possono essere ottenute via Web. Infatti, può accadere facilmente che si abbia un “conflitto di nomi” fra classi prese automaticamente da Internet e classi disponibili localmente. La soluzione sta appunto nel concetto di package.

Per esempio, si supponga di aver creato un package di un solo file `MyClass.java`. Questo significa che ci deve essere un'unica classe `public` in quel file e che il nome della classe dev'essere `MyClass`

```
1 package mypackage;  
2 public class MyClass {  
3 // ...
```

Se qualcuno volesse usare `MyClass` nel proprio programma potrebbe usare il nome in modo completamente specificato

```
1 mypackage.MyClass m = new mypackage.MyClass();
```

La clausola `import` può rendere ciò più semplice:

```
1 import mypackage.*;  
2 // ...  
3 MyClass m = new MyClass();
```

4.10.3 Package e cartelle

Poichè un package si compone di più file `.class` è preferibile organizzarli in una cartella distinta, sfruttando l'organizzazione gerarchica in cartelle disponibile nei moderni sistemi operativi. Questa soluzione risolve altri due problemi:

1. la creazione di nomi dei package unici (poichè associati alla cartella) e
2. il ritrovamento di quelle classi che potrebbero essere disperse in qualche cartella imprecisata.

Questo è ottenuto codificando il percorso della locazione del file `.class` nel nome del package.

L'interpreta Java procede nell'ordine seguente:

1. trova la variabile d'ambiente CLASSPATH (impostata manualmente oppure attraverso i tool Java-based sulla macchina parametro - cp dell'interprete). CLASSPATH contiene una o più cartelle che sono usate come root per la ricerca dei file *.class*
2. partendo dalla root, prende il nome del package e sostituisce ogni punto con uno slash per generare un nome di percorso per la root del CLASSPATH (in modo tale che package mypackage.util diventi mypackage.util dipendentemente dal sistema operativo)
3. Concatena la stringa ottenuta ai percorsi contenuti nel CLASSPATH dove accede per cercare il file *.class* con il nome corrispondente alla classe che si sta cercando di creare.

Abbiamo appena visto che per realizzare una libreria si può ricorrere al costrutto package del Java. Ma come si realizza effettivamente il concetto di “information hiding”? Per chiarire questo aspetto occorre spiegare il concetto di **modificatore** (o **specificatore**) **di accesso**, che si applica tanto alle proprietà di una classe (dati e funzioni membro) quanto alle classi stesse.

4.11 I modificatori di accesso in Java

I modificatori di accesso in Java sono i seguenti:

- public;
- private;
- firendly (o package access);
- protected.

Questi modificatori vanno inseriti subito prima della definizione di ogni membro della classe (sia esso funzione o dato) in modo che ciascun modificatore controlla l'accesso ad un solo membro.

Questo è diverso da quello che accade in C++ in cui un modificatore controllava l'accesso di tutti i membri che seguivano fino a che non si indicava un altro modificatore diverso.

4.11.1 Il modificatore di accesso `friendly`

Se non si indica alcun modificatore per un determinato membro, Java gli assegna un accesso di default che non è rappresentato da alcuna parola chiave sebbene ci si riferisce ad esso come `friendly` o `package access`. Ciò significa che tutte le classi del package corrente hanno accesso al membro `friendly` che invece appare come `private` alle classi fuori dal package. Di conseguenza si dice che questi elementi hanno un `package access`. L'accesso `friendly` permette di raggruppare insieme delle classi correlate in un package cosicché possano facilmente interagire tra loro. In Java il codice di un package non può autonomamente accedere a quello di un altro.

Il solo modo di concedere l'accesso ad un membro è:

- rendere il membro `public` in modo che tutti da qualsiasi parte possono accedervi;
- rendere il membro `friendly` non specificando alcun modificatore e inserire le altre classi nello stesso package in modo tale che queste possano accedervi;
- una sottoclasse può accedere, oltre ai membri pubblici, ai membri `protected` delle superclassi e ai `friendly` di un'altra classe solo se le due classi sono nello stesso package;
- nel caso di membri dato, fornendo dei metodi “accesso/mutator” (noti anche come metodi “get/set”) che leggono e cambiano il valore.

Se i file sorgente Java non importano alcun package particolare e risiedono nella stessa cartella sono implicitamente trattati come un “package di default” per quella cartella e pertanto sono *friendly* tra loro (se non viene specificato alcun modificatore di accesso).

4.11.2 Il modificatore di accesso `public`

Se un membro della classe è preceduto dalla parola chiave `public`, significa che è accessibile a chiunque.

Esempio : Utilizzo del modificatore di accesso `public`

```
1 // Creare una libreria
2 package c05.dessert;
```

```

3 public class Cookie {
4     public Cookie() {
5         System.out.println("Cookie constructor");
6     }
7     void bite() { System.out.println("bite"); }
8 }
9
10 // Usa la liberira
11 import c05.dessert.*;
12 public class Dinner {
13     public Dinner() {
14         System.out.println("Dinner constructor");
15     }
16
17     public static void main(String[] args) {
18         Cookie x = new Cookie();
19         // !x.bite(); // Non si puo' accedere
20     }
21 }

```

4.11.3 Il modificatore di accesso private

La parola chiave `private` è usata per rendere il membro accessibile solo dai metodi della classe in cui è stato dichiarato. Visto che un package potrebbe essere creato da più programmatori, rendendo alcuni membri (metodi o attributi) della propria classe `private` si ha la certezza che le eventuali modifiche che potrebbero riguardare la classe non andranno a coinvolgere le altre classi presenti nel package.

Esempio : Utilizzo del modificatore di accesso private

```

1 // Dimostrazione di dichiarazioni private.
2 class Sundae {
3     private Sundae() {}
4     static Sundae makeASundae() {
5         return new Sundae();
6     }
7 }
8
9 public class IceCream {
10     public static void main(String[] args) {
11         // ! Sundae x = new Sundae();
12         Sundae x = Sundae.makeASundae();
13     }

```

```
14 }
```

Potrebbero essere resi private tutti quei metodi che fungono solo da supporto per quella classe in modo tale da impedirne l'uso accidentale all'interno del package e di conseguenza proibire la modifica o l'eliminazione del metodo. Discorso analogo vale per gli attributi private delle classi. Se non si hanno particolari esigenze, si dovrebbero rendere private tutti i membri dato della classe.

L'uso del private è importante specialmente nel multithreading.

4.11.4 Il modificatore di accesso protected

Se un metodo o un attributo sono preceduti dalla parola chiave `protected`, significa che tali caratteristiche sono accessibili sia nella classe in cui sono stati dichiarati che in una qualsiasi sottoclasse della stessa in qualunque package sia, nelle classi che sono nello stesso package della classe in cui è stata dichiarata.

Per esprimere la relazione di ereditarietà tra una sottoclasse e la classe base, Java introduce la parola chiave `extends`. Pertanto per indicare che una classe eredita metodi e attributi da una classe esistente si dirà che la nuova classe estende una classe esistente.

Esempio : Utilizzo del modificatore di accesso protected

```
1 // La parola chiave protected
2 package example;
3 import java.util.*;
4 public class A {
5     private int i;
6     protected int read() { return i; }
7     protected void set(int ii) { i = ii; }
8     public A(int ii) { i = ii; }
9     public int value(int m) { return m * i; }
10 }
11
12 package example2;
13 import example.A;
14 public class B extends A {
15     private int j;
16     public B(int jj) { super(jj); j = jj; }
17     public void change(int x) { set(x); }
18 }
```

Se si crea un nuovo package e si eredita da una classe in un altro package, i soli membri accessibili sono quelli pubblici del pacchetto originale e quelli `protected` della classe base.

Invece, i membri preceduti dalla parola chiave `final` non possono essere estesi ovvero non possono essere ridefiniti nelle sottoclassi.

4.11.5 Accesso alle classi

In Java i modificatori di accesso sono utilizzati anche per determinare quali classi nella libreria sono accessibili agli utenti di quella libreria. Poichè un'unità di compilazione può appartenere ad un singolo package, tutte le classi in una singola unità di compilazione sono automaticamente *friendly* tra loro. Se si vuole che la classe sia accessibile al programmatore bisogna dichiararla *public* nel seguente modo:

```
1 public class MyClass {  
2     // body  
3 }
```

Nota

Si nota che una classe non può essere dichiarata `private` o `protected`.

Se si vuole che nessuno possa accedere alla classe si possono rendere i costruttori `private`.

Esempio

```
1 // Classe privata mediante i costruttori privati  
2  
3 class Soup {  
4     private Soup() {}  
5     // Permette la creazione mediante un metodo di classe  
6     public static Soup makeSoup() {  
7         return new Soup();  
8     }  
9  
10    /*  
11     Crea un oggetto e restituisce un riferimento su richiesta.  
12     Un esempio di classe "singoletto"  
13     */  
14  
15    private static Soup ps1 = new Soup();  
16    public static Soup access() {
```

```

17         return ps1;
18     }
19
20     public void f() {}
21 }
22
23 class Sandwich {
24     // Uso di Lunch
25     void f() { new Lunch(); }
26 }
27
28 public class Lunch {
29     void test() {
30         // Can'to do this! Private constructor:
31         // ! Soup priv1 = new Soup();
32         Soup priv2 = Soup.makeSoup();
33         Sandwich f1 = new Sandwich();
34         Soup.access().f();
35     }
36 }

```

4.12 Le interfacce

Spesso ci si riferisce al controllo dell'accesso come *implementation hiding*. Il wrapping di dati e metodi nelle classi in aggiunta all'*implementation hiding* viene chiamato **incapsulamento**. Il risultato di quest'operazione è un tipo di dato fornito di caratteristiche e comportamento. Il controllo dell'accesso permette essenzialmente di:

- stabilire cosa possono e non possono fare i programmatori utente;
- separare l'interfaccia dall'implementazione.

Attraverso la parola chiave `interface` Java permette di astrarre completamente un'interfaccia dalla relativa implementazione specificando che **cosa** una classe deve fare ma non **come** deve farlo. Esse forniscono dei *template* degli schemi per la definizione delle classi.

Le interfacce sono simili alle classi ma privi di variabili di istanza e i loro metodi sono privi di corpo. Per implementare un'interfaccia, una classe deve reare l'insieme dei metodi definiti in tale interfaccia con ampia libertà di implementazione.

Le interfacce aggiungono gran parte delle funzionalità necessarie per molte applicazioni, che solitamente utilizzerebbero invece l'ereditarietà multipla in C++.

Esempio : Interfaccia

```
1 access interface InterfaceName {
2     returnType NameOfMethod1(argument-list);
3     returnType NameOfMethod2(argument-list);
4     // ...
5     returnType NameOfMethodN(argument-list);
6     final static type finalNameVar1 = value;
7     final static type finalNameVar2 = value;
8     // ...
9     final static type finalNameVarN = value;
10 }
```

access può essere public oppure può essere l'accesso di default per cui l'interfaccia è disponibile solo nel pacchetto in cui è definita. I metodi specificati in un'interfaccia sono sempre considerati public, e quindi vanno implementati come public.

I metodi dichiarati sono privi di corpo ovvero sono astratti. Toccherà alla classe che implementerà l'interfaccia, definire i rispettivi metodi. Le variabili possono essere indicate nella dichiarazione di interfaccia e sono implicitamente *final* e *static* (non possono essere modificate dalla classe di implementazione) e devono essere inizializzate ad un valore costante.

Se l'interfaccia viene dichiarata *public* allora anche tutti i metodi e le variabili sono considerate implicitamente *public*.

Esempio

```
1 public interface drive {
2     void firstgear();
3     void secondgear();
4     void thirdgear();
5 }
```

Affinchè una o più classi possano implementare un'interfaccia, Java mette a disposizione la parola chiave `implements`.

La sintassi generale è la seguente:

```
1 access class ClassName [extends BaseClass] [implements interface1[,
2     interface2, ... ]]
```

```
3 // body
4 };
```

I metodi che implementano un'interfaccia devono essere dichiarati *public* e inoltre devono corrispondere esattamente a quelli dichiarati nella definizione dell'interfaccia.

4.13 Il riuso delle classi in Java

Una delle caratteristiche principali di Java è la possibilità di riusare il codice. Anziché creare delle classi completamente nuove si possono usare delle classi esistenti che sono già implementate e testate. Esistono due modi di procedere.

1. Nel primo modo semplicemente si creano oggetti per la classe esistente all'interno della nuova classe. Questa modalità è chiamata **composizione** perché la nuova classe è composta di oggetti delle classi esistenti in modo tale che si riusa la funzionalità del codice.
2. Il secondo approccio si basa sul meccanismo di **ereditarietà**, uno dei principi fondamentali della programmazione OO.

4.13.1 Il meccanismo di composizione

Tale meccanismo è molto diffuso. Come già detto consiste nell'inserire i riferimenti agli oggetti in una nuova classe. Si può notare che mentre i tipi primitivi sono automaticamente inizializzati a 0, i riferimenti agli oggetti sono inizializzati a `null`. Il compilatore, infatti, non deve creare degli oggetti di default altrimenti si potrebbero avere costi computazionali aggiuntivi indesiderati.

```
1 class WaterSource {
2     private String s;
3     WaterSource() {
4         System.out.println("WaterSource()");
5         s = new String("Constructed");
6     }
7
8     public String toString() { return s; }
9 }
10
11 public class SprinklerSystem {
12     private String valve1, valve2, valve3, valve4;
```

```

13   WaterSource source;
14   int i;
15   float f;
16   void print();
17   System.out.println("valve1 = " + valve1);
18   System.out.println("valve2 = " + valve2);
19   System.out.println("valve3 = " + valve3);
20   System.out.println("valve4 = " + valve4);
21   System.out.println("i = " + i);
22   System.out.println("f = " + f);
23   System.out.println("source = " + source);
24 }
25
26 public static void main(String[] args) {
27     SprinklerSystem x = new SprinklerSystem();
28     x.print();
29 }

```

Se si vuole un riferimento inizializzato, si può fare nei seguenti punti:

- nel punto in cui gli oggetti sono definiti, quindi saranno sempre inizializzati immediatamente appena è richiamato il costruttore;
- nel costruttore della classe;
- giusto prima di quando si ha bisogno di usare l'oggetto. Questo approccio è spesso chiamato *lazy initialization*. Esso può ridurre i costi aggiuntivi in situazioni dove l'oggetto non deve essere creato molte volte.

```

1  class Soap {
2      private String s;
3      Soap() {
4          System.out.println("Soap()");
5          s = new String("Constructed");
6      }
7      public String toString() {
8          return s;
9      }
10 }
11
12 public class Bath {
13     private String s1 = new String("Happy"), s2 = "Happy", s3, s4;
14     Soap castille;
15     int i; float toy;
16     Bath() {
17         System.out.println("Inside Bath()");

```

```

18     s3 = new String("Joy"); i = 47; toy = 3.14f;
19     castille = new Soap();
20 }
21
22 void print() {
23     if (s4 == null) s4 = new String("Joy");
24     System.out.println("s1 = " + s1);
25     System.out.println("s2 = " + s2);
26     System.out.println("s3 = " + s3);
27     System.out.println("s4 = " + s4);
28     System.out.println("i = " + i);
29     System.out.println("toy = " + toy);
30     System.out.println("castille = " + castille);
31 }
32
33 public static void main(String[] args) {
34     Bath b = new Bath();
35     b.print();
36 }
37 }

```

4.13.2 Il meccanismo di ereditarietà

L'ereditarietà è uno dei concetti fondamentali di Java (e di tutti i linguaggi objected-oriented in generale). Esso entra in gioco ogni qualvolta che si crea una classe perchè ogni classe in Java eredita dalla classe standard di root *Object* (a meno di esplicitarne una in particolare).

Per indicare che una classe eredita i membri da un'altra classe (chiamata **classe base**) si è già detto che Java mette a disposizione la parola chiave `extends`.

```

1 class Cleanser {
2     private String s = new String("Cleanser");
3     public void append(String a) { s += a; }
4     public void dilute() { append("dilute()"); }
5     public void apply() { append("apply()"); }
6     public void scrub { append("scrub()"); }
7     public void print() { System.out.println(s); }
8     public static void main(String[] args) {
9         Cleanser x = new Cleanser();
10        x.dilute(); x.apply(); x.scrub();
11        x.print();
12    }
13 }

```

```

14
15 public class Detergent extends Cleanser {
16     public void scrub() {
17         append("Detergent.scrub()");
18         super.scrub();
19     }
20
21     public void foam() { append("foam()"); }
22     public static void main(String[] args) {
23         Detergent x = new Detergent();
24         x.dilute();
25         x.apply();
26         x.scrub();
27         x.foam();
28         x.print();
29         System.out.println("Testing base class: ");
30         Cleanser.main(args);
31     }
32 }

```

Si può notare che le classi *Cleanser* e *Detergent* hanno entrambe un `main`. Ciò può essere fatto per testare il codice di ogni singola classe. Quando da linea di comando si vorrà lanciare il metodo `main` della classe *Cleanser* si digiterà: `java Cleanser`.

Se si vorrà invocare il metodo `main` della classe *Detergent* si digiterà: `java Detergent`.

Inoltre si può notare che la classe *Detergent* ha impostato dei metodi nella sua interfaccia: `append()`, `dilute()`, `apply()`, `scrub()` e `print()`.

Ciò è dovuto al fatto che *Detergent* è derivata da *Cleanser* quindi prende automaticamente tutti i metodi della sua interfaccia anche se questi non sono definiti esplicitamente in *Detergent*. Si può pensare all'ereditarietà come ad una sorta di riuso dell'interfaccia. È possibile **ridefinire (override)** i metodi ereditati dalla classe base. In tal caso se volessimo richiamare dalla sottoclasse in metodo `scrub()` definito nella classe base, dobbiamo utilizzare la parola chiave `super` che si riferisce alla superclasse da cui è stata derivata la classe corrente.

È sempre possibile definire nella classe derivata dei nuovi membri, non esistenti nella classe base.

Inizializzazione della classe di base Quando si crea un oggetto di una classe derivata, esso contiene al suo interno un *sotto-oggetto* della classe di base. Questo

sotto-oggetto è identico a quello che si avrebbe se istanziassimo la classe base.

Pertanto è importante che il sotto-oggetto sia inizializzato correttamente attraverso l'invocazione del costruttore della classe base che possiede la conoscenza e privilegi opportuni per farlo. Java inserisce automaticamente la chiamata al costruttore della classe base nel costruttore della classe derivata.

Esempio

```
1 class Art {
2     Art() {
3         System.out.println("Art constructor");
4     }
5 }
6
7 class Drawing extends Art {
8     Drawing() {
9         System.out.println("Drawing constructor");
10    }
11 }
12
13 public class Cartoon extends Drawing {
14     Cartoon() {
15         System.out.println("Cartoon Constructor");
16     }
17
18     public static void main(String[] args) {
19         Cartoon x = new Cartoon();
20     }
21 }
```

L'output sarà:

```
Art constructor
Drawing constructor
Cartoon constructor
```

Si può notare come le chiamate avvengano “dall'esterno verso l'interno” ovvero a partire dalla radice della gerarchia di ereditarietà fino ad arrivare alla chiamata dell'ultimo costruttore. L'oggetto istanza della classe base viene creato prima di quello della sottoclasse. Anche se non si crea un costruttore per la classe *Cartoon*, il compilatore ne prevedrà uno di default (a zero argomenti) che invocherà il costruttore della classe base. Questa aggiunta non si ha se l'utente specifica un qualche costruttore.

Se il costruttore della classe base ha degli argomenti in input allora bisogna esplicitare nella sua invocazione la lista di tali argomenti attraverso l'uso della parola chiave `super`.

Esempio

```
1 class Game {
2     Game(int i) {
3         System.out.println("Game constructor");
4     }
5 }
6
7 class BoardGame extends Game {
8     BoardGame(int i) {
9         super(i);
10        System.out.println("BoardGame constructor");
11    }
12 }
13
14 public class Chess extends BoardGame {
15     Chess() {
16         super(11);
17         System.out.println("Chess constructor");
18     }
19
20     public static void main(String[] args) {
21         Chess x = new Chess();
22     }
23 }
```

Se non ci fosse la chiamata al costruttore della classe base in *BoardGame()*, il compilatore non troverebbe il costruttore *Game()*, segnalando un errore. La chiamata al costruttore della classe base, deve essere la prima cosa da fare nel costruttore della classe derivata, altrimenti si verifica un errore di compilazione.

Nella maggior parte dei casi l'approccio per ereditarietà e per composizione verranno usati insieme.

Esempio

```
1 class Plate {
2     Plate(int i) {
3         System.out.println("Plate constructor");
4     }
5 }
```

```

5 }
6
7 class DinnerPlate extends Plate {
8     DinnerPlate(int i) {
9         super(i);
10        System.out.println("DinnerPlate constructor");
11    }
12 }
13
14 class Utensil {
15     Utensil(int i) {
16        System.out.println("Utensil constructor");
17    }
18 }
19
20 class Spoon extends Utensil {
21     Spoon(int i) {
22        super(i);
23        System.out.println("Spoon constructor");
24    }
25 }
26
27 class Fork extends Utensil {
28     Fork(int i) {
29        super(i);
30        System.out.println("Fork constructor");
31    }
32 }
33
34 class Knife extends Utensil {
35     Knife(int i) {
36        super(i);
37        System.out.println("Knife constructor");
38    }
39 }
40
41 class Custom {
42     Custom(int i) {
43        System.out.println("Custom constructor");
44    }
45 }
46
47 public class PlaceSetting extends Custom {
48     Spoon sp;
49     Fork frk;

```

```

50 Knife kn;
51 DinnerPlate pn;
52 PlaceSetting(int i) {
53     super(i + 1);
54     sp = new Spoon(i + 2);
55     frk = new Fork(i + 3);
56     kn = new Knife(i + 4);
57     pl = new DinnerPlate(i + 5);
58     System.out.println("PlaceSetting constructor");
59 }
60
61 public static void main(String[] args) {
62     PlaceSetting x = new PlaceSetting(9);
63 }
64 }

```

4.14 La parola chiave `final`

La parola chiave `final` ha un significato leggermente differente a seconda del contesto in cui viene inserita. Si applica ai dati, ai metodi e alle classi.

4.14.1 `final` applicato ai dati

Molti linguaggi di programmazione hanno un modo particolare per indicare al compilatore che un dato è “costante”. Una costante è utile per due ragioni:

- può essere una costante a tempo di compilazione che non cambierà mai;
- può essere un valore inizializzato a run-time che non si vuole più cambiare in seguito.

Nel primo caso il compilatore userà subito il valore della costante nei calcoli in cui è richiesta in modo da eliminare alcuni sovraccarichi di lavoro a run-time. In Java le costanti di questo genere devono essere di **tipo primitivo** e sono espresse usando la parola chiave `final`. Inoltre deve essere assegnato loro un valore al momento della definizione.

Quando si usa `final` con i **riferimenti** agli oggetti cambiano leggermente le cose, infatti mentre con i tipi primitivi `final` rende il valore costante, con i riferimenti agli oggetti accade che rende i **riferimenti costanti**. Ciò significa che se si è inizializzato un riferimento ad un oggetto, esso non può più essere modi-

ficato sebbene l'oggetto stesso possa esserlo. Java, infatti, non fornisce un modo per rendere degli oggetti (array inclusi) costanti.

Esempio

```
1 class Value {
2     int i = 1;
3 }
4
5 public class FinalData {
6     final int i1 = 9;
7     static final int VAL_TWO = 99;
8     public static final int VAL_THREE = 39;
9     // Costanti al run-time:
10    final int i4 = (int)(Math.random() * 20);
11    static final int i5 = (int)(Math.random() * 20);
12    Value v1 = new Value();
13    final Value v2 = new Value();
14    static final Value v3 = new Value();
15    final int[] a = {1, 2, 3, 4, 5, 6};
16    public void print(String id) {
17        System.out.println(id + ": i4 = " + i4 + ", i5 = " + i5);
18    }
19
20    public static void main(String[] args) {
21        FinalData fd1 = new FinalData();
22        // !fd1.i1++; //Errore: valore non modificabile
23        fd1.v2.i++;
24        fd1.v1 = new Value();
25        for (int i = 0; i < fd1.a.length(); i++) {
26            fd1.a[i]++; // L'oggetto non e' costante
27        }
28        // ! fd1.v2 = new Value(); // Il riferimento e' costante
29        // ! fd1.v3 = new Value(); // Il riferimento e' costante
30        // ! fd1.a = new int[3];
31
32        fd1.print("fd1");
33    }
34 }
```

Si fa notare che i tipi primitivi preceduti da `final` e `static` insieme, con valori costanti, hanno per convenzione nomi con lettere maiuscole e parole separate da underscore.

Java permette la creazione di *blank final* che sono dati dichiarati come *final* senza

un valore iniziale. Questi devono essere inizializzati esplicitamente prima di essere utilizzati nelle classi e assicurano una maggiore flessibilità nell'uso di `final` visto che, per esempio, ora è possibile avere un valore diverso per ogni oggetto pur mantenendo le caratteristiche fornite da `final`.

```
1 class Poppet {}
2
3 class BlankFinal {
4     final int i = 0;
5     final int j; // Final vuoto
6     final Poppet p; // Riferimento final vuoto
7     // Gli attributi final vuoti devono essere inizializzati nel costruttore
8     BlankFinal() {
9         j = 1;
10        p = new Poppet();
11    }
12
13    BlankFinal(int x) {
14        j = x;
15        p = new Poppet();
16    }
17
18    public static void main(String[] args) {
19        BlankFinal bf = new BlankFinal();
20    }
21 }
```

Java permette di rendere `final` gli argomenti di un metodo. Ciò significa che all'interno del metodo non si possono cambiare i riferimenti a cui gli argomenti puntano.

```
1 class Gizmo {
2     public void spin() {}
3 }
4
5 public class FinalArguments {
6     void with(final Gizmo g) {
7         // !g = new Gizmo(); // Illegale -- g e' final
8     }
9
10    void without(Gizmo g) {
11        g = new Gizmo() // OK -- g non e' final
12        g.spin();
13    }
14 }
```

```

15
16 // void f(final int i ) { i++; } // Non modificabile
17
18 // Si puo' solo leggere un final di tipo primitivo
19 int g(final int i) { return i + 1; }
20
21 public static void main(String[] args) {
22     FinalArguments bf = new FinalArguments();
23     bf.without(null);
24     bf.with(null);
25 }

```

4.14.2 final applicato ai metodi

Ci sono due ragioni per dichiarare `final` dei metodi:

- impedire alle classi derivate di cambiare il significato del metodo ereditato;
- i metodi `final` sono più efficienti perchè si permette al compilatore di effettuare ogni chiamata al metodo come una chiamata *inline*

I metodi private sono implicitamente `final` perchè non è possibile accedere ad un metodo private ne sovrascriverlo. Pertanto se si aggiunge `final` ad un metodo private non si associa alcun significato extra.

4.14.3 final applicato alle classi

Quando si dichiara un'intera classe `final`, si specifica che non si desidera derivare delle classi da quella corrente. In altre parole si vuole, per questioni di sicurezza o altro, che non sia possibile specializzare tale classe. Alternativamente si potrebbe desiderare che le attività in cui sono coinvolti gli oggetti di tale classe siano più efficienti possibili.

```

1 class SmallBrain {}
2 final class Dinosaur {
3     int i = 7;
4     int j = 1;
5     SmallBrain x = new SmallBrain();
6     void f() {}
7 }
8
9 // ! Una classe Further estende Dinosaur {}
10 // errore: Non si puo' estendere una classe final

```

```

11 public class Jurassic {
12     public static void main(String[] args) {
13         Dinosaur n = new Dinosaur();
14         n.f();
15         n.i = 40;
16         n.j++;
17     }
18 }

```

4.15 Polimorfismo

Il polimorfismo è un altro caposaldo della programmazione OO. Esso fornisce un'altra dimensione di separazione dell'interfaccia dall'implementazione per dividere il *cosa* dal *come*. Pertanto, il polimorfismo permette di migliorare l'organizzazione del codice e la sua leggibilità, oltre alla creazione di programmi estensibili.

4.15.1 Upcasting

Per spiegare il polimorfismo per inclusione, tipico della programmazione orientata a oggetti, dobbiamo rifarci al concetto di upcasting. L'*upcasting* è un meccanismo che permette di prendere un riferimento ad un oggetto e trattarlo come riferimento al suo tipo base.

```

1  // upcasting
2  class Note {
3      private int value;
4      private Note(int val) { value = val; }
5      public static final Note {
6          MIDDLE_C = new Note(0);
7          C_SHARP = new Note(1);
8          B_FLAT = new Note(2);
9      }
10 }
11
12 class Instrument {
13     public void play(Note n) { System.out.println("Instrument.play()"); }
14 }
15
16 class Wind extends Instrument {
17     // Refine interface method:
18     public void play(Note n) { System.out.println("Wind.play()"); }

```

```

19 }
20
21 public class Music {
22     public static void tune(Instrument i) { i.play(Note.MIDDLE_C); }
23     public static void main(String[] args) {
24         Wind flute = new Wind();
25         tune(flute); // Upcasting as Wind extends Instrument
26         tune(new Instrument())
27     }
28 }

```

4.15.2 Binding

Il *binding* è il meccanismo di collegamento di un'invocazione di metodo al corpo del metodo. Se il binding viene eseguito prima che il programma sia lanciato (dal compilatore o da un linker) allora è denominato **early binding** o *static binding*. Nei linguaggi procedurali, come il C, non esistevano delle opzioni di binding differenti perchè, per default, avevano solo l'early binding.

Esempio

```

1 public static void tune(Instrument i) {
2     // ...
3     i.play(Note.MIDDLE_C);
4 }

```

Se Java supportasse solo l'early binding, come farebbe il compilatore a sapere che tipo di oggetto tra *Wind*, *Brass* e *Stringed* il metodo potrebbe avere in input a run-time?

Se Java supportasse solo l'early binding, come farebbe il compilatore a sapere che tipo di oggetto tra *Wind*, *Brass* e *Stringed* il metodo potrebbe avere in input a run-time?

La soluzione la troviamo nel cosiddetto **late binding** o *binding dinamico*. Quando un linguaggio supporta il late binding, ci deve essere un meccanismo che determina a run-time il tipo dell'oggetto passato e selezionare di conseguenza il metodo appropriato. Il meccanismo di late-binding varia da linguaggio a linguaggio. In Java tale meccanismo è impostato per default a meno che non si dichiara esplicitamente un metodo con la parola chiave `final`. Il compilatore genera del codice più efficiente per i metodi dichiarati `final` anche se nella mag-

gior parte dei casi conviene usarlo solo per una decisione di progetto anzichè per migliorare le prestazioni del programma.

Un'altra eccezione è rappresentata dai metodi `static` per i quali vale il binding statico.

```
1 class Base {
2     public static String whoAmI() {
3         return "classe base";
4     }
5 }
6
7 class Derived extends Base {
8     public static String whoAmI() {
9         return "classe derivata";
10    }
11
12    public static void main(String[] args) {
13        Base prova = new Derived();
14        System.out.println(prova.whoAmI());
15    }
16 }
```

In questo caso il programma risponde con “classe base” in quanto la variabile *prova* è stata dichiarata di tipo *Base* anche se viene chiamato il costruttore della classe *Derived*.

Per comprendere meglio le potenzialità offerte dal polimorfismo mediante il binding dinamico consideriamo il seguente esempio:

```
1 Shape s = new Circle();
```

L'oggetto *Circle* è creato e il suo riferimento viene associato immediatamente a *Shape* (ciò è possibile perchè la classe *Circle* è derivata). Eseguendo `s.draw()` ci aspetteremo di invocare il metodo `draw()` definito all'interno della classe *Shape*, in realtà viene invocato il metodo `draw()` della classe *Circle*. La classe base *Shape* stabilisce un'interfaccia comune per le classi derivate che a loro volta ridefiniscono i metodi fornendo un unico comportamento per ogni specifico tipo di figura geometrica. Il fatto di aver generato automaticamente le figure sta ad evidenziare che il compilatore non è a conoscenza del tipo di oggetto che viene passato ai metodi. Il codice è il seguente:

```
1 class Shape {
2     void draw() {}
```

```

3     void erase() {}
4 }
5
6 class Circle extends Shape {
7     void draw {
8         System.out.println("Circle.draw()");
9     }
10
11     void erase {
12         System.out.println("Circle.erase()");
13     }
14 }
15
16 class Square extends Shape {
17     void draw {
18         System.out.println("Square.draw()");
19     }
20
21     void erase {
22         System.out.println("Square.erase()");
23     }
24 }
25
26 class Triangle extends Shape {
27     void draw {
28         System.out.println("Triangle.draw()");
29     }
30
31     void erase {
32         System.out.println("Triangle.erase()");
33     }
34 }
35
36 public class Shapes {
37     public static Shape randShape() {
38         switch((int) (Math.random() * 3)) {
39             default:
40             case 0:
41                 return new Circle();
42             case 1:
43                 return new Square();
44             case 2:
45                 return new Triangle();
46         }
47     }

```

```

48
49 public static void main(String[] args) {
50     Shape[] s = new Shape[9];
51     for (int i = 0; i < s.length; i++)
52         s[i] = randShape();
53     for (int i = 0; i < s.length; i++)
54         s[i].draw();
55 }
56 }

```

4.15.3 Classi e metodi astratti

In alcune situazioni, si potrebbe desiderare solo l'interfaccia di una classe base senza fornire l'implementazione dei metodi. Nell'esempio relativo agli strumenti musicali la classe base *Instrument* aveva dei *metodi fittizi* perchè il suo scopo è quello di creare un'*interfaccia comune* per tutte le sue sottoclassi. Quindi lo scopo di un'interfaccia comune è quello di esprimere cosa è in comune alle classi derivate. In tal caso la classe *Instrument* viene chiamata **classe astratta**.

Quando si definisce una classe astratta, tutti i metodi delle classi da essa derivate, la cui dichiarazione corrisponde con quelli della classe astratta, sono invocati mediante il meccanismo del binding dinamico. Visto che una classe astratta esprime solo l'interfaccia e non una particolare implementazione, ha senso non istanziarla. Java fornisce la parola chiave `abstract` per indicare che una classe o un metodo sono astratti.

Un metodo astratto è incompleto: è comunque dichiarato ma è privo di corpo. La sintassi per la dichiarazione di un metodo astratto è la seguente:

```

1 abstract typeResult MethodName();

```

Se una classe contiene uno o più metodi astratti allora tale classe deve essere indicata con `abstract`, altrimenti viene segnalato un errore di compilazione. Il compilatore si preoccupa di segnalare degli errori se si tenta di istanziare una classe dichiarata `abstract`.

Per ogni sottoclasse di una classe astratta devo ridefinirne i metodi per tutti i metodi astratti, altrimenti le classi derivate saranno considerate astratte e il compilatore richiederà che siano dichiarate come `abstract`. È possibile anche avere classi astratte che non contengono alcun metodo astratto (per esempio classi di cui non si desidera alcuna istanza).

Esempio

Se consideriamo la definizione della classe *Instrument* alla luce di quello appena visto, avremo:

```
1 abstract class Instrument {
2     int i;
3     public abstract void play();
4     public String what() { return "Instrument"; }
5     public abstract void adjust();
6 }
```

Nota

Usando classi e metodi astratti si rende più esplicita l'astrazione di una classe.

4.15.4 Costruttori e polimorfismo

Cosa accade se all'interno di un costruttore si ha una chiamata ad un metodo (con binding dinamico) dell'oggetto costruito? Quando si invoca definito con binding dinamico in un costruttore si potrebbero avere degli effetti strani e indesiderati.

Consideriamo il seguente codice.

```
1 abstract class Glyph {
2     abstract void draw();
3     Glyph() {
4         System.out.println("Glyph() before draw()");
5         draw();
6         System.out.println("Glyph() after draw()");
7     }
8 }
9
10 class RoundGlyph extends Glyph {
11     int radius = 1;
12     RoundGlyph(int r) {
13         radius = r;
14         System.out.println("RoundGlyph.RoundGlyph(), radius = " + radius);
15     }
16
17     void draw() {
18         System.out.println("RoundGlyph.draw(), radius = " + radius);
19     }
20 }
21
22 public class PolyConstructors {
23     public static void main(String[] args) {
```

```

24     new RoundGlyph(5);
25 }
26 }

```

L'output del programma è il seguente:

Glyph() before draw()

RoundGlyph.draw(), radius = 0

Glyph() after draw()

RoundGlyph.RoundGlyph(), radius = 5

Quando il costruttore di *Glyph* invoca il metodo *draw()*, il valore di *radius* non è quello inizializzato ma 0. Il processo di inizializzazione che si è avuto è il seguente:

1. la memoria allocata per l'oggetto è inizializzata a zero binario prima che accada tutto il resto;
2. sono invocati i costruttori della classe base. A questo punto è invocato il metodo ridefinito *draw()* (in realtà prima che il costruttore di *RoundGlyph* sia chiamato) che scopre il valore di *radius* uguale a 0 (dovuto al passo precedente).
3. I membri sono inizializzati nell'ordine di dichiarazione;
4. è invocato il corpo del costruttore della classe derivata.

Nota

Attenzionee nell'invocazione dei metodi all'interno dei costruttori. Gli unici metodi sicuri da chiamare nei costruttori sono quelli dichiarati *final* o (*private*) nella classe base. Questi metodi non possono essere ridefiniti e non possono riservare alcun tipo di sorpresa.

4.15.5 Ereditarietà e finalizzazione

Quando si usa la composizione per creare una nuova classe non ci si deve preoccupare della finalizzazione dei membri degli oggetti in tale classe. Ogni membro è un oggetto indipendente e viene trattato dal *garbage collector* e finalizzato analogamente agli altri oggetti. Con l'ereditarietà è necessario ridefinire il metodo *finalize()* nella classe derivata se si ha bisogno di compiere qualche particolare operazione prima che parta il garbage collector.

È importante ricordare che bisogna sempre invocare il metodo `finalize()` della classe base altrimenti questa non sarà finalizzata.

4.16 Passaggio di parametri e inizializzazione di oggetti

In Java il passaggio dei parametri è per valore (all'atto della chiamata di un metodo viene creata una copia del parametro attuale che è assegnata al parametro formale).

```
1 public class PassReferences {
2     static void f(int i) {
3         i++;
4         System.out.println(i);
5     }
6
7     public static void main(String[] args) {
8         int x = 0;
9         System.out.println(x);
10        f(x);
11        System.out.println(x);
12    }
13 }
```

Questo programma stampa:

0
1
0

```
1 class Index {
2     int i;
3 }
4
5 public class PassReferences {
6     static void f(Index index) {
7         index.i++;
8         System.out.println(index.i);
9     }
10
11     public static void main(String[] args) {
```

```

12     Index x = new Index();
13     System.out.println(x.i);
14     f(x);
15     System.out.println(x.i);
16 }
17 }

```

Questo programma stampa:

```

0
1
1

```

4.16.1 Clonazione di oggetti

In Java è possibile *clonare* un oggetto ovvero farne una copia locale attraverso il metodo *clone()*. Questo metodo definito *protected* nella classe base *Object* può essere usato all'interno della sottoclasse. Il metodo *clone()* genera un *Object* a cui deve essere applicato il casting al proprio tipo. Anche se *clone()* è definito *protected* nella classe base *Object*, non tutte le classi Java sono clonabili. Se si vuole rendere una classe clonabile bisogna aggiungere del codice specifico. In particolare è necessario:

- che la classe implementi l'*interfaccia Cloneable* (che non definisce alcun metodo);
- creare un proprio metodo *clone()* per la classe rendendolo *public*.

```

1 import java.util.*;
2
3 class MyObject implements Cloneable {
4     private int i;
5     MyObject(int ii) { i = ii; }
6     public Object clone() {
7         Object o = null;
8         try {
9             o = super.clone();
10        } catch(CloneNotSupportedException e) {
11            System.err.println("MyObject can't clone");
12        }
13        return o;
14    }
15 }

```

```

16     public String toString() { return Integer.toString(i); }
17 }
18
19 public class LocalCopy {
20     static MyObject g(MyObject v) {
21         v.i++;
22         return v;
23     }
24
25     static MyObject f(MyObject v) {
26         v = (MyObject) v.clone();
27         v.i++;
28         return v;
29     }
30
31     public static void main(String[] args) {
32         MyObject a = new MyObject(11);
33         MyObject b = g(a);
34         System.out.println("b = " + b);
35         MyObject d = f(b);
36         System.out.println("d = " + d);
37         System.out.println("b = " + b);
38     }
39 }

```

Questo programma stampa:

$b = 12$

$d = 13$

$b = 12$

Anche le classi che vengono coinvolte nella composizione devono essere Cloneable.

```

1 import java.util.*;
2
3 class MyObject implements Cloneable {
4     private int i;
5     MyObject(int ii) { i = ii; }
6     public Object clone() {
7         Object o = null;
8         try {
9             o = super.clone();
10        } catch (CloneNotSupportedException e) {}

```

```

11
12     return o;
13 }
14 }
15
16 class MyObject2 implements Cloneable {
17     private MyObject i;
18     MyObject(MyObject ii) { i = ii; }
19     public Object clone() {
20         Object o = null;
21         try {
22             o = super.clone();
23         } catch(CloneNotSupportedException e) {}
24         return o;
25     }
26 }

```

4.16.2 Classi a sola lettura

Java permette di creare delle classi a sola-lettura i cui oggetti sono immutabili. Infatti, si possono definire classi prive di metodi che causano il cambiamento dello stato interno degli oggetti ed evitare così il problema dell'aliasing. Esempi di classe *read-only* sono le classi “wrapper” della libreria standard di Java per tutti i tipi primitivi. Ad esempio:

```

1 // The Integer class cannot be changed
2 import java.util.*;
3 public class ImmutableInteger {
4     public static void main(String[] args) {
5         Integer v[] = new Integer[10];
6         for (int i = 0; i < 10; i++)
7             v[i] = new Integer(i);
8         for (int i = 0; i < 10; i++)
9             v[i]++; // equal to v[i] = new Integer(v[i] + 1);
10    }
11 }

```

Inoltre è possibile anche creare delle classi read-only proprie. Tutti i dati sono *private* e nessuno dei metodi *public* modifica tali dati. La creazione di una classe immutabile, pur essendo una soluzione elegante, può causare frequenti attivazioni del garbage collector oltre ad avere un costo aggiuntivo dovuto alla creazione di nuovi oggetti. Questo problema è particolare sentito per esempio per la classe *String*.

La soluzione consiste nel creare una classe simile ma modificabile e saltare da una all'altra a seconda delle esigenze.

```
1 // Gli oggetti che non possono essere modificati sono immuni agli effetti di
  alias
2 public class Immutable1 {
3     private int data;
4     public Immutable(int init) { data = init; }
5     public int read() { return data; }
6     public boolean nonzero() { return data != 0; }
7     public Immutable1 quadruple() { return new Immutable1(data * 4); }
8     static void f(Immutable i1) {
9         Immutable quad = i1.quadruple();
10        System.out.println("i1 = " + i1.read());
11        System.out.println("quad = " + quad.read());
12    }
13
14    public static void main(String[] args) {
15        Immutable1 x = Immutable1(47);
16        System.out.println("x = " + x.read());
17        f(x);
18        x = x.quadruple().quadruple().quadruple().quadruple();
19        System.out.println("x = " + x.read());
20    }
21 }
```

La classe *Immutable2* contiene metodi che mantengono gli oggetti immutati, producendo nuovi oggetti qualvolta che è necessaria una modifica di stato. Viceversa *Mutable* contiene metodi che modificano gli oggetti passati ai metodi *add()* e *multiply()*. I metodi statici *modify1()* e *modify2()* producono gli stessi risultati. Tuttavia mentre il primo chiede la creazione di 4 oggetti, il secondo richiede la creazione di solo due oggetti.

```
1 class Mutable {
2     private int data;
3     public Mutable(int init) { data = init; }
4     public Mutable add(int x) {
5         data += x;
6         return this;
7     }
8
9     public Mutable multiply(int x) {
10        data *= x;
11        return this;
12    }
13 }
```

```

13
14     public Immutable2 makeImmutable2() {
15         return new Immutable2(data);
16     }
17 }
18
19 public class Immutable2 {
20     private int data;
21     public Immutable2(int init) { data = init; }
22     public int read() { return data; }
23     public boolean nonzero() { return data != 0; }
24     public Immutable2 add(int x) { return new Immutable2(data + x); }
25     public Immutable2 multiply(int x) { return new Immutable2(data * x); }
26     Mutable makeMutable() { return new Mutable(data); }
27     public static Immutable2 modify1(Immutable2 y) {
28         Immutable2 val = y.add(12);
29         val = val.multiply(3);
30         val = val.add(11);
31         val = val.multiply(2);
32         return val;
33     }
34
35     public static Immutable2 modify2(Immutable2 y) {
36         Mutable m = y.makeMutable();
37         m.add(12).multiply(3).add(11).multiply(2);
38         return m.makeImmutable2();
39     }
40 }
41
42 public class MainClass {
43     public static void main(String[] args) {
44         Immutable i2 = new Immutable2(47);
45         Immutable r1 = modify(i2);
46         Immutable r2 = modify(i2);
47         System.out.println("i2 = " + i2.read());
48         System.out.println("r1 = " + r1.read());
49         System.out.println("r2 = " + r2.read());
50     }
51 }

```

Come già osservato, l'immutabilità di una classe può causare dei problemi di inefficienza.

Esempio : Concatenazione di stringhe

```
String s = "abc" + foo + "def" + Integer.toString(47);
```

La soluzione al problema consiste nell'uso di un “compagno di classe” che sia mutevole.

Per *String* il compagno di classe pensato dai progettisti Java si chiama *StringBuffer*. Il compilatore crea automaticamente un oggetto *StringBuffer* per valutare certe espressioni, come quelle degli operatori `+` e `+=` della classe *String*.

4.16.3 Inizializzazione dei membri

Java garantisce che alcune variabili siano inizializzate coerentemente prima di essere usate. In particolare, vengono inizializzate tutte le variabili di istanza e di classe. Se sono di tipo primitivo viene assegnato un valore di default. Se sono il riferimento a un oggetto viene assegnato valore `null`. Per variabili locali a un metodo viene segnalato un errore al momento della compilazione.

Un membro di tipo dato può essere inizializzato attraverso il valore restituito da un metodo. L'ordine di inizializzazione è determinato dall'ordine in cui le variabili sono definite nella classe.

```
1 class Measurement {
2     Depth o = new Depth();
3     boolean b = true;
4     // ....
5 }
6 class CInit {
7     int i = f();
8     int j = g(i);
9     int f() { return 5; }
10    int g(int a) { return a + 1; }
11 }
```

Non possiamo mai avere, però, una situazione di questo tipo.

```
1 class CInit {
2     int j = g(i);
3     int i = f();
4     // ...
5     int f() { return 5; }
6     int g(int a) { return a + 1; }
7 }
```

Un array in java è una sequenza di elementi (oggetti o tipi primitivi) dello stesso tipo impacchettati insieme e identificati da un unico nome.

Un array in Java si definisce nel seguente modo:

`int[] a1 oppure int a1[]`

L'inizializzazione per gli array può avvenire in qualsiasi punto del codice ed avviene nel seguente modo:

`int[] a1 = {1, 2, 3, 4, 5}`

È possibile anche inizializzare gli array in altri modi senza però rispettare la compatibilità con le versioni precedenti del JDK.

Ogni array ha un membro intrinseco, `length`, a cui possiamo accedere (ma non modificare) che indica il numero di elementi presenti nell'array. In Java si contano gli elementi di un array a partire da 0 pertanto l'ultimo elemento di un array avrà sempre posizioni `length - 1`.

```
1 public class ArrayInit {  
2     public static void main(String[] args) {  
3         Integer[] a = {  
4             new Integer(1),  
5             new Integer(2),  
6             new Integer(3),  
7         };  
8  
9         Integer[] b = new Integer[3];  
10        b[0] = new Integer(1);  
11        b[1] = new Integer(2);  
12        b[2] = new Integer(3);  
13    }  
14 }
```

4.17 Trattamento delle eccezioni

4.17.1 Il trattamento degli errori

Non tutti gli errori presenti in un programma possono essere rilevati in compilazione. Quando si verifica una condizione anormale in una sequenza di codice al momento dell'esecuzione si dice che si è verificata un'*eccezione*. Nei linguaggi di programmazione che non supportano la gestione delle eccezioni, gli errori

devono essere controllati e gestiti manualmente dal programmatore a costo di notevoli sforzi. Ciò avviene solitamente mediante l'utilizzo di codici di errore.

Java supporta la gestione degli errori, rifacendosi al modello C++ che a sua volta si basa sul modello di Ada. Una **condizione eccezionale** è un problema che impedisce la regolare esecuzione del programma, ovvero l'elaborazione non può continuare perchè non si hanno abbastanza informazioni per trattare il problema nel contesto corrente. Per gestire un'eccezione si potrebbe uscire dal contesto corrente relegando il tutto ad un contesto più alto. L'uscita dal contesto corrente corrisponde al **sollevamento di un'eccezione**.

Quando si solleva un'eccezione vengono eseguite le seguenti operazioni:

1. viene creato l'*oggetto eccezione* come un qualsiasi oggetto Java (nella memoria heap tramite l'operatore *new*);
2. il flusso di esecuzione corrente viene interrotto e il riferimento all'oggetto eccezione viene **espulso** dal contesto corrente;
3. il meccanismo di *gestione delle eccezioni* rileva la situazione e comincia a cercare il punto opportuno dove far proseguire l'esecuzione del programma.

Il punto opportuno da cui far riprendere l'esecuzione del programma si chiama *gestore dell'eccezione*. Esso ha il compito di recuperare la situazione in modo tale che il programma possa o cercare qualche linea di esecuzione alternativa o semplicemente continuare. La sezione di codice che potrebbe generare delle eccezioni e che è seguita dal codice per il loro trattamento, è chiamata *zona protetta* o **guarded zone**.

Modello a terminazione o ripresa? Supportata da Java e C++: si presuppone che l'errore è talmente critico da non poter ritornare al punto in cui è stata sollevata l'eccezione. Si riprende l'esecuzione dall'istruzione successiva al gestore dell'eccezione.

4.17.2 Sollevare un'eccezione

Per lanciare un'eccezione in modo esplicito si utilizza l'istruzione `throw`. La sua sintassi generale è la seguente:

```
1 throw ThrowableInstance;
```

dove *ThrowableInstance* è istanza della classe *Throwable* o istanza di una sua sottoclasse. Un semplice caso di sollevamento di eccezione potrebbe essere il passaggio di un riferimento ad un oggetto non ancora inizializzato.

```
1 String t = null;
2 if (t == null) throw new NullPointerException();
3 System.out.println("hello");
```

dove *NullPointerException* è una classe predefinita di eccezioni.

Esistono due costruttori per tutte le eccezioni standard:

- un costruttore di default;
- costruttore che un argomento di tipo stringa in input.

```
1     if (t == null)
2         throw new NullPointerException("t == null");
3
```

4.17.3 Catturare un'eccezione

Se un metodo solleva un'eccezione, questa deve essere in qualche modo *catturata* e *trattata*. Per catturare l'eccezione si può impostare un blocco speciale chiamato *try block* in cui si inserisce la sezione di codice che potrebbe generare l'eccezione. La sintassi del *try block* è la seguente:

```
1 try {
2     // Code that might generate exceptions
3 }
```

Il gestore dell'eccezione segue il blocco *try* ed è introdotto dalla parola chiave *catch*. Poiché un blocco *try* si potrebbero sollevare più eccezioni di tipo differenti, al blocco *try* è possibile far seguire diverse clausole *catch*, una per ogni tipo di eccezione che si vuole gestire. La sintassi è la seguente:

```
1 try {
2     // Code that might generate exceptions
3     ist1();
4     ist2();
5     System.out.println("ok");
6 } catch (Type1 id1) {
7     // Handle exceptions of Type1
8 } catch (Type1 id1) {
9     // Handle exceptions of Type1
```

```

10 }
11
12 System.out.println("hello");

```

Ogni proposizione `catch` è come un piccolo metodo che ha come argomento un tipo particolare. Gli identificatori (*id1*, *id2* e così via) possono essere usati all'interno del gestore come se fossero argomenti di metodi. Anche se l'identificatore non viene utilizzato all'interno del blocco, deve essere sempre indicato. I gestori devono essere inseriti subito dopo il *try block*. Quando viene sollevata un'eccezione vengono seguiti i seguenti passi.

1. Il meccanismo di gestione cerca il primo gestore il cui tipo di argomento corrisponde con il tipo dell'eccezione sollevata.
2. Quindi entra nel blocco del rispettivo `catch` e l'eccezione viene gestita.
3. La ricerca dei gestori termina quando il blocco `catch` è eseguito

Dopo la gestione dell'eccezione si passa ad eseguire la prima istruzione dopo l'ultimo blocco `catch`. Si noti che in questo caso non si ha bisogno di alcuna istruzione supplementare (come il `break` nello `switch`) per evitare che le restanti `catch` siano eseguite. Il gestore è **unico**, sebbene all'interno del *try block* diverse invocazioni di metodi potrebbero generare la stessa eccezione.

È possibile avere blocchi `try` annidati.

```

1 try { // Istruzioni
2     try { // Istruzioni
3
4     } catch (A a1) { // Gestore A }
5 } catch (B b1) { // Gestore B }

```

Più in generale i blocchi *try* potranno essere annidati ma in metodi differenti.

```

1 void external_mthd() {
2     // Istruzioni
3     try {
4         // Istruzioni
5     } catch (B b1) { // Gestore B }
6 }
7
8 void internal_mthd() {
9     // Istruzioni
10    try {
11        // Istruzioni

```

```

12     } catch (A a1) { // gestore A }
13 }

```

Se un'eccezione non viene intercettata e gestita quando si verifica, il controllo viene immediatamente restituito al metodo che invocato il metodo che l'ha prodotta. Si può progettare il software in modo che l'eccezione sia intercettata e gestita al livello più esterno, ma se non viene gestita neppure qui, il controllo passa al metodo che lo ha invocato e così via.

La *propagazione delle eccezioni* continua finchè l'eccezione non viene intercettata e gestita, oppure fino a quando non si esce dal metodo main, nel qual caso termina il programma e si produce il messaggio d'errore relativo all'eccezione.

È possibile anche pensare a un gestore che catturi qualsiasi eccezione. Per fare ciò si cattura la classe base delle eccezioni *Exception*

```

1 catch (Exception e) {
2     System.err.println("Caught an exception");
3 }

```

È consigliabile inserire questo gestore solo alla fine della lista degli altri gestori di eccezioni specifiche.

4.17.4 La parola chiave `finally`

L'istruzione `try` può avere una clausola `finally` opzionale: essa definisce una sezione di codice che viene eseguita indipendentemente da come si sia usciti dal *try block*. Spesso viene usata per garantire il rilascio della memoria o di altre risorse a prescindere da quello che accade nel *try block*. Se non si verificano eccezioni, le istruzioni nella clausola `finally` vengono eseguite dopo che si è completato il *try block*. Se si verifica un'eccezione nel *try block*, il controllo passa alla clausola `catch` appropriata e poi alla clausola `finally`.

La clausola `finally` se presente, dev'esser posta dopo tutte le clausole `catch`.

```

1 try {
2     // Istruzioni
3 } catch (A a1) { // Gestore A
4 } catch (B b1) { // Gestore B
5 } catch (C c1) { // Gestore C
6 } finally {
7     // Activities that happen every time
8 }

```

Un *try block* non deve essere necessariamente seguito da una clausola *catch*: se non ci sono, si può usare solo il blocco *final*. Ad esempio:

```
1 try {  
2     System.out.println(Integer.parseInt(numString));  
3 } catch (NumberFormatException exception) {  
4     System.out.println("Eccezione intercettata");  
5 } finally {  
6     System.out.println("Fatto");  
7 }
```

4.17.5 La clausola *throws*

Java fornisce anche una sintassi che permette di indicare quali sono le eccezioni sollevate da un metodo in modo tale che il programmatore possa gestirle. Se un metodo può provare un'eccezione che non è in grado di gestire, deve generalmente specificare questo comportamento in modo tale che i metodi chiamati si preparino a questa eccezione. A tale scopo si utilizza la clausola *throws* nella dichiarazione del metodo che elenca i tipi di eccezione che un metodo dovrebbe sollevare.

Questa sintassi viene chiamata *descrizione dell'eccezione* o **exception specification** e fa parte della dichiarazione del metodo subito dopo la lista degli argomenti.

```
1 type MethodName(argumentList) throws exceptionList  
2 {  
3     // body  
4 }
```

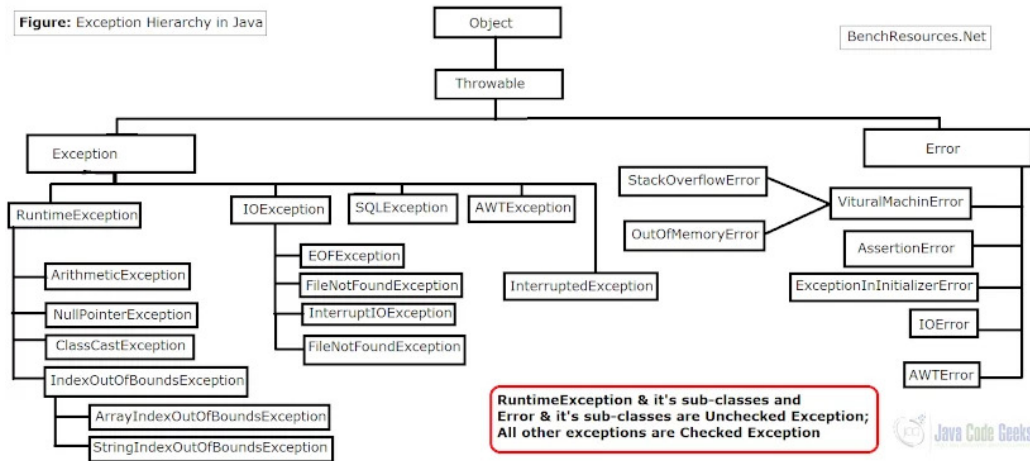
Java garantisce la verifica in compilazione della corretta gestione delle eccezioni. In questo differisce dal C++ che permette la cattura delle violazioni delle descrizioni delle eccezioni solo in esecuzione.

Tutti i tipi di eccezione in Java sono derivati dalla classe *Throwable*. Da *Throwable* sono derivate due sottoclassi che portano a due gerarchie di ereditarietà distinte. La prima fa capo alla classe *Exception* utilizzata per le **condizioni eccezionali** che i programmi utente dovrebbero catturare. Da questa classe saranno poi derivate tutte le sottoclassi relative alle eccezioni personalizzate.

L'altro ramo della gerarchia è capeggiata dalla classe *Error* che definisce le eccezioni da catturare in **circostanze normali** del programma. Questo tipo di

eccezione viene utilizzata al run-time per indicare errori che interessano l'esecuzione del programma come ad esempio il messaggio di overflow dello stack. Java definisce diverse classi di eccezioni all'interno del package *java.lang* che viene implicitamente importato in tutti i programmi.

Figure: Exception Hierarchy in Java



Le eccezioni più generali sono quelle derivate da *RuntimeException*. In Java queste eccezioni non devono essere inserite nell'elenco *throws* del metodo. Inoltre, sono definite eccezioni non controllate perchè il compilatore non controlla se un metodo gestisce o lancia queste eccezioni.

Il pacchetto *java.lang* contiene anche delle eccezioni che devono essere necessariamente nell'elenco *throws* del metodo se tale metodo è in grado di generare una di queste eccezioni e non la gestisce. Questo tipo di eccezioni sono chiamate **eccezioni controllate**. In generale, è indispensabile elencare nella clausola *throws* le eccezioni sollevabili in un metodo e non gestite, escluse quelle di tipo *RuntimeException* e rispettive sottoclassi.

La creazione di eccezioni personalizzate Oltre alle eccezioni presenti nel package *java.lang* sono disponibili altri tipi di eccezioni correlate alle diverse librerie di classi. Java permette di creare delle eccezioni personalizzate che servono a coprire i casi in cui non sono previste nelle librerie standard. Per creare una nuova eccezione si cerca di derivarla da una già esistente che un significato analogo. Ad esempio:

```
1 class MyException extends Exception {
```

```

2 public MyException() {}
3 public MyException(String msg) { super(msg); }
4 }
5
6 public class FullConstructors {
7     public static void f() throws MyException {
8         System.out.println("Throwing MyException from f()");
9         throws new MyException();
10    }
11
12    public static void g() throws MyException {
13        System.out.println("Throwing MyException from g()");
14        throw new MyException("Originated in g()");
15    }
16
17    public static void main(String[] args) {
18        try {
19            f();
20        } catch (MyException e) {
21            e.printStackTrace(System.err);
22        }
23        try {
24            g();
25        } catch (MyException e) {
26            e.printStackTrace(System.err);
27        }
28    }
29 }

```

L'output di questo programma è il seguente:

```

    Throwing MyException from f();
        MyException
    at FullConstructors.f(FullConstructors.java:16)
    at FullConstructors.main(FullConstructors.java:24)

    Throwing MyException from g()
        MyException: Originated in g()
    at FullConstructors.g(FullConstructors.java:20)
    at FullConstructors.main(FullConstructors.java:29)

```

4.18 I contenitori

In Java è disponibile il tipo predefinito *array*, a cui si aggiungono delle librerie che introducono un ragionevole insieme di *classi contenitore*, dette *container class*. Tali classi sono spesso chiamate anche *collection* sebbene in Java 2 il nome ***Collection*** sta ad indicare un particolare sottoinsieme di librerie.

4.18.1 Array in Java

Ci sono due caratteristiche che distinguono gli array dagli altri tipi di contenitori:

1. Efficienza.
2. Tipo.

L'array è il modo più efficiente che Java fornisce per memorizzare ed accedere direttamente ad una sequenza di oggetti (intesi come riferimenti ad oggetti). Il prezzo di questa efficienza è quello di avere una dimensione predefinita che non può essere modificata durante il proprio ciclo di vita. Come alternativa, Java fornisce la classe *ArrayList* che però paga la maggiore flessibilità con la minore efficienza.

Rispetto ad un contenitore generico, l'array contiene **un solo tipo specifico di oggetto**. È conveniente usare gli array quando si vuole guadagnare in termini di efficienza e controllo sul tipo di oggetti. Negli altri casi, infatti, potrebbero risultare un po' restrittivi. L'unico membro accessibile di un array è *length* che dice il numero di oggetti contenuti nell'array mentre la sintassi `[]` è l'unico modo per accedere all'oggetto di un array.

In Java se si superano i confini specificati per un'istanza di array (o di classe contenitore), viene generata un'eccezione a run-time. Al contrario, in C++ l'operatore `[]` non effettua alcun controllo sui limiti del contenitore.

A differenza degli array, le classi contenitore possono contenere solo *riferimenti ad oggetti*. Si aggira questo limite ricordando che in Java sono disponibili delle classi "wrapper" come *Integer*, *Double*, etc. che permettono di inserire dei valori primitivi in un contenitore. Tuttavia, si sottolinea che è più efficiente e semplice creare un array di tipi primitivi anziché un contenitore di classi "wrapper".

In C/C++ non è possibile restituire un intero array ma solo un puntatore all'array (con tutti i problemi dovuti al fenomeno di *memory leak*). Java utilizza lo stes-

so approccio restituendo, però, un array di riferimenti agli oggetti senza alcuna preoccupazione sulla dimensione.

Java include nella sua libreria *java.util*, la classe *Arrays* che contiene un insieme di metodi statici che supportano le funzioni di utilità per gli array.

```
1 public class Arrays {  
2     // Suppresses default constructors,  
3     // ensuring non-instantiability.  
4     private Arrays() {}  
5 }
```

Non ci sono altri costruttori, quindi tutti i metodi definiti devono essere di classe per poter garantire l'utilizzabilità di *Arrays*.

Ci sono quattro funzioni di base:

- *equals()*: confronta per uguaglianza due array;
- *fill()*: riempie un array con un valore;
- *sort()*: ordina un array;
- *binarySearch()*: ritrova un elemento in array ordinato.

Tutti questi metodi sono overloaded per tutti i tipi primitivi e gli oggetti di classe *Object*.

Ai metodi elencati prima si aggiunge *asList()* che prende degli array e li inserisce in un altro contenitore chiamato *List*.

Copia di un array La libreria standard di Java fornisce un metodo *static*, *System.arraycopy()*, che effettua la copia di un array più velocemente del classico *for*. Questo metodo è overloaded per trattare tutti i tipi.

Gli argomenti del metodo *arraycopy()* sono:

- l'array sorgente;
- l'indirizzo dell'array sorgente da cui iniziare la copia;
- l'array di destinazione;
- l'indirizzo dell'array di destinazione dove inizia la copia;
- il numero di elementi da copiare.

Se si violano i confini dell'array, verrà sollevata un'eccezione. Nel seguente esempio si mostra come possono essere copiati sia tipi primitivi che oggetti.

```
1 // Esempio di uso di System.arraycopy()
2 import com.prova.util.*;
3 import java.util.*;
4 public class CopyingArrays {
5     public static void main(String[] args) {
6         int[] i = new int[25];
7         int[] j = new int[25];
8         Arrays.fill(i, 47);
9         Arrays.fill(j, 99);
10        System.arraycopy(i, 0, j, 0, i.length);
11        int[] k = new int[10];
12        Arrays.fill(k, 103);
13        System.arraycopy(i, 0, k, 0, k.length);
14        Arrays.fill(k, 103);
15        System.arraycopy(k, 0, i, 0, k.length);
16        Integer[] u = new Integer[10];
17        Integer[] v = new Integer[5];
18        Arrays.fill(u, new Integer(47));
19        Arrays.fill(v, new Integer(99));
20        System.arraycopy(v, 0, u, u.length / 2, v.length);
21    }
22 }
```

Si ricorda che nel caso degli oggetti sono duplicati solo i riferimenti agli oggetti, non c'è alcuna duplicazione degli oggetti stessi, detta *shallow copy*. Per effettuare la *deep copy* invece bisogna effettuare la clonazione.

Confronto di array La classe `Arrays` fornisce il metodo `equals()` overloaded per tutti i tipi primitivi e per gli oggetti *Object*. Questo metodo confronta i corrispondenti elementi di due array e restituisce *true* se i due array sono uguali.

```
1 public static boolean equals(long[] a, long[] a2) {
2     if (a == a2)
3         return true;
4     if (a == null || a2 == null)
5         return false;
6     int length = a.length;
7     if (a2.length != length)
8         return false;
9     for (int i = 0; i < length; i++)
10        if (a[i] != a2[i])
11            return false;
```

```

12     return true;
13 }
14
15 public static boolean equals(char[] a, char[] a2) {
16     // Body
17 }
18
19 ...
20
21 public static boolean equals(Object[] a, Object[] a2) {
22     if (a == a2)
23         return true;
24     if (a == null || a2 == null)
25         return false;
26     int length = a.length;
27     if (a2.length != length)
28         return false;
29     for (int i = 0; i < length; i++) {
30         Object o1 = a[i];
31         Object o2 = a2[i];
32         if (!(o1 == null ? o2 == null : o1.equals(o2)))
33             return false;
34     }
35     return true;
36 }

```

Il confronto di uguaglianza per array di oggetti si basa sui contenuti mediante *Object.equals()*. La classe *Object* implementa *equals* nel modo più restrittivo ($x == y$), mentre la classe *String* lo ridefinisce in modo da effettuare un confronto fra stringhe. Il polimorfismo del metodo fa il resto.

Comparazione degli elementi di un array L'approccio di Java è basato sulla tecnica del *callback*, per cui la parte del codice che varia da caso a caso è incapsulata all'interno della rispettiva classe mentre la parte di codice che resta invariante richiamerà il codice che cambia.

Nell'ordinamento:

- **Parte invariante:** algoritmo di ordinamento (a bolle, quicksort, mergesort, etc.). Incluso nella classe *Arrays*.
- **Parte variabile:** confronto di due istanze di una classe, Incluso nel codice della classe di appartenenza.

Esempio : Parte invariante

```
1 public static void sort(Object[] a) {
2     Object[] aux = (Object[]) a.clone();
3     mergeSort(aux, a, 0, a.length);
4 }
5
6 private static void mergeSort(Object[] src, Object[] dest, int low, int
    high) {
7     int length = high - low;
8
9     if (length < 7) {
10         for (int i = low; i < high; i++) {
11             for (int j = i; j > low && dest[j-1] >= dest[j]; j--)
12                 swap(dest, j, j-1);
13         }
14         return;
15     }
16
17     int mid = (low + high) / 2;
18     mergeSort(dest, src, low, mid);
19     mergeSort(dest, src, mid, high);
20     // If list is already sorted, just copy from src to dest. This is an
    optimization.
21     if (src[mid - 1] <= src[mid]) {
22         System.arraycopy(src, low, dest, low, length);
23         return;
24     }
25
26     // Merge sorted halves (now in src) into dest
27     for (int i = low, p = low, q = mid; i < high; i++) {
28         if (q >= high || p < mid && src[p] <= src[q])
29             dest[i] = src[p++];
30         else
31             dest[i] = src[q++];
32     }
33 }
```

In Java 2 ci sono due modi per offrire funzionalità di confronto. Il primo è attraverso il metodo di confronto naturale che viene comunicato alla classe mediante l'implementazione dell'interfaccia *java.lang.Comparable*. Questa semplice interfaccia è fornita da un singolo metodo,

public int compareTo(Object o)

che prende un oggetto generico *Object* come argomento e restituisce un valore negativo se l'argomento è più piccolo dell'oggetto corrente, zero se è uguale e un valore positivo se è maggiore.

Ad esempio:

```
1 class Studente implements Comparable {
2     int matricola;
3     int numeroEsami;
4     String cognome;
5     String nome;
6     Studente(int matricola, int numeroEsami, String cognome, String nome) {
7         this.matricola = matricola;
8         this.numeroEsami = numeroEsami;
9         this.cognome = cognome;
10        this.nome = nome;
11    }
12
13    public int compareTo(Object o) {
14        Studente s = (Studente) o;
15        if (s.numeroEsami == numeroEsami)
16            return 0;
17        else if (s.numeroEsami < numeroEsami)
18            return -1;
19        else
20            return 1;
21    }
22 }
```

Se volessimo ordinare *s* in base al cognome, allora si procede nel seguente modo:

```
1 class StudenteComp implements Comparator {
2     public int compare(Object o1, Object o2) {
3         Studente s1 = (Studente) o1;
4         Studente s2 = (Studente) o2;
5         return s1.cognome.compareTo(s2.cognome);
6     }
7 }
```

Lo stesso approccio si utilizza se volessimo ordinare *s* in base alla matricola.

Il secondo modo è quello di creare una classe che implementa l'interfaccia *Comparator* che ha due metodi:

public int compare(Object o1, Object o2)

public boolean equals(Object obj)

Il metodo *compare* deve restituire un valore intero negativo, nullo o positivo a seconda del risultato del confronto.

4.18.2 I contenitori

I contenitori di Java 2 si basano su due distinti concetti:

- **Collection**: un gruppo di *singoli* elementi a cui spesso sono applicati dei vincoli (ad esempio un oggetto di *List* deve contenere elementi in una particolare sequenza o una classe *Set* non può contenere elementi duplicati).
- **Map**: un gruppo di *coppie* “chiave-oggetto valore”.

La distinzione tra le due categorie di contenitori in Java è quindi basata sul numero di elementi contenuti in ogni locazione del contenitore.

La categoria *Collection* contiene un elemento in ogni locazione. Essa include il tipo *List* che contiene un gruppo di elementi in una particolare sequenza e il tipo *Set* che permette solo l’aggiunta di un elemento di un dato tipo.

L’*ArrayList* è un tipo *List* e *HashSet* è un tipo di *Set*. Per aggiungere elementi a qualsiasi *Collection* si usa il metodo *add()*. Per quanto riguarda il riempimento di un contenitore, si può usare il metodo *fill()* anche se solo per il tipo *List*.

La categoria dei contenitori *Map* contiene coppie di chiave-valore come se fosse una piccola tabella di un database relazionale. Se si ha un contenitore *Map* che associa le regioni italiane con i rispettivi capoluoghi di regione e si vuole conoscere il capoluogo di regione della Puglia vi si accede come se si avesse un array indicizzato (i contenitori *Map* sono chiamati anche *array associativi*). Per aggiungere degli elementi ad un *Map* si usa il metodo *put()* che acquisisce in input una chiave ed un valore come argomenti. Un contenitore *Map* può restituire

1. un *Set* contenente tutte le sue chiavi,
2. una *Collection* contenente tutti i suoi valori,
3. un *Set* di coppie.

I contenitori *Map*, come gli array, possono essere facilmente estesi a dimensioni multiple senza l’aggiunta di nuovi concetti ma semplicemente creando un *Map* i cui valori sono a loro volta dei *Map*.

Visualizzare i Contenitori A differenza degli array, che consentono la stampa di un elemento alla volta in un ciclo for, i contenitori si possono visualizzare con facilità.

```
1  import java.util.*;
2  public class PrintingContainers {
3      static Collection fill(Collection c) {
4          c.add("dog");
5          c.add("dog");
6          c.add("cat");
7          return c;
8      }
9
10     static Map fill(Map m) {
11         m.put("dog", "Bosco");
12         m.put("dog", "Spot");
13         m.put("cat", "Rags");
14         return m;
15     }
16
17     public static void main(String[] args) {
18         System.out.println(fill(new ArrayList()));
19         System.out.println(fill(new HashSet()));
20         System.out.println(fill(new HashMap()));
21     }
22 }
23
```

Per l'esempio precedente avevamo il seguente output:

[dog, dog, cat]

[cat, dog]

{cat = Rags, dog = Spot}

Svantaggi dei Contenitori Quando si usano i contenitori Java si perde l'informazione relativa al tipo di oggetto che si inserisce in essi. Ciò porta alle seguenti considerazioni:

- non essendoci controllo sul tipo degli oggetti inclusi nel contenitore, non è possibile essere sicuri di avere dei contenitori di oggetti omogenei;

- l'unica cosa che il contenitore conosce è che contiene dei riferimenti ad oggetti, pertanto è necessario effettuare un cast al tipo corretto prima di utilizzarli.

Se in un contenitore di riferimenti ad oggetti di tipo “cerchio” si inserisce un oggetto di tipo “quadrato”, nel momento in cui si andranno a manipolare gli oggetti del contenitore trattandoli tutti come “cerchio” verrà sollevata un’eccezione a run-time. L’identificazione al run-time del tipo è comunque possibile in Java.

```

1  public class Cat {
2      private int catNumber;
3      Cat(int i) { catNumber = i; }
4      void print() {
5          System.out.println("Cat #" + catNumber);
6      }
7  }
8
9  public class Dog {
10     private int dogNumber;
11     Dog(int i) { dogNumber = i; }
12     void print() {
13         System.out.println("Dog #" + dogNumber);
14     }
15 }
16
17 import java.util.*;
18 public class CatsAndDogs {
19     public static void main(String[] args) {
20         ArrayList cats = new ArrayList();
21         for (int i = 0; i < 7; i++)
22             cats.add(new Cat(i));
23
24         cats.add(new Dog(7));
25         for (int i = 0; i < cats.size(); i++)
26             ((Cat)cats.get(i)).print();
27         // La presenza del tipo Dog viene rilevata a run-time.
28     }
29 }
30

```

4.18.3 Iteratori

Un *iterator* è un oggetto che si muove attraverso una sequenza di oggetti e ne seleziona uno qualsiasi senza che il programmatore sia a conoscenza della strut-

tura sottostante della sequenza. Ad ogni contenitore si può chiedere, mediante il metodo *iterator()*, di generare un opportuno iteratore, che altro non è che un'istanza di una classe che implementa l'interfaccia *Iterator*. L'interfaccia *Iterator* di Java permette di:

1. restituire il **primo** elemento della sequenza sulla prima chiamata del metodo *next()*;
2. prendere l'oggetto **successivo** nella sequenza con l'invocazione del metodo *next()*;
3. verificare se ci sono altri oggetti nella sequenza con *hasNext()*;
4. rimuovere l'ultimo elemento restituito dall'iteratore con il metodo *remove()*.

Un esempio di iteratore è il seguente.

```
1 import java.util.*;
2 public class CatsAndDogs2 {
3     public static void main(String[] args) {
4         ArrayList cats = new ArrayList();
5         for (int i = 0; i < 7; i++)
6             cats.add(new Cat(i));
7         Iterator e = cats.iterator();
8         while(e.hasNext())
9             ((Cat)e.next()).print();
10    }
11 }
```

4.18.4 Funzionalità di List

L'interfaccia *List* mantiene gli elementi nell'ordine in cui sono inseriti. Fornisce metodi per l'inserimento e l'eliminazione dei suoi elementi. Una *List* produrrà un *ListIterator* che potrà spostarsi nella lista in entrambe le direzioni al suo interno. Per specializzare il comportamento di *List* sono disponibili le sottoclassi:

- **ArrayList**. È una *List* implementata con un array. Permette un accesso rapido agli elementi ma risulta lento nell'inserire o rimuovere gli elementi al centro del contenitore.

ListIterator dovrebbe essere utilizzato solo per accedere agli elementi e non per il loro inserimento ed eliminazione.

- **LinkedList.** Fornisce un ottimo accesso sequenziale con inserimenti ed eliminazioni poco costosi dal centro delle *List*. Risulta relativamente lenta negli accessi diretti. Supporta i metodi *addFirst()*, *addLast()*, *getFirst()*, *getLast()*, *removeFirst()* e *removeLast()* che gli permettono di essere usata come uno stack o una coda.

4.18.5 Funzionalità di Set

L'interfaccia *Set* ha esattamente la stessa interfaccia di *Collection* così non abbiamo funzionalità extra come per i tipi *List*. Un *Set* contiene solo un'istanza di ogni valore dell'oggetto. L'interfaccia *Set* non garantisce l'ordine degli oggetti inseriti. L'univocità degli elementi è stabilita mediante il metodo *equals()*. Classi che implementano l'interfaccia *Set* sono:

- **HashSet.** È utilizzato per *Set* che richiedono tempi di accesso più brevi. Per gli oggetti si definisce un metodo *hashCode()*.

```

1 class Studente {
2     private int matricola;
3     private String cognome;
4     Studente(int matricola, String cognome) {
5         this.matricola = matricola;
6         this.cognome = cognome;
7     }
8
9     public boolean equals(Object o) {
10         return ((Studente) o).getMatricola() == matricola;
11     }
12
13     int getMatricola() { return matricola; }
14
15     public int hashCode() {
16         return new Integer(matricola).hashCode();
17     }
18
19     public String toString() { return cognome; }
20 }
21
22
23 import java.util.HashSet;
24 class Tester {
25     public static void main(String[] args) {
26         HashSet s = new HashSet();

```

```

27         s.add(new Studente(1, "Bianchi"));
28         s.add(new Studente(2, "Rossi"));
29         s.add(new Studente(1, "Verde")); // Non viene inserito perche'
        la matricola 1 e' gia presente
30         System.out.println(s);
31     }
32 }
33

```

- **TreeSet**. È un *Set* ordinato attraverso un albero. In tal modo si può estrarre una sequenza ordinata da un *Set*.

Pur accettando l'inserimento di valori duplicati, si può notare che quando il contenuto del *Set* viene visualizzato si vede che essi non sono stati considerati. I ***Set ordinati*** aggiungono funzionalità nuove, rispetto ai *Set* comuni, che permettono di gestire l'ordinamento degli elementi inseriti.

4.18.6 Funzionalità di Map

L'interfaccia *Map* permette di accedere agli oggetti di un contenitore attraverso un altro oggetto. A tale scopo abbiamo i seguenti metodi:

- *put(Object key, Object value)*: aggiunge un valore e lo associa con una chiave;
- *get(Object key)*: restituisce il valore corrispondente alla chiave;
- *containsKey(Object key)*, *containsValue(Object key)*: verifica se *Map* contiene un valore o una chiave.

La libreria standard di Java include due tipi differenti di *Map*: **HashMap** e **TreeMap**. Entrambe le classi hanno la stessa interfaccia ma si distinguono per efficienza.

- **HashMap** garantisce elevate prestazioni per la ricerca di una chiave perchè è un'implementazione basata sulla tabella hash. Infatti ogni oggetto Java può produrre un codice hash (di tipo *int*) attraverso il metodo *hashCode()* definito nella classe *Object*. *HashMap* utilizza tale codice per effettuare una ricerca efficiente sugli elementi.

Il metodo *HashCode()* usa per default l'indirizzo dell'oggetto per poter generare il valore intero. Può quindi accadere che oggetti differenti per indirizzo, ma identici per valore, siano indicizzati in modo diverso.

```

1 public class ObjectHashCode {
2     public static void main(String[] args) {
3         System.out.println((new Object()).hashCode());
4         System.out.println((new Object()).hashCode());
5     }
6 }

```

Per risolvere il problema si può sovrascrivere il metodo *hashCode()* per la classe di oggetti chiave da memorizzare in una Map. Questo è quello che avviene per *String* o *Integer*.

- **TreeMap.** Le coppie chiave-valore sono ordinate (attraverso i metodi dell'interfaccia *Comparable*). *TreeMap* dispone di un metodo *subMap()* che restituisce una porzione dell'albero costruito per mantenere ordinati gli oggetti.

Vediamo un semplice esempio di *HashMap*.

```

1 class Studente {
2     private int matricola;
3     private String cognome;
4     Studente(int matricola, String cognome) {
5         this.matricola = matricola;
6         this.cognome = cognome;
7     }
8
9     public boolean equals(Object o) {
10        return ((Studente) o).getMatricola() == matricola;
11    }
12
13    int getMatricola() { return matricola; }
14
15    public int hashCode() {
16        Integer i = new Integer(matricola);
17        return i.hashCode();
18    }
19
20    public String toString() { return cognome; }
21 }
22
23 class Counter {
24     private int = 1;
25     Counter(int i) { this.i = i; }
26     public String toString() {
27         return Integer.toString(i);

```

```

28     }
29 }
30
31 import java.util.*;
32
33 class Statistics {
34     public static void main(String[] args) {
35         HashMap hm = new HashMap();
36         hm.put(new Studente(1, "Bianchi"), new Counter(1));
37         hm.put(new Studente(2, "Rossi"), new Counter(2));
38         hm.put(new Studente(2, "Verde"), new Counter(3));
39         hm.put(new Studente(3, "Bianchi"), new Counter(4));
40     }
41 }

```

L'output di questo codice è il seguente:

{Bianchi=1, Rossi=3, Bianchi=4}

Con un TreeMap invece:

```

1 class Studente implements Comparable {
2     private int matricola;
3     private String cognome;
4     Studente(int matricola, String cognome) {
5         this.matricola = matricola;
6         this.cognome = cognome;
7     }
8
9     int getMatricola() { return matricola; }
10
11     public int compareTo(Object o) {
12         Studente s = (Studente) o;
13         Integer i = new Integer(s.matricola);
14         Integer m = new Integer(matricola);
15         return m.compareTo(i);
16     }
17
18     public String toString() { return cognome; }
19 }
20
21 class Counter {
22     private int i = 1;
23     Counter(int i) { this.i = i; }
24     public String toString() {
25         return Integer.toString(i);

```

```

26     }
27 }
28
29
30
31 import java.util.*;
32 class Statistics {
33     public static void main(String[] args) {
34         TreeMap tm = new TreeMap();
35         tm.put(new Studente(1, "Bianchi"), new Counter(1));
36         tm.put(new Studente(2, "Rossi"), new Counter(2));
37         tm.put(new Studente(2, "Verde"), new Counter(3));
38         tm.put(new Studente(3, "Bianchi"), new Counter(4));
39     }
40 }

```

4.19 RTTI - Run-Time Type Identification

Per comprendere il funzionamento di RTTI in Java bisogna capire come sono rappresentate al run-time le informazioni sul tipo, cioè sulla classe. Ciò è realizzato attraverso un tipo speciale di oggetto chiamato *Class Object* che contiene le informazioni sulla classe, per questo talvolta è chiamato meta-classe.

Le informazioni come attributi, metodi, modalità di accesso, etc. sono tutte presenti nel *.class*. Durante la compilazione viene generato un oggetto *Class* per ogni classe che costituisce il programma.

Gli oggetti di *Class* relativi alle varie classi che compongono un programma non sono caricati tutti in memoria prima di iniziare l'esecuzione. Quando al run-time si istanzia una classe, la *Java Virtual Machine*, su cui sta girando il programma, prima verifica se l'oggetto *Class* corrispondente è caricato. In caso negativo la JVM lo carica ricercando il file *.class* con quel nome.

Il metodo `forName()` è un metodo statico di *Class* che serve per ottenere un riferimento a un oggetto *Class*. Esso prende un oggetto di tipo *String* contenente il nome testuale della classe di cui si vuole il riferimento e restituisce un riferimento a *Class*.

Alternativamente, per ottenere un riferimento a un oggetto *Class* si può anche ricorrere al *lettera di class* (*class literal*), dato dal nome della classe seguito da *.class*. I vantaggi di questa notazione sono i seguenti.

- Semplicità.
- Efficienza.
- Controllo di esistenza della classe durante la compilazione.

Il letterale di classe funzione, oltre che con le classi, con gli array, con i tipi primitivi e con le interfacce. Per i “wrapper” dei tipi primitivi c’è anche un campo standard chiamato **TYPE**. Questo campo produce un riferimento all’oggetto *Class* per il tipo primitivo associato tale che si hanno le seguenti equivalenze.

boolean.class	Boolean.TYPE
char.class	Character.TYPE
byte.class	Byte.TYPE
short.class	Short.TYPE
int.class	Integer.TYPE
long.class	Long.TYPE
float.class	Float.TYPE
double.class	Double.TYPE
void.class	Void.TYPE

Le forme di RTTI viste finora, includono:

- il classico *cast* che usa RTTI per assicurarsi che il cast è corretto e solleva un’eccezione *ClassCastException* se è stato ottenuto un cast non corretto.
- l’oggetto *Class* rappresentante il tipo dell’oggetto.

L’oggetto *Class* può essere interrogato per ottenere utili informazioni al run-time.

In C++ il classico cast non compie una RTTI. Dice semplicemente al compilatore di trattare l'oggetto come di un altro tipo.

In Java, che esegue il controllo di tipo, questo tipo di cast è spesso chiamato “*type safe downcast*”.

Un'altra forma di RTTI in Java è ottenuta attraverso l'uso della parola chiave `instanceof` che indica se un oggetto è istanza di un particolare tipo e restituisce un *boolean*.

```
if (m instanceof Dog) ((Dog)m).bark();
```

L'istruzione precedente, verifica se l'oggetto *m* appartiene alla classe *Dog* prima di effettuare il casting, altrimenti si potrebbe sollevare una *ClassCastException*.

Segue un esempio di uso di `instanceof`.

```
1 class Pet {}
2 class Dog extends Pet {}
3 class Pug extends Dog {}
4 class Counter { int i; }
5
6 import java.util.*;
7 public class PetCount {
8     static String[] typenames = { "Pet", "Dog", "Pug" };
9
10    public static void main(String[] args) {
11        ArrayList pets = new ArrayList();
12        try {
13            Class[] petTypes = {
14                Class.forName("Dog"),
15                Class.forName("Pug"),
16            };
17
18            for (int i = 0; i < 5; i++) {
19                pets.add(
20                    petTypes[(int)(Math.random() * petTypes.length)].
21                    newInstance();
22                )
23            }
24        } catch (InstantiationException e) {
25            System.err.println("Cannot instantiate");
26            return;
27        } catch (IllegalAccessException e) {
28            System.err.println("Cannot access");
29        }
```

```

28         return;
29     } catch (ClassNotFoundException e) {
30         System.err.println("Cannot find class");
31         return;
32     }
33 }
34 ...
35
36 HashMap h = new HashMap();
37 for (int i = 0; i < typenames.length; i++)
38     h.put(typenames[i], new Counter());
39 for (int i = 0; i < pets.size(); i++) {
40     Object o = pets.get(i);
41     if (o instanceof Pet)
42         ((Counter) h.get("Pet")).i++;
43     if (o instanceof Dog)
44         ((Counter) h.get("Dog")).i++;
45     if (o instanceof Pug)
46         ((Counter) h.get("Pug")).i++;
47 }
48
49 for (int i = 0; i < pets.size(); i++)
50     System.out.println(pets.get(i).getClass());
51
52 for (int i = 0; i < typenames.length; i++)
53     System.out.println(typenames[i] + " quantity: " + ((Counter) h.get(
54         typenames[i])).i);
55 }

```

Quando si dispone di un oggetto, si può estrarre il riferimento all'oggetto *Class* relativo alla sua classe richiamando un metodo che è implementato in *Object*: *getClass()*.

Nel precedente esempio, alternativamente all'uso di *Class.forName* si possono usare i letterali *class*. Ad esempio: `Class[] petTypes = Dog.class, Pug.class`

In questo caso la creazione di *petTypes* non deve essere inclusa in un blocco `try`, perchè viene valutata al compile-time, diversamente dal metodo *Class.forName()*.

L'uso dell'operatore *instanceof* potrebbe risultare spesso molto noioso perchè lo si deve specificare per il confronto di ogni tipo di oggetto distinto. La classe *Class* mette a disposizione il metodo *isInstance* che fornisce un modo per invocare dinamicamente l'operatore *instanceof*. Ad esempio, il seguente codice:

```

1 Class[] petTypes = {Dog.class, Pug.class};
2
3 for (int i = 0; i < pets.size(); i++) {
4     Object o = pets.get(i);
5     if (o instanceof Pet)
6         ...
7     if (o instanceof Dog)
8         ...
9     ...
10 }

```

Diventa:

```

1 for (int i = 0; i < petTypes.length; i++) {
2     if (petTypes[i].isInstance(o)) {
3         String key = petTypes[i].toString();
4         ((Counter) h.get(key)).i++;
5     }
6 }

```